

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

ПРОГРАМНА ІНЖЕНЕРІЯ В БІОМЕДИЧНИХ ДОСЛІДЖЕННЯХ Практикум

Навчальний посібник

Рекомендовано Методичною радою КПІ ім. Ігоря Сікорського
як навчальний посібник для здобувачів ступеня бакалавра
за освітньою програмою «Медична інженерія»
спеціальності 163 Біомедична інженерія

Укладачі: В. В. Шликов, Н. О. Бритва

Електронне мережеве навчальне видання

Київ
КПІ ім. Ігоря Сікорського
2025

УДК 004.43:519.6
П43

Укладачі: *Шликов Владислав Валентинович, доцент, доктор техн. наук,
Бритва Надія Олександрівна, лаборант каф. біомедичної інженерії*

Рецензенти: *Алхімова С.М., к.т.н., доцент кафедри біомедичної кібернетики
ФБМІ КПІ ім. Ігоря Сікорського
Костін В.А., д.т.н., п.н.с. відділу фізико-хімічних досліджень
матеріалів №22 Інституту електрозварювання ім. Є.О. Патона*

Відповідальний
редактор *Соломін А. В. канд. фіз.-мат. наук, доцент*

*Гриф надано Методичною радою КПІ ім. Ігоря Сікорського
(протокол № 5 від 06.03.2025 р.)
за поданням вченої ради факультету біомедичної інженерії
(протокол № 10 від 24.02.2025 р.)*

П43 Програмна інженерія в біомедичних дослідженнях [Електронний ресурс]: практикум:
навч. посіб. для здобувачів ступеня бакалавра за освіт. програмою «Медична інженерія»
спец. 163 Біомедична інженерія / КПІ ім. Ігоря Сікорського ; уклад.: В. В. Шликов, Н.О.
Бритва. – Електрон. текст. дані (1 файл). – Київ : КПІ ім. Ігоря Сікорського, 2025. – 163 с.

У навчальному посібнику розглянуто основи розробки програмного забезпечення на мові Python для проведення наукових досліджень в галузі біомедичної інженерії. Зокрема, приділено увагу таким методам об'єктно-орієнтованого програмування: реалізація графічного інтерфейсу, функції роботи з матрицями та масивами, візуалізація даних з використанням двовимірної 2D і тривимірної 3D графіки, обробка сигналів і зображень, методи навчання нейронних мереж, функції для обчислення математичної статистики, реєстрація сигналів з використанням Arduino.

УДК 004.43:519.6

Реєстр. № НП 24/25-402. Обсяг 7,4 авт. арк.

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
проспект Берестейський, 37, м. Київ, 03056
<https://kpi.ua>

Свідоцтво про внесення до Державного реєстру видавців, виготовлювачів
і розповсюджувачів видавничої продукції ДК № 5354 від 25.05.2017 р.

© В. В. Шликов, Н. О. Бритва, 2025
© КПІ ім. Ігоря Сікорського, 2025

ЗМІСТ

ВСТУП	5
ПРАКТИЧНА РОБОТА № 1	6
1.1. Основні теоретичні відомості	6
1.2. Порядок роботи	20
1.3. Завдання для самостійної роботи	26
1.4. Контрольні питання	27
ПРАКТИЧНА РОБОТА № 2	28
2.1. Основні теоретичні відомості	28
2.2. Порядок роботи	43
2.3. Завдання для самостійної роботи	47
2.4. Контрольні питання	50
ПРАКТИЧНА РОБОТА № 3	51
3.1. Основні теоретичні відомості	51
3.2. Порядок роботи	65
3.3. Завдання для самостійної роботи	70
3.4. Контрольні питання	73
ПРАКТИЧНА РОБОТА № 4	74
4.1. Основні теоретичні відомості	74
4.2. Порядок роботи	84
4.3. Завдання для самостійної роботи	90
4.4. Контрольні питання	91
ПРАКТИЧНА РОБОТА № 5	92
5.1. Основні теоретичні відомості	92
5.2. Порядок роботи	101
5.3. Завдання для самостійної роботи	105
5.4. Контрольні питання	109
ПРАКТИЧНА РОБОТА № 6	110
6.1. Основні теоретичні відомості	110

6.2. Порядок роботи	115
6.3. Завдання для самостійної роботи	120
6.4. Контрольні питання	123
ПРАКТИЧНА РОБОТА № 7.....	124
7.1. Основні теоретичні відомості	124
7.2. Порядок роботи	129
7.3. Завдання для самостійної роботи	136
7.4. Контрольні питання	140
ПРАКТИЧНА РОБОТА № 8.....	141
8.1. Основні теоретичні відомості	141
8.2. Порядок роботи	147
8.3. Завдання для самостійної роботи	155
8.4. Контрольні питання	156
ОФОРМЛЕННЯ І ОЦІНЮВАННЯ ПРАКТИЧНИХ РОБІТ	157
9.1. Звіт з практичної роботи	157
9.2. Оцінювання практичних робіт.....	159
ПЕРЕЛІК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ.....	160

ВСТУП

Основною метою навчальної дисципліни «Програмна інженерія в біомедичних дослідженнях» є формування у студентів здатності розв’язувати спеціалізовані задачі й практичні проблеми програмної інженерії в біомедичних дослідженнях, що передбачає застосування програмних засобів для реалізації методів статистики, фільтрації сигналів, обробки зображень, проектування нейронних мереж, взаємодії прикладних програм з мікроконтролерами на основі спеціалізованих бібліотек на мові програмування Python.

Навчальна дисципліна «Програмна інженерія в біомедичних дослідженнях» вивчає застосування програмних засобів для обробки результатів біомедичних досліджень шляхом реалізації спеціальних методів статистики, фільтрації сигналів, обробки зображень, проектування нейронних мереж, взаємодії прикладних програм з мікроконтролерами.

Метою практичних занять є набуття студентами умінь та практичних навичок використання програмних засобів реалізації методів обробки даних (спеціалізовані бібліотеки Python), програмних засобів створення прикладних програм (середовище PyCharm, мова Python), програмних засобів взаємодії програм на Python з мікроконтролерами Arduino UNO.

Практичні заняття мають дослідницький характер, у кожному з них стисло викладені основні теоретичні положення, методика і порядок виконання роботи та завдання для самостійного написання та виконання програмного коду на мові Python. Під час кожного практичного заняття передбачається реалізація простих GUI-додатків для побудови графіків і складних математичних функції, що входять до складу спеціалізованих бібліотек для обробки даних: Tkinter (GUI), Matplotlib, NumPy, SciPy, SciPy.Stats (Statistics), Cv2 (OpenCV), Sklearn (Machine Learning), PyFirmata (Arduino Controllers).

ПРАКТИЧНА РОБОТА № 1

ВИКОРИСТАННЯ КОНТРОЛЕРУ ARDUINO ТА БІБЛІОТЕКИ PYFIRMATA

Мета заняття: набути навичок роботи з контролером Arduino із застосуванням функцій бібліотек PyFirmata і pySerial використовуючи мову програмування Python.

1.1. Основні теоретичні відомості

Arduino є торговою маркою апаратно-програмних засобів, під якою випускається кілька плат з мікроконтролером і плат розширення. Більшість плат із контролером Arduino забезпечено мінімально необхідним набором обв'язки для нормальної роботи мікроконтролера (стабілізатор живлення, кварцовий резонатор, ланцюжки скидання тощо) та створення прототипів простих систем, моделей та експериментів у галузі електроніки, автоматики, приладобудування та робототехніки (рис. 1.1) [12].

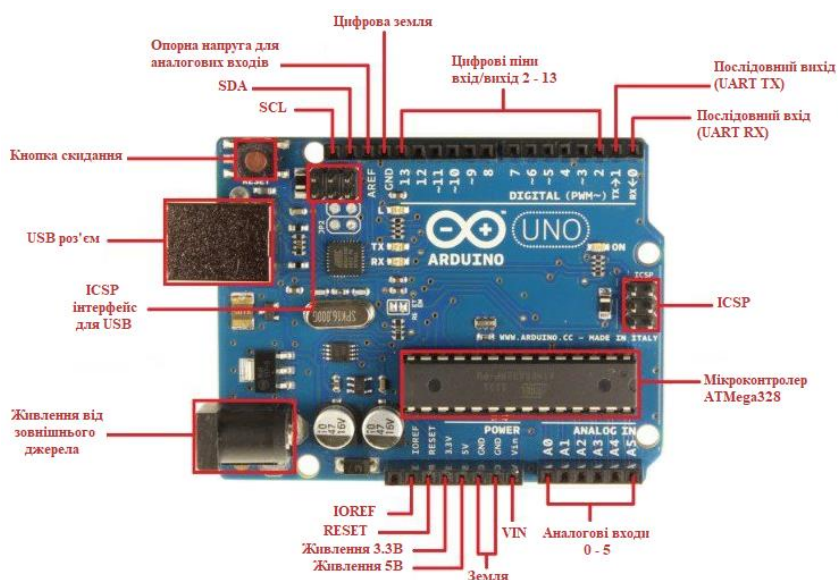
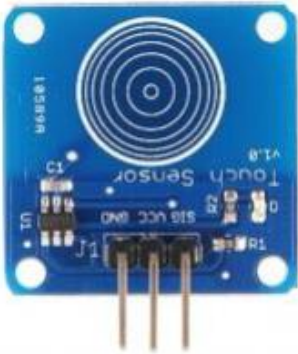


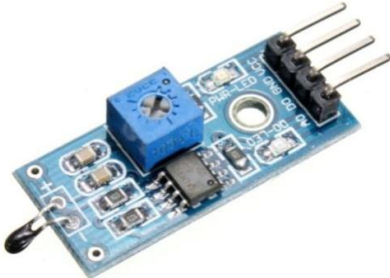
Рисунок 1.1 – Плата Arduino UNO [20]

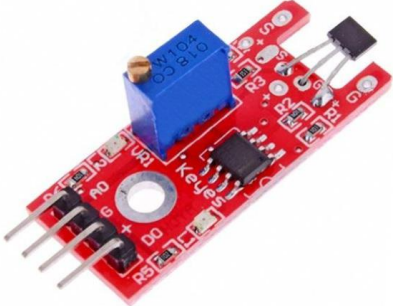
У апаратну частину Arduino не входить корпусний або монтажний конструктив. Розробник вибирає спосіб встановлення та механічного захисту

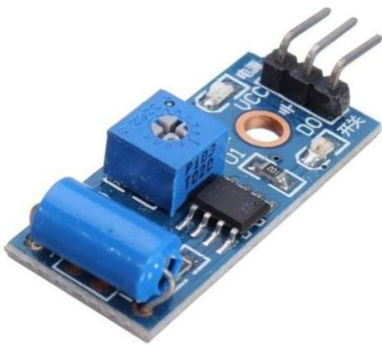
плат самостійно або за допомогою сторонніх компаній. Сторонніми виробниками випускаються набори робототехнічної електромеханіки, орієнтованої працювати разом із платами Arduino. Незалежними виробниками також випускається велика гама всіляких датчиків та виконавчих пристроїв, що сумісні з Arduino (табл. 5.1) [19].

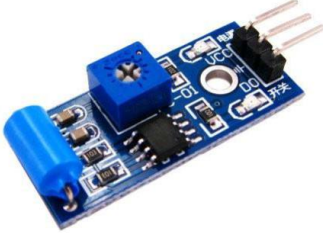
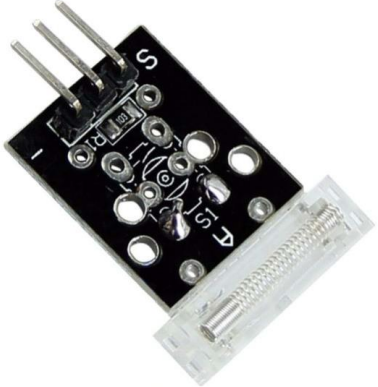
Таблиця 5.1 – Датчики та виконавчі пристрої сумісні з Arduino [21]

Датчики та виконавчі пристрої	Основні характеристики пристрою
	Мікросхема ємнісного датчика торкання TTP223B: – діапазон напруги живлення: від 2 до 5.5 В; – інтерфейси: GND - загальний, VCC - живлення, SIG (DI) - вихід; – індикатор живлення: зелений світлодіод; – область сенсора: позначена область на платі; – розмір плати: 24×24 мм.
	Універсальний звуковий датчик на LM393: – аналоговий вихід напруги з мікрофона; – цифровий вихід порогового компаратора; – компаратор напруги: LM393; – індикатор живлення; – робоча напруга: 4-6В; – розміри плати: 32×17×8 мм.
	Датчик швидкості на мікросхемі LM393: – напруга живлення: від 3,6 до 5В; – тип виходу: аналоговий і цифровий; – поточний компаратор: LM393; – виводи модуля: VCC - живлення +5В, GND - загальна земля, DO - цифровий вихід, AO - аналоговий вихід.

Датчики та виконавчі пристрої	Основні характеристики пристрою
	П'ятивольтовий кроковий двигун 28BYJ-48 5В: – напруга живлення: 5 В (постійний струм); – кількість фаз: 4, номінальна частота: 100 Гц; – кількість кроків: 64, крок: 5.625 град.; – крутний момент: 34,3 Н/м; – номінальна тяга: 3500 г/см.
	Датчик вологості і температури HTU21: – робочий діапазон: 0 до 100% RH; – відносна точність датчика вологості: $\pm 3\%$ RH (макс.) у діапазоні 0 - 80% RH; – точність датчика температури: $\pm 0.4^{\circ}\text{C}$ (макс) у діапазоні від -10 до 85°C; – робочий діапазон температур: -40 до $+125^{\circ}\text{C}$; – діапазон робочої напруги датчика: 1.9 - 3.6 В; – напруга живлення: 5 - 6В, струм: 150 мкА; – інтерфейс: I2C, адреса I2C пристрої: 0x40; – нагрівач: інтегрований на платі модуля.
	Цифровим модулем з термістором NTC: – тип датчика: NTC термістор; – діапазон вимірюваної температури: $20 - 80^{\circ}\text{C}$; – робоча напруга: 3 - 5 В; – макс. вихідний струм: 15 мА; – цифровий компаратор: LM393; – поріг спрацьовування виходу: налаштовується; – розмір: 3.2×1.4 см.

Датчики та виконавчі пристрої	Основні характеристики пристрою
	Ультразвуковий датчик відстані HC-SR04: – тип сенсору: HC-SR04; – робоча напруга: 3.8 - 5.5 В, струм: 8 мА; – частота: 40 кГц, ширина імпульсів: 10 мкс; – дистанція: мінімальна 0 см, максимальна 15 см; – роздільна здатність: 3 мм; – кут: 15 градусів; – зовнішні габарити: 37×20×15 мм.
	Цифровий датчик перешкоди на LM393: – відстань виявлення перешкоди: від 2 до 30 см; – робоча напруга від 3.3 до 5 В; – компаратор напруги: LM393; – виводи датчика: 3.3 - 5 В VCC - живлення, GND - загальна земля, OUT - цифровий вихід; – розміри модуля: 3.2×1.4 см.
	Магнітний датчик Хола KY-024: – тип: датчик Хола (SS49E); – подвійний компаратор: LM393; – живлення: 3,3 - 5 В, робоча напруга: DC 5 В; – чутливість: регульована потенціометром; – виводи: A0 - аналоговий вихід, D0 - цифровий вихід, G - GND (загальна земля); – розмір: 32×11×20 мм.

Датчики та виконавчі пристрої	Основні характеристики пристрою
	Датчик нахилу на мікросхемі LM393: – компаратор: LM393; – напруга живлення: 3.3 - 5 В; – індикатор виходу: світлодіодний; – вихідний сигнал: цифровий; – чутливість: регульована резистором; – розміри плати: 30×15 мм.
	Цифровий датчик алкоголю MQ-3: – напруга живлення: 5 В; – струм: 150 мА; – діапазон вимірювання: 0,05 мг/л - 10 мг/л; – чутливість: налаштовується резистором; – розміри плати: 25,4×25,4 мм.
	Цифровий датчик рівня рідин HASL FR4: – тип датчика: аналоговий; – робоча напруга: DC3-5V, струм: менше 20 мА; – площа виявлення: 40×16 мм; – робоча температура: 10 - 30°C; – вологість: 10% - 90% без конденсації; – розмір: 62×20×8 мм, вага: 3,5 г.
	Цифровий датчик вогню на LM393: – робоча напруга 3.3 – 5 В; – довжина хвилі: 760 - 1100 нм; – відстань: до 80 см; – кут спрацьовування: до 60 град.; – чутливість: регулюється потенціометром; – розміри: 47.2×14×7.3 мм, вага: 2.1 г.

Датчики та виконавчі пристрої	Основні характеристики пристрою
	Датчик освітленості на мікросхемі LM393: – чутливий елемент: фоторезистор; – робоча напруга: від 3.3 В до 5 В, струм >15 мА; – компаратор LM393, – регулювання порога спрацьовування: тример; – вихід компаратора: цифровий - 0 чи 1; – вихід датчика освітленості: аналоговий; – розміри: 3.2×1.4 см.
	Датчик вібрації SW-420: – напруга живлення: від 3.3 до 5 В; – компаратор: LM393; – вихід: цифровий; – чутливість: регульована резистором; – розміри плати: 3.2×1.4 см.
	Цифровий датчик удару KY-031: – напруга живлення: 5 В; – виводи: 2, VCC - живлення, DATA - цифровий, GND - загальна земля; – вихід: цифровий - 0 чи 1; – розміри плати: 3.2×1.4 см.
	Датчик згину Sparkfun: – тип датчика: тензорезистор; – напруга живлення: 5 В; – опір: 2.5 кОм; – вихід: аналоговий; – розміри плати: 5.6 см.

Програмування мікроконтролеру Arduino ведеться через свою безкоштовну програмну оболонку Arduino IDE, розроблену компанією Arduino Software. У цій оболонці є текстовий редактор, менеджер проектів, препроцесор, компілятор та інструменти для завантаження програми у мікроконтролер (<https://docs.arduino.cc/software/ide>).

Мова програмування Arduino називається Arduino C і є мовою C++ з фреймворком Wiring, яка має деякі відмінності в частині написання програмного коду, що компілюється і збирається за допомогою avr-gcc, з особливостями, що полегшують написання програми завдяки набору бібліотек, які включають функції та об'єкти. При компіляції IDE створює тимчасовий файл з розширенням *.cpp.

Середовище Arduino IDE не пропонує жодних налаштувань компілятора та мінімізує інші налаштування, що спрощує програмування та зменшує ризик виникнення проблем, але підтримує директиви препроцесора, такі як #define, #include та багато інших. Програміст повинен написати дві обов'язкові для Arduino функції setup() та loop(). Перша викликається одноразово під час старту, а друга виконується у нескінченному циклі.

Окрім операторів setup() і loop() при написанні коду можуть використовуватись оператори мови C++ [1]:

- керуючі оператори (if, else, while, for, return, go to та інші) ;
- арифметичні оператори (+, *, / та інші);
- логічні оператори (&&, ||, !);
- бінарні оператори (++ , -- та інші)

В основному використовуються типи даних аналогічні до мови C++, але окремо додано спеціальні константи такі як HIGH, LOW, INPUT, OUTPUT та інші.

Більш детальний опис цих і інших складових коду можна знайти за посиланням на сайті «Інформаційний ресурс про електроніку та програмування» <https://itmaster.biz.ua/electronics/arduino.html>.

Мікроконтролер Arduino дає можливість знімати дані відразу з декількох сенсорів одночасно та керувати кроковим двигуном і мотором постійного струму. Наприклад, до контролеру Arduino UNO можна підключити декілька давачів лінії KY-033 з компаратором на мікросхемі LM393 та інфрачервоним сесором TCRT5000 та DC-двигун (прямий) Readytosky 2212 920KV CW (рис. 1.2).

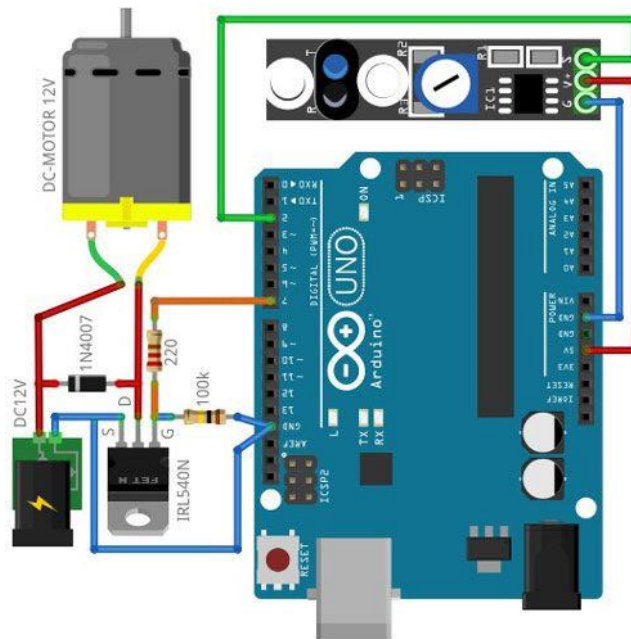


Рисунок 1.2 – Схема підключення декількох приладів до Arduino UNO [22]

Таку схему підключення (рис. 1.2) можна застосувати для мобільного робота, який повинен рухатись по заданій траєкторії з регульованою швидкістю за рахунок керування обертами двигуна Readytosky 2212 та відстежування напрямку руху по лінії за рахунок реєстрації перешкод давачем лінії KY-033. Щоб змусити робота рухатися лінією достатньо двох датчиків лінії KY-033, які встановлюються один від одного на відстані, що дорівнює ширині смуги маршруту. Програма постійно опитує стан датчиків лінії та залежно від показань видає живлення на DC-двигуни правого чи лівого колеса. Подібні задачі одночасного опитування декількох давачів і регулювання обертів двигунів вирішуються під час керування протезами верхніх та нижніх кінцівок.

Для застосування Python в практичних електронних схемах з використанням Arduino необхідно виконати наступні дії:

1. Завантажити та встановити середовище програмування Arduino IDE

(<https://www.arduino.cc/en/software>):

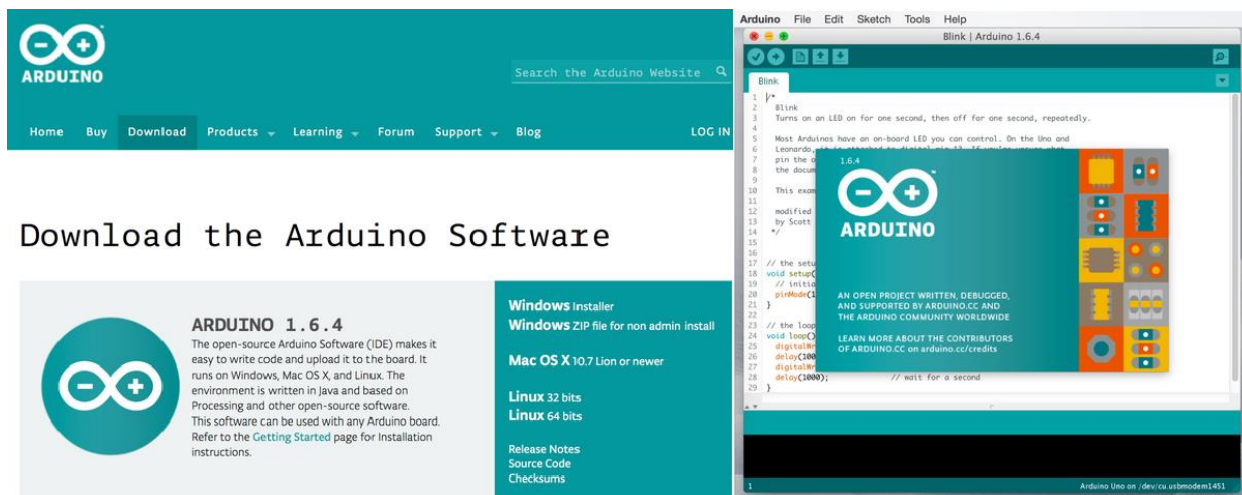


Рисунок 1.3 – Встановлення середовища Arduino IDE

Після встановлення запустіть програму Arduino та дочекайтесь появи екрана-заставки, у текстовому редакторі має відкритися програма за замовчуванням під назвою Sketch (рис. 1.3):

```
void setup() {
  // put your setup code here, to run once:
}
void loop() {
  // put your main code here, to run repeatedly:
}
```

2. Підключити мікроконтролер Arduino до USB порту комп'ютера:



Рисунок 1.4 – Підключення контролеру Arduino UNO до USB порту

Від'єднайте всі дроти, які можуть бути підключені до Arduino і підключіть плату до комп'ютера (рис. 1.4). Залежно від версії Arduino, кабель USB (Type-A) підключатиметься безпосередньо до плати контролера за допомогою стандартного мікро-USB або міні-USB штекера (<https://docs.arduino.cc/hardware/uno-rev3>). Наприклад, на платі контролера Arduino UNO rev3 є порт **USB Plug** (рис. 1.1).

3. Обрати послідовний порт і версію мікроконтролера Arduino:

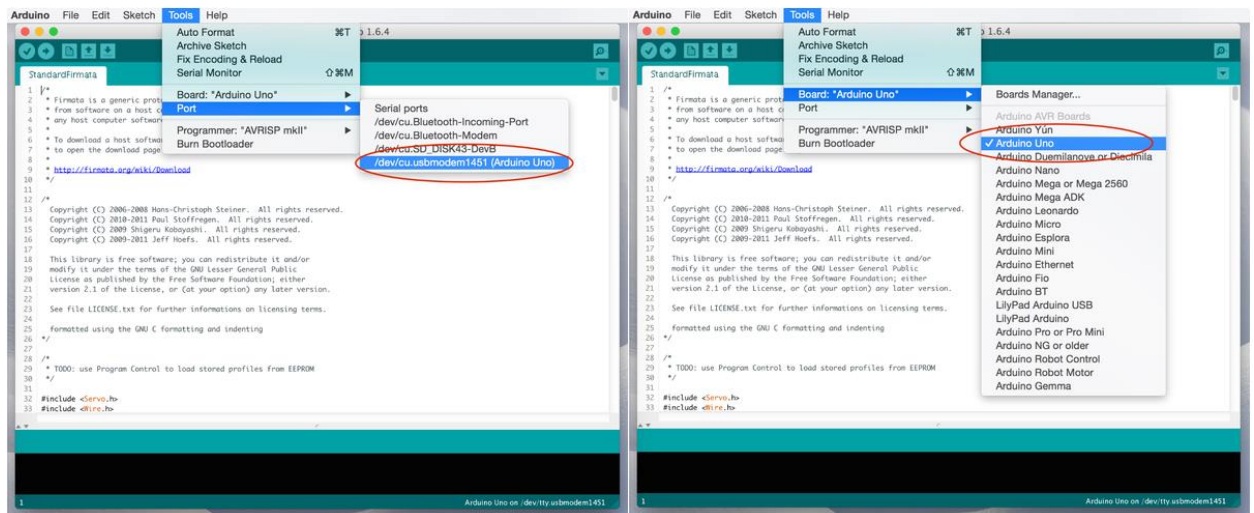


Рисунок 1.5 – Налаштування підключення контролера в Arduino IDE

Після підключення підключення плати Arduino до комп'ютера, потрібно повідомити програмі Arduino IDE, як знайти плату та який маємо тип плати (рис. 1.5).

Спочатку в середовищі програмування Arduino IDE перейдіть до пункту меню Tools → Port, у якому буде список доступних портів USB. Якщо плата Arduino встановлена вірно, один із портів має мати номер послідовного 'COM' порту, а потім назву плати. Оберіть цей пункт у меню Port, щоб вибрати правильний послідовний USB порт.

Детально з інструкцією встановлення можна ознайомитись на сайті «Docs.Arduino.cc» <https://docs.arduino.cc/software/ide-v2/tutorials/ide-v2-board-manager>.

4. Завантажити стандартний скетч Firmata для використання Python

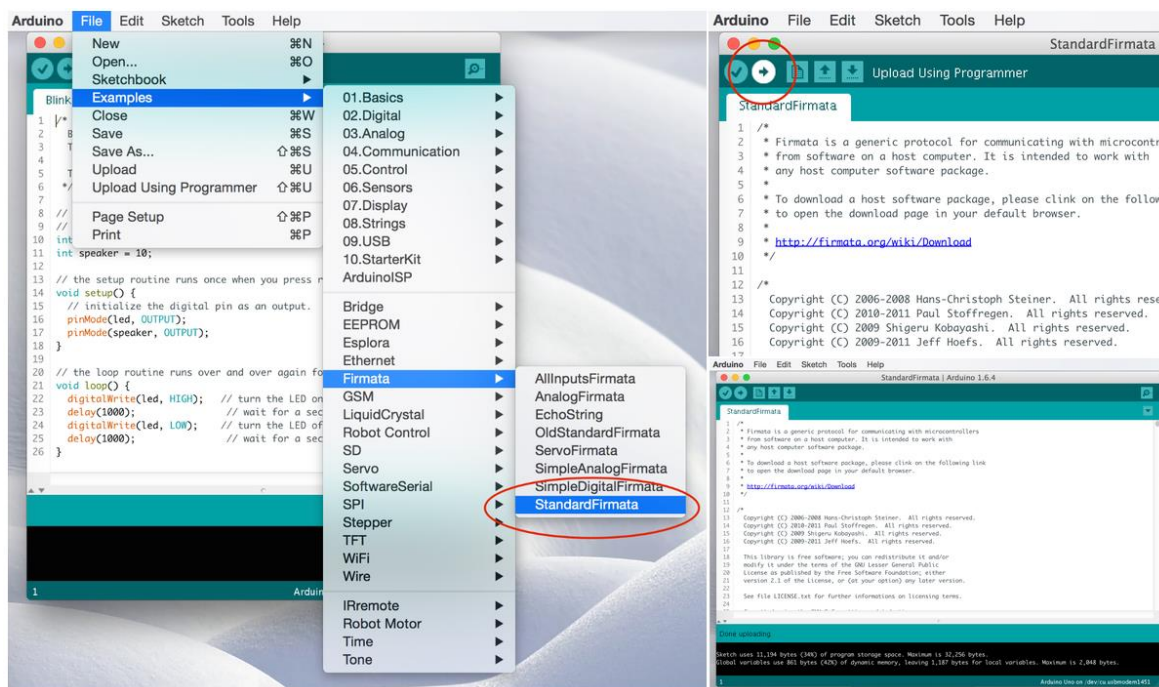


Рисунок 1.6 – Завантаження стандартного скетчу Firmata

У Arduino IDE за допомогою меню File → Examples → Firmata → Standard Firmata відкрийте стандартний скетч Firmata (рис. 1.6). Відкриється програма на мові C++ для Arduino, яка використовує протокол Firmata для зв'язку з мікроконтролером Arduino (<https://doc.arduino.ua/ru/prog/Firmata>). Щоб використовувати бібліотеку PyFirmata на Python не потрібно розуміти фактичний код на C++.

5. Завантажити скетч Standard Firmata у мікроконтролер Arduino

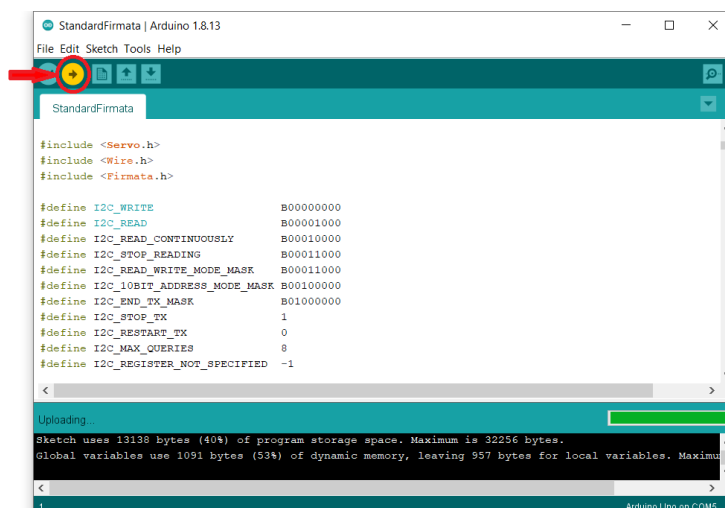


Рисунок 1.7 – Завантаження скетчу Standard Firmata у мікроконтролер

У Arduino IDE за допомогою меню Sketch → Upload завантажте скетч Standard Firmata у мікроконтролер Arduino (рис. 1.7).

6. Зробити налаштування у програмі PyCharm програмування на Python

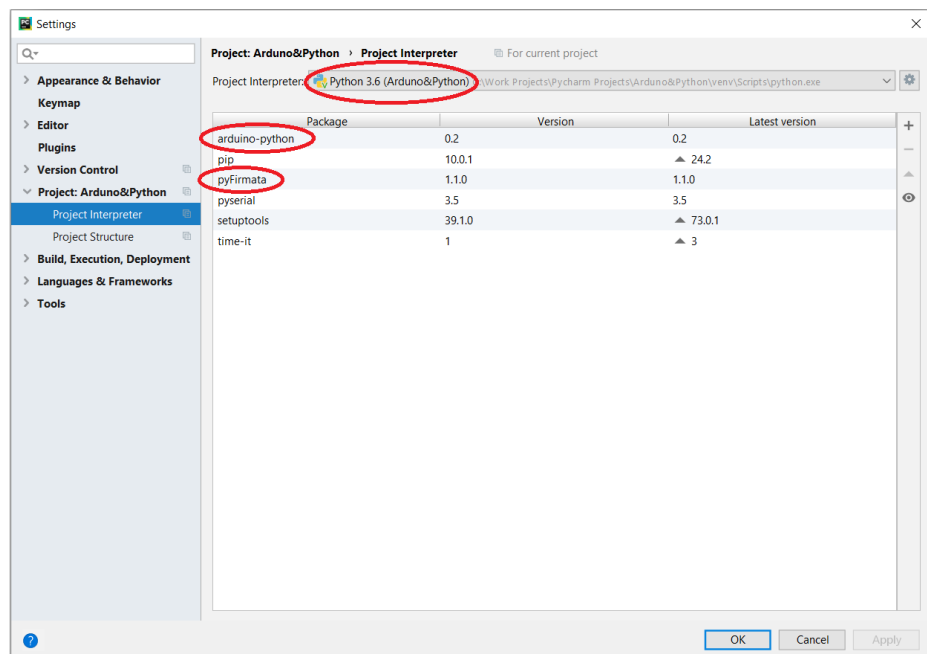


Рисунок 1.8 – Налаштування у програмі PyCharm

Для проектів на Python у середовищі програмування PyCharm необхідно обрати компілятор Python 3.6 та додати бібліотеки підтримки контролера Arduino: PyFirmata, і Arduino-Python або pySerial (рис. 1.8).

У бібліотеці PyFirmata реалізовано протокол Firmata, що призначено для зв'язку мікроконтролера Arduino з програмою на Python. Використання бібліотек PyFirmata і Arduino-Python звільняє від необхідності написання власних протоколів та об'єктів для взаємодії контролера Arduino та програмного забезпечення.

Бібліотека pySerial містить важливі компоненти, які необхідні для зв'язку Arduino і Python. Ця бібліотека забезпечує зв'язок з Arduino шляхом інкапсуляції доступу до послідовних портів. Модуль pySerial забезпечує доступ до параметрів послідовного порту через функції Python і дозволяє

безпосередньо налаштувати послідовний порт. Функції бібліотеки pySerial є мостом для передачі даних між контролером Arduino і Python [13].

Щоб використовувати бібліотеку pySerial у програмі PyCharm потрібно перейти у меню Sketch → "+", знайти у вікні Available Packages пакет pySerial і встановити його натиснувши кнопку Install Packages (рис. 1.9).

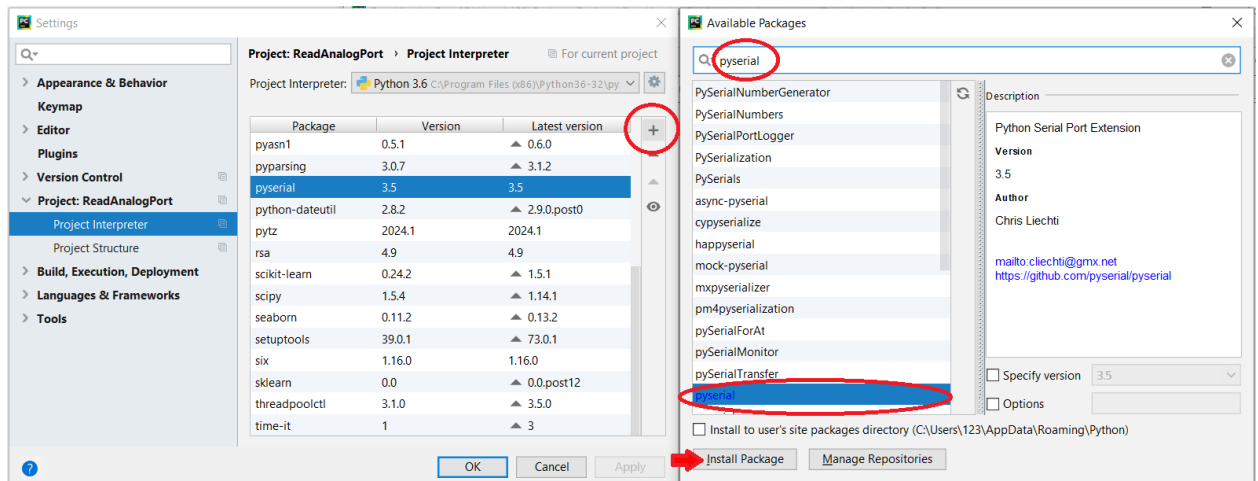


Рисунок 1.9 – Встановлення бібліотеки pySerial у програмі PyCharm

Щоб перевірити роботу бібліотеки pySerial використаємо простий скетч на Arduino, що реалізує зв'язок через послідовний порт, який можна захопити на інтерпретаторі Python. У Arduino IDE за допомогою меню File → Examples → 01.Basic → DigitalReadSerial завантажить потрібний скетч DigitalReadSerial.ino у мікроконтролер Arduino (рис. 1.10).

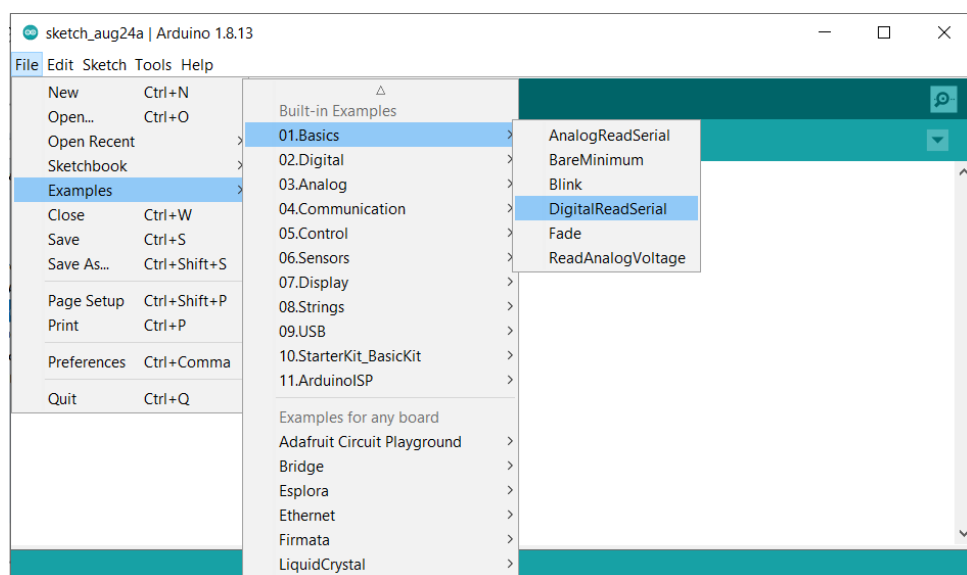


Рисунок 1.10 – Завантаження скетчу DigitalReadSerial.ino у контролер Arduino

Скомпілюйте та завантажте програму DigitalReadSerial.ino у пам'ять мікроконтролера Arduino (рис. 1.7). У скетчі на послідовний порт передається статус цифрового виводу *pin2*, через який передвються дані зі швидкістю обміну бітами 9600 біт/с.

Програмний код у Python, в якому опитується давач на виводі *pin2* через послідовний порт 'COM5' має вигляд:

```
import serial.tools.list_ports
from datetime import datetime
from openpyxl import Workbook
import time

if __name__ == '__main__':
    # Получаем список доступных Serial портов
    ports = list(serial.tools.list_ports.comports())

    # Выводим информацию о каждом порте
    for port in ports:
        print(f"Порт: {port.device}")
        print(f"Описание: {port.description}")
        print(f"Производитель: {port.manufacturer}\n")

    # Создаем новый файл Excel
    workData = Workbook()
    sheet = workData.active
    # Записываем заголовок
    sheet.append(["ID записи", "Время", "Контакт (у.е.)"])
    id_record = 1 # Инициализируем ID записи

    while True:
        # Открываем Serial порт ('COMX' замените на имя вашего порта)
        ser = serial.Serial('COM5', 9600)

        # Отправляем строку "Hello, Arduino!" на Arduino, предварительно
        # преобразовав ее в байты
        ser.write(b'Hello, Arduino!')
        # Читаем ответ от Arduino через Serial порт
        response = ser.readline()
        # Декодируем ответ из байтов в строку с использованием UTF-8
        decoded_response = response.decode('utf-8')
        time.sleep(0.01)

        # Получаем текущее время
        current_time = datetime.now().strftime('%H:%M:%S')
        # Читаем ответ от Arduino через Serial порт
        response = ser.readline()
        # Декодируем ответ из байтов в строку с использованием UTF-8
        decoded_data = response.decode('utf-8').strip()
        time.sleep(0.01)

        # Записываем данные в ячейки
        sheet.append([id_record, current_time, float(decoded_data)])
        # Увеличиваем ID
        id_record += 1
```

```
# Сохраняем файл
workData.save('data.xlsx')

# Закрываем порт
ser.close()
print(decoded_response)
```

Дані з сенсору на виводі *pin2*, що зчитані функцією `ser.readline()` із бібліотеки `pySerial` записуються у файл `'data.xlsx'` з часовми міками, які отримуються за допомогою функції `datetime.now().strftime()` з бібліотеки `Time`. Результат вконання коду на Python також виводиться у вікно терміналу програми `PyCharm`:

```
Порт: COM5
Описание: Arduino Uno (COM5)
Производитель: Arduino Srl (www.arduino.org)
1
1
11
0 ...
```

Аналогічно, є можливість перевірити роботу бібліотеки `pyFirmata`, якщо використати простий скетч на Arduino, що можна завантажити до мікроконтролера у Arduino IDE за допомогою меню `File → Examples → Firmata → StandatdFirmata.ino`.

1.2. Порядок роботи

Завантажити на контролер Arduino код на мові C++ з підтримкою бібліотеки `Firmata.h` та `Wire.h`, який знаходиться у середовищі програмування Arduino IDE: `File → Examples → Firmata → StandatdFirmata.ino`. Скорочений приклад скетчу для контролера Arduino, який підтримує надсилання та отримання повідомлень через цифровий і аналоговий порти з використанням протоколу `Firmata`, має вигляд:

```
#include <Wire.h>
#include <Firmata.h>

byte analogPin, digitalPin;

void analogWriteCallback(byte pin, int value)
{
    pinMode(pin, OUTPUT);
    analogWrite(pin, value);
}
```

```

void digitalWriteCallback(byte port, int value)
{
    byte pin, lastPin, pinValue, mask;

    mask = 1;
    lastPin = port * 8 + 8;
    pinValue = ((byte)value & mask) ? 1 : 0;

    for (pin = port * 8; pin < lastPin; pin++) {
        pinMode(pin, INPUT_PULLUP);
        Firmata.setPinState(pin, pinValue);
    }
}

/* utility functions */
void wireWrite(byte data)
{
    #if ARDUINO >= 100
        Wire.write((byte)data);
    #else
        Wire.send(data);
    #endif
}

byte wireRead(void)
{
    #if ARDUINO >= 100
        return Wire.read();
    #else
        return Wire.receive();
    #endif
}

void setup()
{
    Firmata.setFirmwareVersion(0, 1);
    Firmata.attach(ANALOG_MESSAGE, analogWriteCallback);
    Firmata.begin();
}

void loop() {
    // put your main code here, to run repeatedly:

    while(Firmata.available()) {
        Firmata.processInput();
    }
    for(analogPin = 0; analogPin < TOTAL_ANALOG_PINS; analogPin++) {
        Firmata.sendAnalog(analogPin, analogRead(analogPin));
    }
    for(digitalPin = 0; digitalPin < TOTAL_PINS; digitalPin++) {
        Firmata.sendDigital(digitalPin, digitalRead(digitalPin));
    }
}

```

Частина 1. Читання даних з цифрового виводу Arduino.

Послідовний порт підключення контролера Arduino на мові Python вказується у функції `Arduino('COM5')`, а підключення сенсора на цифровий порт `'d:10:i'` та `'d:13:o'` вказується у функції `board.get_pin()` з бібліотеки `PyFirmata`:

```

# Connect to serial port COM5
board = pyfirmata.Arduino('COM5')
print("Communication successfully started")

```

```

# Start iterator to receive input data
it = pyfirmata.util.Iterator(board)
it.start()

# Sensor on digital port D:10 and D:13
digital_input = board.get_pin('d:10:i')
led = board.get_pin('d:13:o')

sw = digital_input.read()
if sw is True:
    led.write(1)
else:
    led.write(0)

```

Функція `digital_input.read()` читає дані з цифрового порту 'd:10:i', що вставлено у режим на введення даних ':i', а функція `led.write(1)` вмикає чи `led.write(0)` вимикає світлодіод, який знаходиться на цифровому порту 'd:13:o', що вставлено на виведення даних ':o'.

У підсумку код читання даних з цифрового виводу Arduino має вигляд:

```

import pyfirmata
import time

if __name__ == '__main__':
    board = pyfirmata.Arduino('COM5')
    print("Communication successfully started")

# Start iterator to receive input data
it = pyfirmata.util.Iterator(board)
it.start()
print("Successfully started iterator to receive input data")

potentiometer = board.analog[0]
potentiometer.enable_reporting()

digital_input = board.get_pin('d:10:i')
led = board.get_pin('d:13:o')

while True:
    print(potentiometer.read())
    time.sleep(0.1)

    sw = digital_input.read()
    if sw is True:
        led.write(1)
    else:
        led.write(0)
    time.sleep(0.1)

```

Вивід даних у вікні терміналу PyCharm:

```

Communication successfully started
Successfully started iterator to receive input data

```

```
0.0
0.219
28.2463
28.0274
28.0274
0.0
```

Частина 2. Читання даних з аналогового виводу Arduino.

Послідовний порт підключення контролера Arduino на мові Python вказується у функції `Arduino('COM5')`, а підключення сенсора на аналоговому порту `A0` вказується у функції `board.analog[0]` з бібліотеки `PyFirmata`:

```
# Connect to serial port COM5
board = pyfirmata.Arduino('COM5')
print("Communication successfully started")

# Start iterator to receive input data
it = pyfirmata.util.Iterator(board)
it.start()

# Sensor on analog port A0
potentiometer = board.analog[0]
potentiometer.enable_reporting()
print(potentiometer.read())
```

Функція `potentiometer.read()` буде читати дані з аналогового порту та виводити результат у вікно терміналу за допомогою функції `print()`.

У підсумку програмний код читання даних з аналогового виводу контролера Arduino:

```
import PySimpleGUI as sg
import os.path
import pyfirmata
import time

def goto(linenum):
    global label
    label = linenum

left_col = [
    [sg.Listbox(values=[], enable_events=True,
size=(40,20),key='-FILE LIST-')],
    [sg.Button('Plot', size=(10, 2), pad=((180, 0), 20),
font='Helvetica 10')],
    [sg.Button('Stop', size=(10, 2), pad=((180, 0), 10),
font='Helvetica 10', key = ("OFFGENERATE"))]
]
layout = [[sg.Column(left_col, element_justification='c')]]

window = sg.Window('Multiple Format Image Viewer',
layout,resizable=True)
```



```

# ----- Event Loop -----
folder = os.getcwd()
file_list = os.listdir(folder) # get list of files in folder

try:

    if __name__ == '__main__':
        board = pyfirmata.Arduino('COM5')
        print("Communication Successfully started")

        # Create bunch of useful variables
        button = board.digital[5]
        potentiometer1 = board.analog[1]
        potentiometer2 = board.analog[2]
        potentiometer3 = board.analog[3]
        potentiometer4 = board.analog[4]

        # Create enable of reporting
        potentiometer1.enable_reporting()
        potentiometer2.enable_reporting()
        potentiometer3.enable_reporting()
        potentiometer4.enable_reporting()

        potentiometer_s = [potentiometer1, potentiometer2,
potentiometer3, potentiometer4]
        meter_index = 0
        previous_button_state = 0

        # Start iterator to receive input data
        it = pyfirmata.util.Iterator(board)
        it.start()
        print("Successfully started iterator to receive input data")

        label = 1
        print("Ready to receive input data")
        event, values = window.read()
        while True:

            print("Set point input")
            if label == 1:

                # Folder name was filled in, make a list of files in the folder
                if event == 'Plot':
                    filename = os.path.join("Python.jpg")
                    print("Successfully read file name")

                    # Setup button
                    button.mode = pyfirmata.INPUT
                    print("Successfully setup buttona")

                    # Read reporting of potentiometers
                    for potentiometer in potentiometer_s:
                        potentiometer.read()
                    print("Successfully setup potentiometers")

                    meter_step = 0
                    for potentiometer in potentiometer_s:
                        # We run the loop at 100Hz
                        meter_step += 1
                        time.sleep(0.01)

                    # Get button current state
                    button_state = button.read()

```

```

print("Successfully get button current state")

if button_state != previous_button_state:
    if button_state == 0:
        print("Button released")
        # Power off current LED
        # and power on next LED
        potentiometer_s[meter_index].write(0)
        meter_index += 1
        print("Successfully power off current LEDs")

        time.sleep(0.1)
        fnames = [f for f in file_list if os.path.isfile(
            os.path.join(folder, f)) and
            f.lower().endswith((".png", ".jpg", "jpeg", ".tiff", ".bmp"))]
        window['-FILE LIST-'].update(fnames)
    else:
        potentiometer_s[meter_index].read()
        meter_index += 1
        print("Successfully power on current LEDs")

        # Save current button state as previous
        # for the next loop iteration
        if meter_index >= len(potentiometer_s):
            meter_index = 0
        previous_button_state = button_state

        time.sleep(0.1)
        fnames = [f for f in file_list if os.path.isfile(
            os.path.join(folder, f)) and
            f.lower().endswith((".png", ".jpg", "jpeg", ".tiff", ".bmp"))]
        print("Successfully update path to files")

        window['-FILE LIST-'].update(fnames)

        if meter_step >= len(potentiometer_s):
            print("LEDs released")
            label = 0
            continue

        window['Plot'].update(filename)

    if label == 0:
        print("Loop released")
        event, values = window.read()
        goto(1)

    if event == "OFFGENERATE": # Folder name was filled in, make a
        list of files in the folder
        print("Successfully stopped process")
        break
    elif event == sg.WIN_CLOSED:
        print("Successfully closed process")
        break

    if event in (sg.WIN_CLOSED, 'Exit'):
        break

    if event == sg.WIN_CLOSED or event == 'Exit':
        break

except:

```

```

        folder = '.'
        file_list = os.listdir(folder)
        fnames = [f for f in file_list if os.path.isfile(
os.path.join(folder, f)) ]
        window['-FILE LIST-'].update(fnames)
        print("Successfully except processing")

# ----- Close & Exit -----
window.close()

```

Вивід даних у графічному вікні PyCharm бібліотеки PySimpleGUI (рис. 1.1):

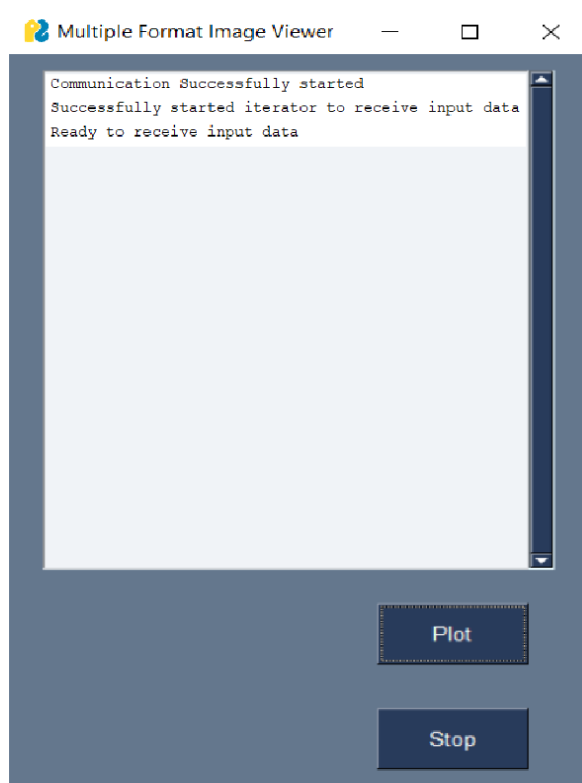


Рисунок 1.11 – Вивід даних у графічному вікні PyCharm

1.3. Завдання для самостійної роботи

Завантажити на контролер Arduino код з підтримкою бібліотеки Firmata.h та Wire.h, який є у середовищі програмування Arduino IDE: File → Examples → Firmata → StandatdFirmata.ino.

Підключити контролер Arduino UNO до послідовного порту 'COM5', цифрові сенсори підключити на цифровий порт 'd:10:i' та 'd:13:o', у режимі введення 'i' та виведення 'o' даних, а аналоговий давач підключити до аналогового порту A0.

Реалізувати послідовне читання даних з аналогового та цифрового виводу Arduino:

```
import pyfirmata
import time

if __name__ == '__main__':
    board = pyfirmata.Arduino('COM5')
    print("Communication Successfully started")

    # Start iterator to receive input data
    it = pyfirmata.util.Iterator(board)
    it.start()
    print("Successfully started iterator to receive input data")

    potentiometer = board.analog[0]
    potentiometer.enable_reporting()

    digital_input = board.get_pin('d:10:i')
    led = board.get_pin('d:13:o')

    while True:
        print(potentiometer.read())
        time.sleep(0.1)

        sw = digital_input.read()
        if sw is True:
            led.write(1)
        else:
            led.write(0)
        time.sleep(0.1)
```

У цьому завданні для перевірки деяких основних функцій протоколу Firmata необхідно використати потенціометр (potentiometer) на аналоговому виході та світлодіод (led) на цифровому виході Arduino. Після завантаження коду Firmata на плату Arduino, можна керувати світлодіодом із програмного стосунку на Python.

1.4. Контрольні питання

1. Які давачі та виконавчі пристрої сумісні з контролером Arduino?
2. Як підключаються давачі до плати мікроконтролера Arduino?
3. Які можливості надає у Python бібліотека pySerial?
4. Які можливості надає у Python бібліотека PyFirmata?
5. Що потрібно зробити у Arduino IDE для програмування на Python?

ПРАКТИЧНА РОБОТА № 2

РОЗРОБКА ГРАФІЧНОГО ІНТЕРФЕЙСУ КОРИСТУВАЧА (GUI) З ВИКОРИСТАННЯМ БІБЛІОТЕКИ TKINTER

Мета заняття: набути навичок розробки графічного інтерфейсу (GUI) із застосуванням функцій бібліотеки Tkinter використовуючи мову програмування Python.

2.1. Основні теоретичні відомості

Бібліотека Tkinter найчастіше використовується для розробки графічного інтерфейсу користувача (GUI) на Python. Вона є стандартним інтерфейсом з набором інструментів Tk GUI. Оскільки Tk і Tkinter доступні на більшості платформ та у системі Windows, розробка GUI програм за допомогою Tkinter є найшвидшою та найпростішою.

У мові Python доступні деякі інші бібліотеки для створення власних програм із графічним інтерфейсом. Це такі бібліотеки: Kivy, Python Qt, wxPython, PySimpleGUI.

Віджети в Tkinter є елементами програми GUI, що надають різні програмні елементи керування (такі як мітки, кнопки, поля зі списком, прапорці, панелі меню, радіокнопки та багато іншого) для взаємодії з користувачем використовуючи інструментарій об'єктно-орієнтованого інтерфейсу Tk GUI.

Модуль Tkinter надає різноманітні загальні елементи графічного інтерфейсу, які можна використовувати для створення інтерфейсу GUI, наприклад кнопки, меню та різноманітні поля введення та області відображення. ці елементи мають назву віджети. Кожен окремий віджет є об'єктом Python. Під час створення віджета він передається до батьківського елемента як параметр функції створення віджета.

Бібліотека Tkinter підтримує наступні основні віджети [2]:

Label	Використовується для відображення тексту або зображення на екрані.
Button	Використовується для додавання кнопок до програми.
Canvas	Використовується для малювання малюнків та інших макетів, таких як тексти, графіка тощо.
ComboBox	Містить стрілку вниз для вибору зі списку доступних параметрів.
CheckButton	Відображає кілька параметрів для користувача як перемикачі.
RadioButton	Дозволяє користувачеві вибрати тільки один з елементів набору.
Entry	Використовується для введення однорядкового тексту від користувача.
Frame	Використовується як контейнер для зберігання та організації віджетів.
Message	Працює так само, як і Label і стосується багаторядкового тексту, який не редагується.
Scale	Використовується для надання графічного зображення повзунка, який дозволяє вибрати будь-яке значення з цієї шкали.
Scrollbar	Використовується для прокручування вмісту вниз, забезпечує контролер слайдів.
SpinBox	Дозволяє користувачеві вибирати з заданого набору значень.
Text	Дозволяє користувачеві редагувати багаторядковий текст і вформатування так, як це має відображатися.
Menu	Використовується для створення всіх видів меню, які використовуються програмою.

У бібліотеці Tkinter віджети є об'єктами, а екземпляри класів представлено кнопками, мітками, рамками тощо:

```
from tkinter import *
from tkinter.ttk import *

# об'єкт головного вікна під назвою root
root = Tk()

# надання заголовка головному вікну
root.title("Перша програма")

# рамка у вікні
frame = Frame(root)

# метод упакування
frame.pack()

# кнопка всередині рамки
button = Button(frame, text='КПІ')
button.pack()

# Мітка (Label) - це те, що буде виведено
# показати на вікні
label = Label(root, text="Привіт !").pack()

# виклик методу основного циклу,
# щоб код продовжував відображатися
root.geometry("300x200+200+200")
root.mainloop()
```

Створення нового віджета не означає, що він з'явиться на екрані. Щоб відобразити його, нам потрібно викликати спеціальний метод: `grid`, `pack` (приклад вище) або `place` [3].

- | | |
|----------------------|--|
| <code>pack()</code> | Менеджер розташування сітки поміщає віджети у двовимірну таблицю. Головний віджет розбивається на кілька рядків і стовпців, і кожна частина отриманої таблиці може містити віджет. |
| <code>grid()</code> | Головний віджет розбивається на кілька рядків і стовпців, і кожна частина отриманої таблиці може містити віджет. |
| <code>place()</code> | Менеджер розташування пакує віджети в рядки або стовпці. |

Модуль `tkinter.ttk` забезпечує доступ до набору віджетів на тему Tk, представлених в Tkinter. Основна ідея `tkinter.ttk` полягає в тому, щоб відокремити, наскільки це можливо, код який реалізує поведінку віджета, відносно коду, що реалізує його зовнішній вигляд. Модуль `tkinter.ttk` використовується для створення графічного інтерфейсу користувача (GUI) для програм, які не можуть бути створені самим Tkinter (рис. 2.1):

```
from tkinter import *

# об'єкт головного вікна під назвою root
root = Tk()

# надання заголовка головному вікну
root.title("Програма з віджетом tkinter.ttk")

# рамка у вікні
frame = Frame(root)

# метод упакування
frame.pack()

# Створити кнопку
btn = Button(root, text = 'Натисни мене !', bd = '5',
             command = root.destroy)
# Встановити положення кнопки у верхній частині вікна.
btn.pack(side = 'top')

# Відкрите вікно розміром 400x200
root.geometry('400x200')
root.mainloop()
```

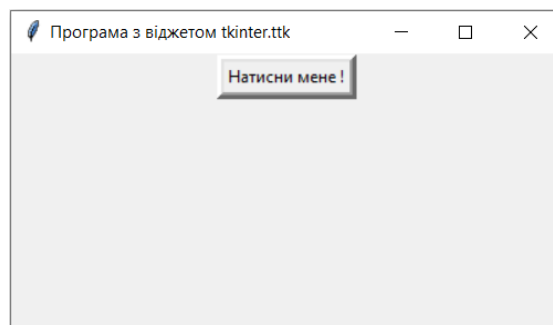


Рисунок 2.1 – Створення кнопок з модулем `tkinter.ttk`

Віджет `Label` (мітка) використовується для реалізації вікон відображення, де можна розмістити текст або зображення. Текст, який відображається цим віджетом, може бути змінений у будь-який час. Він також використовується для виконання таких завдань, як підкреслення частини тексту та переносу тексту на кілька рядків. Важливо зауважити, що

мітка Label може використовувати лише один шрифт одночасно для відображення тексту. Щоб використовувати мітку, необхідно вказати, що потрібно в ній відображати (рис. 2.2):

```
from tkinter import *

# об'єкт головного вікна під назвою top
top = Tk()

# надання заголовка головному вікну
top.title("Вікно реєстрації користувача")

# мітка для імені користувача
user_name = Label(top, text = "Ім'я").place(x = 40, y = 60)

# мітка для user_password
user_password = Label(top, text = "Пароль").place(x = 40, y = 100)
submit_button = Button(top, text = "Надіслати").place(x = 40, y = 130)
user_name_input_area = Entry(top, width = 30).place(x = 110, y = 60)
user_password_entry_area = Entry(top, width = 30).place(x = 110, y = 100)

# Відкрите вікно розміром 450x300
top.geometry("450x250")
top.mainloop()
```

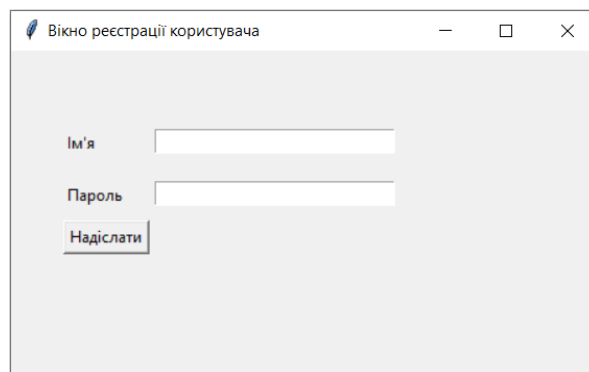


Рисунок 2.2 – Визначення віджету Label

Для реалізації вибору одного з багатьох використовується стандартний віджет Radiobutton із бібліотеки Tkinter. Перемикачі Radiobutton можуть містити текст або зображення, і можуть бути пов'язані функцією або методом Python з кожною кнопкою.

Визначення віджету Radiobutton має вигляд: `button = Radiobutton` (основа, `text="Ім'я кнопки"`, `змінні = "спільна змінна"`, `значення = "значення кожної кнопки"`, `параметри= значення, ...`). Коли кнопка натиснута, Tkinter автоматично викликає цю функцію або метод (рис. 2.3 а):

```

from tkinter import *

# Створення робочого вікна
m = Tk()

# надання заголовка головному вікну
m.title(" Перемикачі Radiobutton ")

# Рядкова змінна Tkinter
# можливість зберігати будь-яке значення рядка
v = StringVar(m, "1")

# Dictionary to create multiple buttons
# Перемикачі у формі кнопкового поля
# Щоб відобразити вікно кнопок,
# параметр індикатора має бути встановлено на 0.
values = {"RadioButton 1" : "1",
          "RadioButton 2" : "2",
          "RadioButton 3" : "3",
          "RadioButton 4" : "4",
          "RadioButton 5" : "5"}

# Цикл використовується для створення кількох радіокнопок
# замість створення кожної кнопки окремо
for (text, value) in values.items():
    Radiobutton(m, text = text, variable = v,
                value = value, indicator = 0,
                background = "red").pack(fill = X, ipady = 5)
# Нескінченний цикл можна завершити перериванням з клавіатури або миші
# або будь-якою попередньо визначеною функцією (destroy()) чи cntrl-c

# Відкрите вікно розміром 400x200
m.geometry("400x200")
mainloop()

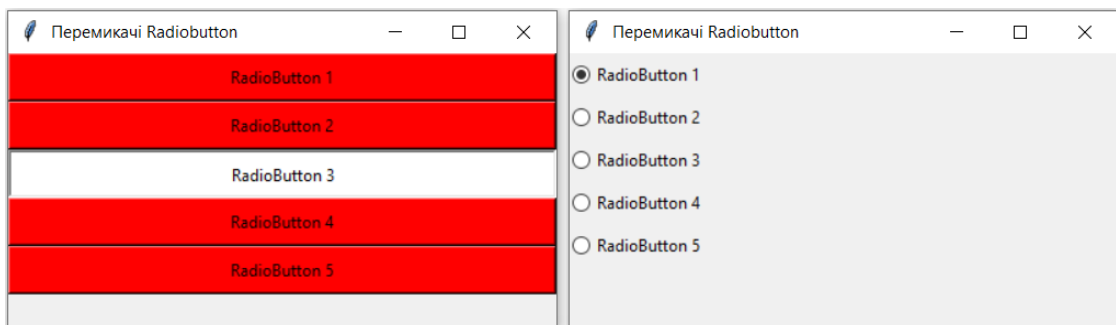
```

Щоб змінити блоки кнопок на стандартні перемикачі маємо видалити індикатор у визначення віджету Radiobutton (рис. 2.3. б):

```

# Змінимо блоки кнопок на стандартні перемикачі
# Для цього видаляємо індикатор.
for (text, value) in values.items():
    Radiobutton(m, text = text, variable = v,
                value = value).pack(fill = X, ipady = 5)
# Нескінченний цикл можна завершити перериванням з клавіатури

```



а)

б)

Рисунок 2.3 – Визначення віджету Radiobutton

Віджет `Checkbutton` є стандартним відметом Tkinter, який використовується для реалізації вибору включення/вимкнення. Кнопки з прапорцями можуть містити текст або зображення (рис. 2.4).

```
from tkinter import *

root = Tk()
# надання заголовка головному вікну
root.title(" Вибір включення/вимкнення ")

w = Label(root, text = 'Зробіть свій вибір', font = "14")
w.pack()

Checkbutton1 = IntVar()
Checkbutton2 = IntVar()
Checkbutton3 = IntVar()
Button1 = Checkbutton(root, text = "Заява",
    variable = Checkbutton1,
    onvalue = 1,
    offvalue = 0,
    height = 2,
    width = 10)
Button2 = Checkbutton(root, text = "Студент",
    variable = Checkbutton2,
    onvalue = 1,
    offvalue = 0,
    height = 2,
    width = 10)
Button3 = Checkbutton(root, text = "Курс",
    variable = Checkbutton3,
    onvalue = 1,
    offvalue = 0,
    height = 2,
    width = 10)
Button1.pack()
Button2.pack()
Button3.pack()

root.geometry("400x200")
mainloop()
```

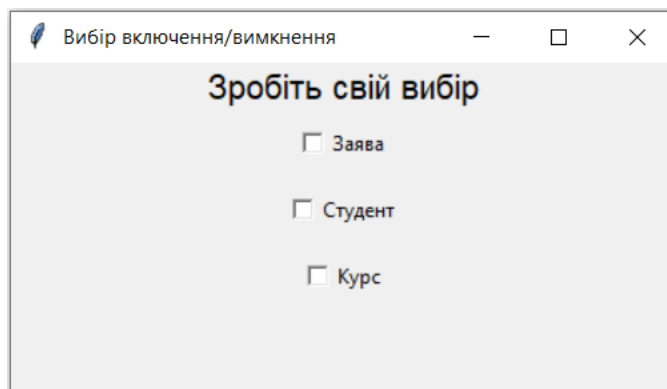


Рисунок 2.4 – Визначення віджету `Checkbutton`

Комбінований список Combobox є комбінацією списку та поля введення. Це один із віджетів Tkinter, який містить стрілку вниз для вибору зі списку параметрів, що допомагає вибирати поле відповідно до списку відображених параметрів. Коли користувач натискає стрілку розкривного меню в полі введення, внизу поля введення відображається спливаюче вікно прокрученого списку. Вибраний параметр відображатиметься в полі введення, лише якщо вибрано параметр зі списку (рис. 2.5).

```
import tkinter as tk
from tkinter import ttk

window = tk.Tk()
# надання заголовка головному вікну
window.title(" Комбінований список Combobox ")

ttk.Label(window, text = "Вибрати місяць :",
          font = ("Times New Roman", 10)).grid(column = 0,
          row = 15, padx = 10, pady = 25)

n = tk.StringVar()
monthchoosen = ttk.Combobox(window, width = 27, textvariable = n)
monthchoosen['values'] = (' Січень',
                          ' Лютий',
                          ' Березень',
                          ' Квітень',
                          ' Травень',
                          ' Червень',
                          ' Липень',
                          ' Серпень',
                          ' Вересень',
                          ' Жовтень',
                          ' Листопад',
                          ' Грудень')
monthchoosen.grid(column = 1, row = 15)
# Показує лютий як значення за замовчуванням monthchoosen.current(1)

window.geometry('400x250')
window.mainloop()
```

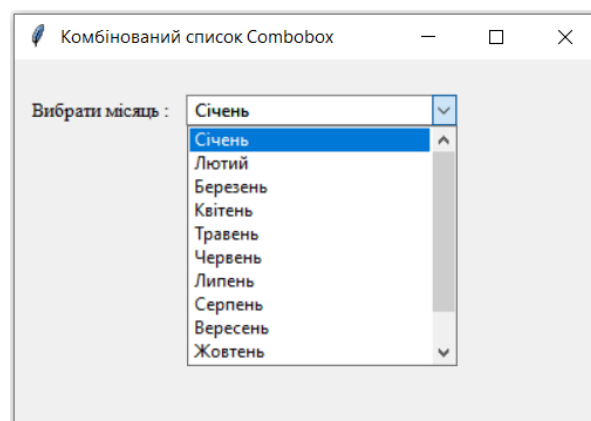


Рисунок 2.5 – Визначення віджету Combobox

Віджет Text використовується, якщо необхідно вставити багаторядкові текстові поля. Цей віджет можна використовувати для різних програм, де потрібен багаторядковий текст, наприклад, для обміну повідомленнями, виправлення чи відображення інформації та багатьох інших завдань. У текстовий віджет можна вносити мультимедійні файли, такі як зображення та посилання на ресурси (рис. 2.6) [2]:

```
from tkinter import *
from tkinter.ttk import *

window = Tk()
# надання заголовка головному вікну
window.title(" Обмін повідомленнями ")

def Take_input(text):
    Label(window, text=text, font=('Helvetica 13 bold')).pack()
    INPUT = inputtxt.get("1.0", "end-1c")
    print(INPUT)
    if (INPUT == "120"):
        Output.insert(END, 'Вірно')
    else:
        Output.insert(END, "Невірно")

l = Label(text="Скільки 24 * 5 = ? ")
inputtxt = Text(window, height=10, width=25, bg="light yellow")
Output = Text(window, height=5, width=25, bg="light cyan")
Display = Button(window, text="Показати", command= lambda:
    Take_input("Відповідь дана"))

l.pack()
inputtxt.pack()
Display.pack()
Output.pack()

# Відкрите вікно розміром 600x300
window.geometry("300x350")
window.mainloop()
```

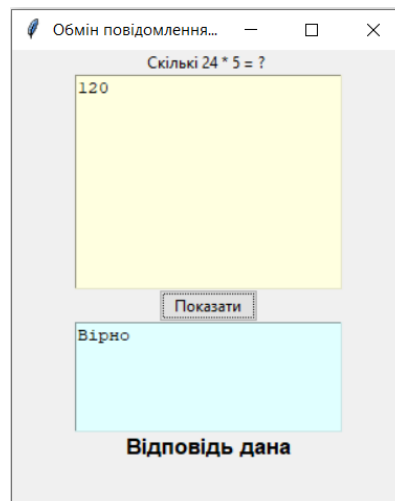


Рисунок 2.6 – Визначення віджету Text

Віджет Progressbar відображає процес або показує, що щось відбувається: `widget_object = Progressbar(parent, **options)`. Цей віджет може працювати в одному з двох режимів: у *невизначеному режимі* віджет анімується і показує, що щось відбувається, а індикатор підстрибує між кінцями віджету; у *детермінованому режимі* віджет показує індикатор, який переміщається з початку до кінця під керуванням програми (рис. 2.7):

```
from tkinter import *
from tkinter.ttk import *

# створення вікна tkinter
root = Tk()
# надання заголовка головному вікну
root.title(" Візуалізація процесу ")

# Віджет панелі прогресу
progress = Progressbar(root, orient = HORIZONTAL,
    length = 100, mode = 'determinate')
# Функція, відповідальна за оновлення
# значення індикатора виконання
def bar():
    import time
    progress['value'] = 20
    root.update_idletasks()
    time.sleep(1)
    progress['value'] = 40
    root.update_idletasks()
    time.sleep(1)
    progress['value'] = 50
    root.update_idletasks()
    time.sleep(1)
    progress['value'] = 60
    root.update_idletasks()
    time.sleep(1)
    progress['value'] = 80
    root.update_idletasks()
    time.sleep(1)
    progress['value'] = 100
    progress.pack(pady = 10)

# Ця кнопка буде ініціалізована індикатор прогресу
Button(root, text = 'Start', command = bar).pack(pady = 10)

# Відкрите вікно розміром 350x100
root.geometry("350x100")
root.mainloop()
```

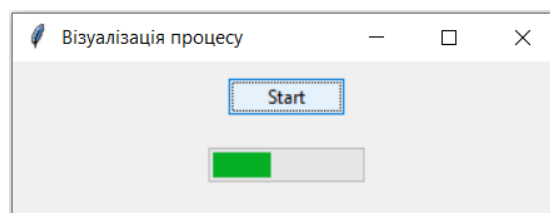


Рисунок 2.7 – Визначення віджету Progressbar у детермінованому режимі

Анімація віджету Progressbar у невизначеному режимі (рис. 2.8):

```
from tkinter import *
from tkinter.ttk import *
root = Tk()
# надання заголовка головному вікну
root.title(" Візуалізація процесу ")
progress = Progressbar(root, orient = HORIZONTAL,
    length = 100, mode = 'indeterminate')

def bar():
    import time
    progress['value'] = 20
    root.update_idletasks()
    time.sleep(0.5)
    progress['value'] = 40
    root.update_idletasks()
    time.sleep(0.5)
    progress['value'] = 50
    root.update_idletasks()
    time.sleep(0.5)
    progress['value'] = 60
    root.update_idletasks()
    time.sleep(0.5)
    progress['value'] = 80
    root.update_idletasks()
    time.sleep(0.5)
    progress['value'] = 100
    root.update_idletasks()
    time.sleep(0.5)
    progress['value'] = 80
    root.update_idletasks()
    time.sleep(0.5)
    progress['value'] = 60
    root.update_idletasks()
    time.sleep(0.5)
    progress['value'] = 50
    root.update_idletasks()
    time.sleep(0.5)
    progress['value'] = 40
    root.update_idletasks()
    time.sleep(0.5)
    progress['value'] = 20
    root.update_idletasks()
    time.sleep(0.5)
    progress['value'] = 0

progress.pack(pady = 10)
Button(root, text = 'Start', command = bar).pack(pady = 10)
# Відкрите вікно розміром 350x100
root.geometry("350x100")
mainloop()
```

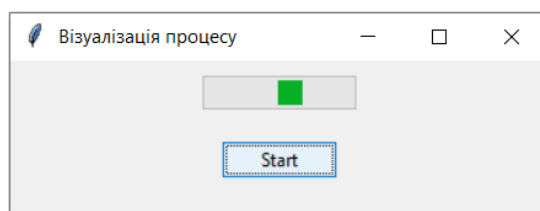


Рисунок 2.8 – Визначення віджету Progressbar у невизначеному режимі

Віджет Frame (Рамка) формує прямокутну область на екрані. Фрейм також можна використовувати як базовий клас для реалізації складних віджетів. Він використовується для організації групи віджетів (рис. 2.9):

```
from tkinter import *

root = Tk()
root.title(" Візуалізація фрейму з кнопками ")

w = Label(root, text='КПІ ім. Ігоря Сікорського', font = "14")
w.pack()

frame = Frame(root)
frame.pack()
bottomframe = Frame(root)
bottomframe.pack( side = BOTTOM )

b1_button = Button(frame, text ="    КПІ 1 корп.    ", fg ="red")
b1_button.pack( side = LEFT)

b2_button = Button(frame, text ="    КПІ 2 корп.    ", fg ="brown")
b2_button.pack( side = LEFT )

b3_button = Button(frame, text ="    КПІ 3 корп.    ", fg ="blue")
b3_button.pack( side = LEFT )

root.geometry("400x100")
root.mainloop()
```

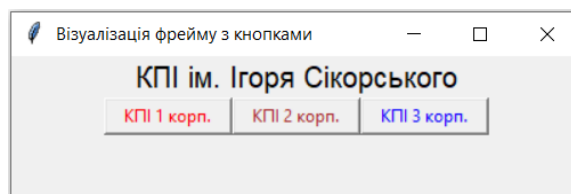


Рисунок 2.9 – Визначення віджету Frame (рамка)

Віджет відкриття файлів Askopenfile() необхідний під час роботи з графічним інтерфейсом користувача (GUI) для відкриття файлів та читання даних з нього. Цього можна досягти за допомогою функції open() (вбудованої в Python), вказавши шлях до конкретного файлу та виконавши такі дії (рис. 2.10):

```
from tkinter import *
from tkinter.ttk import *
# імпорт функції askopenfile
# з діалогового вікна файлу класу
from tkinter.filedialog import askopenfile

root = Tk()
root.title(" Відкриття файлу для читання ")
```



```

# Ця функція буде використана для відкриття
# файл у режимі читання та лише файли Python
# буде відкрито
def open_file():
    file = askopenfile(mode='r', filetypes=[('Python Files', '*.py')])
    if file is not None:
        content = file.read()
        print(content)
btn = Button(root, text='Open', command = lambda:open_file())
btn.pack(side = TOP, pady = 10)

root.geometry('400x100')
mainloop()

```

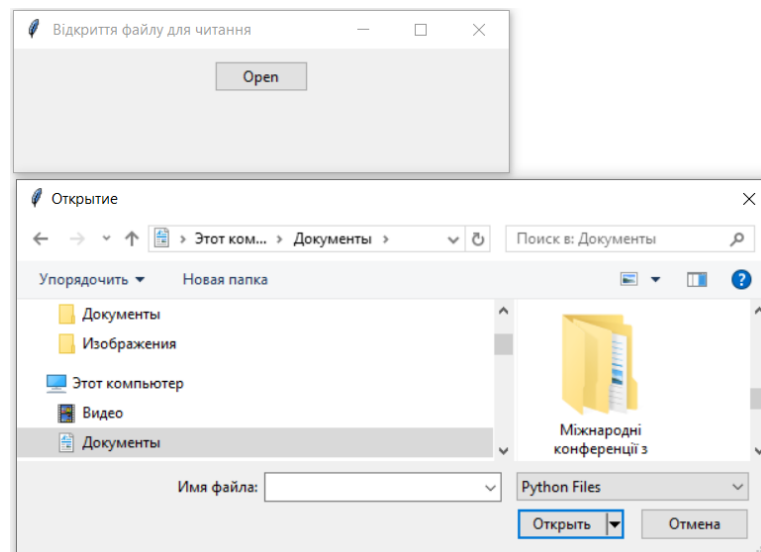


Рисунок 2.10 – Визначення віджету відкриття файлів Askopenfile()

Віджет відкриття і збереження файлів Asksaveasfile() необхідний під час роботи з файлами: відкрити файли, виконати над ними операції, зберегти файл. Віджет `asksaveasfile()` є функцією, яка використовується для збереження файлу (розширення можна встановити явно або за замовчуванням). Функція `asksaveasfile()` знаходиться в діалоговому вікні файлу класу (рис. 2.11):

```

from tkinter import *
from tkinter import ttk
# імпортувати лише asksaveasfile з діалогового вікна file
# який використовується для збереження файлу з будь-яким розширенням
from tkinter.filedialog import asksaveasfile

root = Tk()
root.title(" Збереження файлу після оброблення ")

# функція для виклику, коли користувач натискає
# кнопка збереження, відкривається діалогове вікно файлу

```

```

# відкрити та попросити зберегти файл
def save():
    files = [('All Files', '*.*),
            ('Python Files', '*.py'),
            ('Text Document', '*.txt')]
    file = asksaveasfile(filetypes = files, defaultextension = files)

    btn = ttk.Button(root, text = 'Save', command = lambda : save())
    btn.pack(side = TOP, pady = 20)

root.geometry('400x100')
mainloop()

```

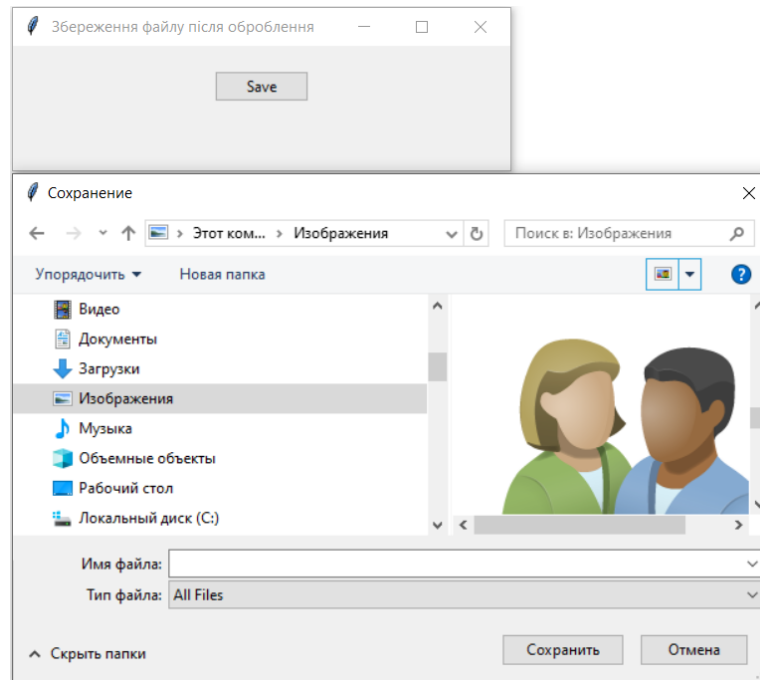


Рисунок 2.11 – Визначення віджету збереження файлів `Asksaveasfile()`

Віджет Menu (меню) є важливою частиною будь-якого графічного інтерфейсу користувача. Зазвичай меню використовують для забезпечення зручного доступу до різноманітних операцій, таких як збереження або відкриття файлу, вихід з програми або маніпулювання даними. Меню верхнього рівня відображаються безпосередньо під рядком заголовка кореневого або будь-якого іншого вікна верхнього рівня. Елементи підменю розташовуються з прив'язкою до батьківського вікна (2.12):

```

from tkinter import *
from tkinter.ttk import *
from time import strftime

# створення вікна tkinter
root = Tk()
root.title(' Menu simple program ')

```

```

# Створення панелі меню
menubar = Menu(root)

# Додавання меню «Файл» і команд
file = Menu(menubar, tearoff = 0)
menubar.add_cascade(label='File', menu = file)
file.add_command(label='New File', command = None)
file.add_command(label='Open...', command = None)
file.add_command(label='Save', command = None)
file.add_separator()
file.add_command(label='Exit', command = root.destroy)

# Додавання меню редагування та команд
edit = Menu(menubar, tearoff = 0)
menubar.add_cascade(label='Edit', menu = edit)
edit.add_command(label='Cut', command = None)
edit.add_command(label='Copy', command = None)
edit.add_command(label='Paste', command = None)
edit.add_command(label='Select All', command = None)
edit.add_separator()
edit.add_command(label='Find...', command = None)
edit.add_command(label='Find again', command = None)

# Додавання меню довідки
help_ = Menu(menubar, tearoff = 0)
menubar.add_cascade(label='Help', menu = help_)
help_.add_command(label=' Help', command = None)
help_.add_command(label='Demo', command = None)
help_.add_separator()
help_.add_command(label='information', command = None)

# дисплей меню
root.config(menu = menubar)
root.geometry('400x100')
root.mainloop()

```

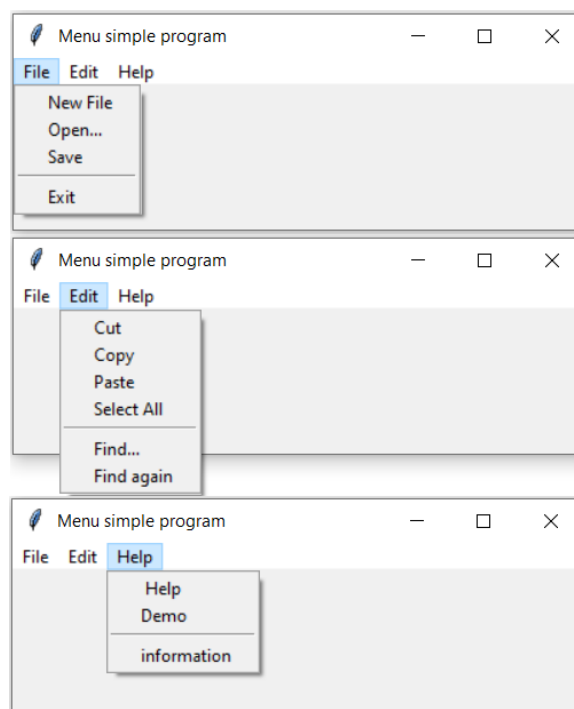


Рисунок 2.12 – Визначення віджету Menu

Розробка власного проекту будуватиметься на створенні багатьох об'єктів (віджетів). Меню графічного інтерфейсу (GUI) може містити багато елементів, причому елементи меню можуть представляти як стандартні віджети так і містити інші елементи. Залежно від того, який тип елементів ми хочемо додати в меню, відрізнятиметься метод, який використовується для їхнього додавання.

2.2. Порядок роботи

Частина 1. Використання менеджера pack із бібліотеки Tkinter для розміщення графічного інтерфейсу (GUI).

Підключити до програми на мові Python бібліотеку Tkinter.ttk та зробити структуру з використанням менеджера pack для відображення фрейму Frame(), кнопок Button(), тексту Text() і віджету мітки Label():

```
frame = Frame(self, relief=RAISED, borderwidth=1)
frame.pack(fill=BOTH, expand=True)

lbl = Label(self, text="Вікно вводу", width=15)
lbl.pack(side=TOP, anchor=N, padx=5, pady=5)

area = Text(self, width=50)
area.pack(side=LEFT, pady=5, padx=5, expand=True)

cbtn = Button(self, text="Закрити")
cbtn.pack(side=BOTTOM, padx=5, pady=5)

obtn = Button(self, text="Готово")
obtn.pack(side=BOTTOM, padx=5, pady=5)
```

Визначити геометрію графічного інтерфейсу за допомогою функції root.geometry() з бібліотеки Tkinter, який має оновлюватись:

```
root = Tk()
root.geometry("500x300+200+200")
root.mainloop()
```

Перші два числа у рядку функції `geometry("500x300+200+200")` є аргументами, які задають ширину та висоту вікна, а функція `mainloop()` дозволяє оновлювати відображення інтерфейсу.

У підсумку програмний код реалізації графічного інтерфейсу з використанням менеджера `pack` із бібліотеки `Tkinter` має вигляд:

```
from tkinter import Tk, TOP, LEFT, BOTTOM, RAISED, BOTH, Text, W, N,
E, S
from tkinter.ttk import Frame, Button, Label, Style

class Example(Frame):

    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        self.master.title("Кнопки і Вікна в Tkinter")
        self.style = Style()
        self.style.theme_use("default")

        frame = Frame(self, relief=RAISED, borderwidth=1)
        frame.pack(fill=BOTH, expand=True)

        self.pack(fill=BOTH, expand=True)

        lbl = Label(self, text="Вікно вводу", width=15)
        lbl.pack(side=TOP, anchor=N, padx=5, pady=5)

        area = Text(self, width=50)
        area.pack(side=LEFT, pady=5, padx=5, expand=True)

        cbtn = Button(self, text="Закрити")
        cbtn.pack(side=BOTTOM, padx=5, pady=5)
        obtn = Button(self, text="Готово")
        obtn.pack(side=BOTTOM, padx=5, pady=5)

def main():

    root = Tk()
    root.geometry("500x300+200+200")
    app = Example()
    root.mainloop()

if __name__ == '__main__':
    main()
```

Вивід поля для введення тексту (`Text`) та візуалізація кнопок (`Button`) із використанням менеджера `pack` графічного інтерфейсу бібліотеки `Tkinter`:

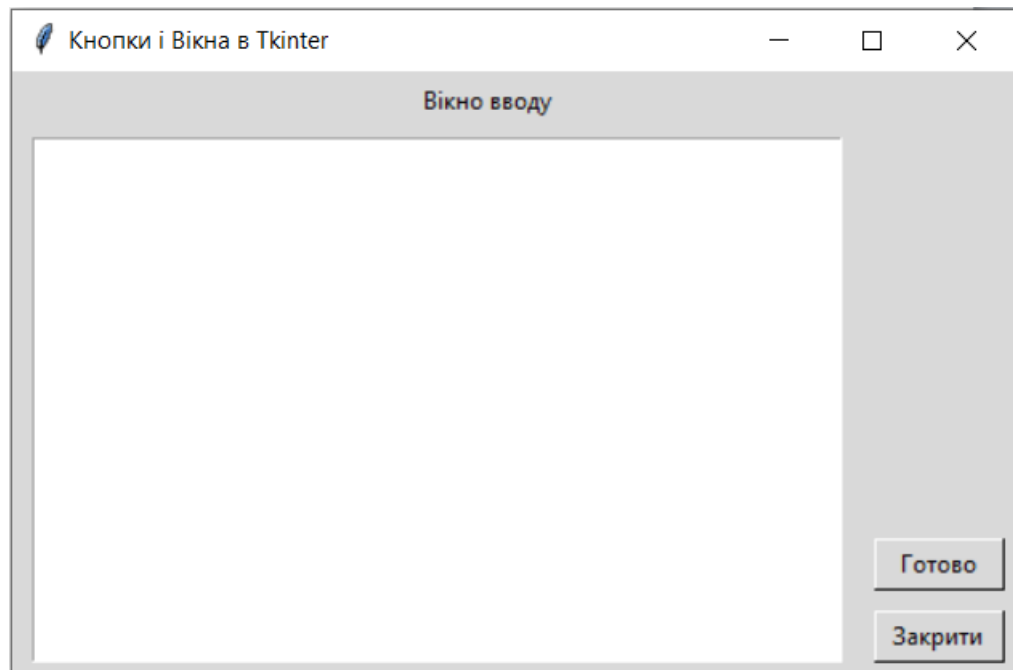


Рисунок 2.13 – Вивід поля для введення тексту
із використанням менеджера pack

Частина 2. Використання менеджера grid із бібліотеки Tkinter для розміщення графічного інтерфейсу (GUI).

Підключити до програми на мові Python бібліотеку Tkinter.ttk та зробити структуру з використанням менеджера grid для відображення заголовку Title(), кнопок Button(), тексту Text() і віджету мітки Label():

```
self.master.title("Діалогове вікно в Tkinter")
self.pack(fill=BOTH, expand=True)

lbl = Label(self, text="Вікно вводу")
lbl.grid(sticky=W, pady=4, padx=5)

area = Text(self)
area.grid(row=1, column=0, columnspan=2, rowspan=4,
          padx=5, sticky=E+W+S+N)

abtn = Button(self, text="Активация")
abtn.grid(row=1, column=3)

cbtn = Button(self, text="Закрити")
cbtn.grid(row=2, column=3, pady=4)

hbtn = Button(self, text="Допомога")
hbtn.grid(row=5, column=0, padx=5)

obtn = Button(self, text="Готово")
obtn.grid(row=5, column=3)
```

Визначити геометрію графічного інтерфейсу за допомогою функції `root.geometry()` з бібліотеки Tkinter, який має оновлюватись:

```
root = Tk()
root.geometry("350x300+300+300")
root.mainloop()
```

У підсумку програмний код реалізації графічного інтерфейсу з використання менеджера `grid` із бібліотеки Tkinter має вигляд:

```
from tkinter import Tk, Text, BOTH, W, N, E, S
from tkinter.ttk import Frame, Button, Label, Style

class Example(Frame):

    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):

        self.master.title("Діалогове вікно в Tkinter")
        self.pack(fill=BOTH, expand=True)

        self.columnconfigure(1, weight=1)
        self.columnconfigure(3, pad=7)
        self.rowconfigure(3, weight=1)
        self.rowconfigure(5, pad=7)

        lbl = Label(self, text="Вікно вводу")
        lbl.grid(sticky=W, pady=4, padx=5)

        area = Text(self)
        area.grid(row=1, column=0, columnspan=2, rowspan=4,
                  padx=5, sticky=E+W+S+N)

        abtn = Button(self, text="Активация")
        abtn.grid(row=1, column=3)

        cbtn = Button(self, text="Закрити")
        cbtn.grid(row=2, column=3, pady=4)

        hbtn = Button(self, text="Допомога")
        hbtn.grid(row=5, column=0, padx=5)

        obtn = Button(self, text="Готово")
        obtn.grid(row=5, column=3)

def main():

    root = Tk()
    root.geometry("350x300+300+300")
```

```

app = Example()
root.mainloop()

if __name__ == '__main__':
    main()

```

Вивід поля для введення тексту (Text) та візуалізація кнопок (Button) із використанням менеджера grid графічного інтерфейсу бібліотеки Tkinter:

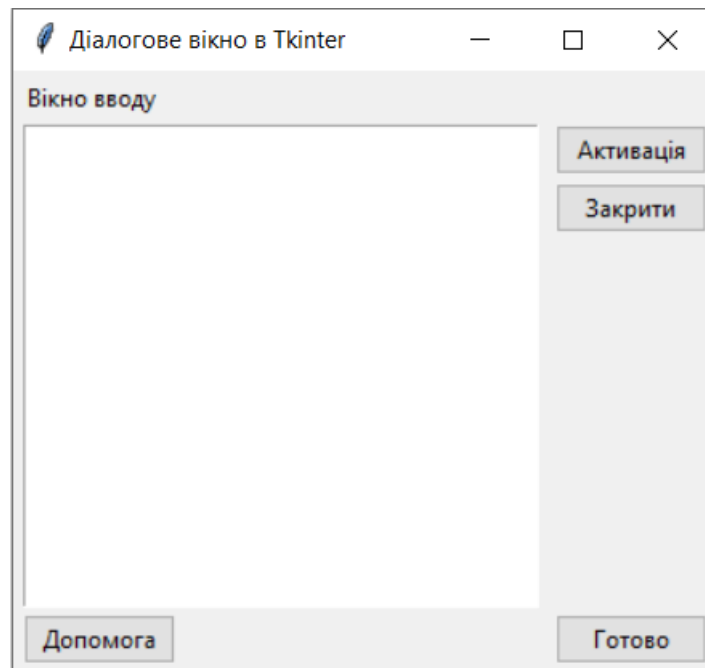


Рисунок 2.14 – Вивід поля для введення тексту
із використанням менеджера grid

2.3. Завдання для самостійної роботи

Підключити до програми на мові Python бібліотеку PySimpleGUI та зробити структуру інтерфейсу layout для відображення списку Listbox() і кнопок Button():

```

layout = [sg.vtop([sg.Listbox(list(range(10)), size=(5,5)),
sg.Multiline(size=(45,15))]),
[sg.Button('Ok'), sg.Button('Cancel')]]

```


Заголовок програми з прив'язкою до структури layout задається у функції Window():

```
window = sg.Window('Element Alignment on PySimpleGUI', layout)
```

У підсумку програмний код з використанням бібліотеки PySimpleGUI може мати вигляд:

```
import PySimpleGUI as sg

layout = [sg.vtop([sg.Listbox(list(range(10)), size=(5,5)),
sg.Multiline(size=(45,15))]),
[sg.Button('Ok'), sg.Button('Cancel')]]

window = sg.Window('Element Alignment on PySimpleGUI', layout)

while True:
    event, values = window.read()
    if event in (sg.WIN_CLOSED, 'Cancel'):
        break

window.close()
```

Візуалізація графічного інтерфейсу (GUI) з використанням бібліотеки PySimpleGUI за представленим кодом має вигляд:

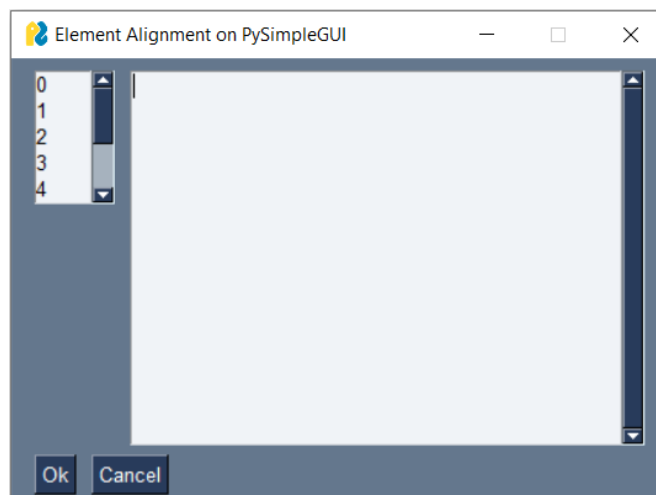


Рисунок 2.15 – Візуалізація графічного інтерфейсу з використанням бібліотеки PySimpleGUI

У вікно графічного інтерфейсу (GUI), створеного з використанням бібліотеки PySimpleGUI, можна вивести табличне значення будь-якої

маєматичної функції та надалі зчитувати і редагувати отримані табличні числові дані (рис. 2.16):

```
import math
import random
import PySimpleGUI as sg

sg.theme('Table of function')

class Fake():
    @classmethod
    def number(self, max=1000):
        return random.randint(0,max)

# Table of function
data = []
for x in range(-100,100):
    y = math.sin(x/20)*50
    data.append([x, y])
print(data)

headings = ['column x', 'column y']
# ----- Window Layout -----
layout = [[sg.Table(values=data, headings=headings,
                    max_col_width=25,
                    auto_size_columns=True,
                    justification='left',
                    background_color='gray', # Bg not changing color
                    alternating_row_color='white',
                    num_rows=3,
                    enable_events=True,
                    key='table of data',
                    tooltip='This is a table')],
          [sg.Button('Read')],
          [sg.StatusBar(key='bar of data',
                        size=(30, 0),
                        text='Double select Rows and press Read',
                        text_color='red')]]

# ----- Create Window -----
window = sg.Window('The Table Element', layout)

# ----- Event Loop -----
while True:
    event, values = window.read()

    if event is None:
        break

    if event == 'Read':
        t = values['table of data'];
        num = t[0]
        load = [num, data[num][0], data[num][1]]
        sg.Popup(event, load)

    if event == sg.WIN_CLOSED:
        break

window.close()
```

Візуалізація графічного інтерфейсу (GUI) з табличним представленням значень простої математичної функції:

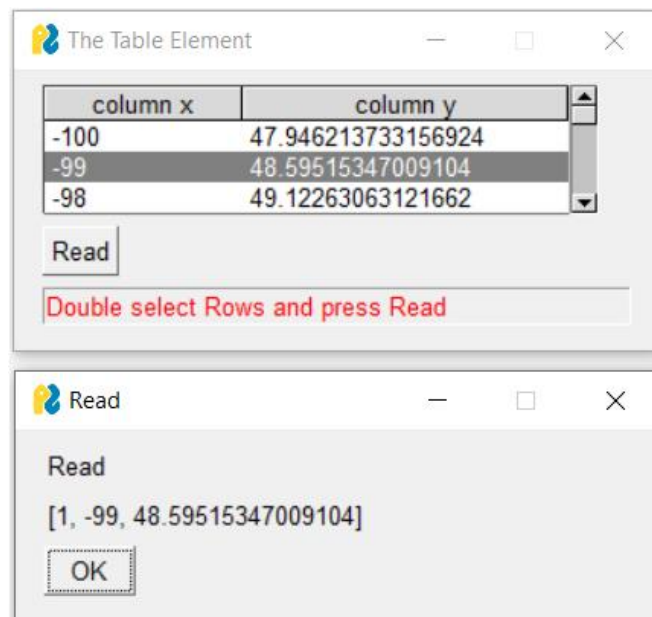


Рисунок 2.16 – Візуалізація графічного інтерфейсу з табличним представленням математичної функції

2.4. Контрольні питання

1. Які основні віджети підтримує бібліотека Tkinter?
2. Які є спеціальні методи для відображення віджетів?
3. Які можливості має модуль tkinter.ttk?
4. Які функції введення і виведення даних має віджет Text?
5. Які особливості анімації процесу має віджет Progressbar?
6. Які особливості організації групи модулів має віджет Frame?
7. Які віджети використовуються для відкриття і збереження файлів?
8. Які можливості надає віджет Menu?
9. Які можливості надає бібліотека PySimpleGUI?
10. Які особливості має віджет Table з бібліотеки PySimpleGUI?

ПРАКТИЧНА РОБОТА № 3

МАТЕМАТИЧНІ ОБЧИСЛЕННЯ З ВИКОРИСТАННЯМ ФУНКЦІЙ БІБЛІОТЕК NUMPY ТА MATH

Мета заняття: набути навичок використання функцій бібліотек Math та NumPy для рішення алгебраїчних рівнянь використовуючи мову програмування Python.

3.1. Основні теоретичні відомості

Математичні функції мови програмування Python зібрані у пакеті `math.py`, який включено до бібліотеки Math мови програмування Python. Підключити модуль Math до програми можна за допомогою зарезервованого слова `import`:

```
import math #підключаємо до програми стандартний модуль math
# тепер можна користуватись функціями з даного модуля
# наприклад визвати функцію обчислення квадратного кореня
math.sqrt(9) #зверніть увагу комбіноване ім'я - модуль.функція
```

Якщо назва модуля дуже довга, то для спрощення можна задати для модуля псевдонім:

```
# замість math можна використовувати псевдонім m замість math
import math as m # псевдонім m замість math
m.sqrt(9)
m.sqrt(9)
```

Можна підключати до проекту не весь модуль а окремі функції або змінні з модуля:

```
# імпортуємо математичні функції корінь квадратний, синус та косинус
from math import sqrt, cos, sin
# тепер можна звертатись до цих функцій
# зверніть увагу, що в такому разі ім'я модуля не потрібне
cos(3.14)
```

Аналогічно можна задати псевдоніми для окремих імпортованих функцій або змінних:

```
# задаємо псевдонім для функції визначення квадратного кореня
from math import sqrt as sq # псевдонім sq замість math.sqrt
sq(9) #використовуємо псевдонім
```

Модуль `Math` має великий набір функцій і констант для розв'язку математичних і фізичних задач. Розглянемо детальніше математичні функції стандартного модулю [4]:

`math.ceil(x)` – округлення до найближчого більшого числа;

`math.copysign(x, y)` – повертає число, що має модуль такий, як у числа x , а знак такий як у числа y ;

`math.fabs(x)` – повертає модуль числа x ;

`math.factorial(x)` – повертає факторіал числа x ;

`math.floor(x)` – округлення до найближчого меншого числа;

`math.fmod(x, y)` – повертає залишок від ділення x на y ;

`math.frexp(x)` – повертає мантису і експоненту числа x ;

`math.ldexp(x, i)` – повертає $x * 2^i$, це обернена функція до функції `math.frexp()`;

`math.fsum(послідовність)` – повертає суму всіх членів у послідовності;

`math.isfinite(x)` – повертає логічне значення `True`, якщо x є скінченне число;

`math.isinf(x)` – повертає логічне значення `False`, якщо x не є нескінченністю;

`math.isnan(x)` – перевіряє чи є x NaN (Not a Number – не числом), повертає логічне значення `False`, якщо x – число;

`math.modf(x)` – повертає дробову і цілу частину числа x . Обидва числа мають той же знак, що і x ;

`math.trunc(x)` – відкидає дробову частину числа x ;

`math.exp(x)` – обчислює e^x ;

`math.expm1(x)` – обчислює $e^x - 1$;

`math.log(x, [base])` – обчислює логарифм числа x за основою `base`, якщо `base` не вказано, обчислюється натуральний логарифм;

$\text{math.log1p}(x)$ – обчислює натуральний логарифм від $(1 + x)$;
 $\text{math.log10}(x)$ – обчислює десятковий логарифм x ;
 $\text{math.log2}(x)$ – обчислює логарифм числа x за основою 2;
 $\text{math.pow}(x, y)$ – обчислює значення x^y ;
 $\text{math.sqrt}(x)$ – обчислює квадратний корінь з x ;
 $\text{math.acos}(x)$ – обчислює x , результат повертає в радіанах;
 $\text{math.asin}(x)$ – обчислює арксинус x , результат повертає в радіанах;
 $\text{math.atan}(x)$ – обчислює арктангенс x , результат повертає в радіанах;
 $\text{math.atan2}(y, x)$ – обчислює арктангенс від величини y/x , результат повертає в радіанах з урахуванням чверті, у якій знаходиться точка (y, x) ;
 $\text{math.cos}(x)$ – обчислює косинус кута x (x вказується в радіанах);
 $\text{math.sin}(x)$ – обчислює синус кута x (x вказується в радіанах);
 $\text{math.tan}(x)$ – обчислює тангенс кута x (x вказується в радіанах);
 $\text{math.hypot}(x, y)$ – обчислює гіпотенузу трикутника з катетами x і y ;
 $\text{math.degrees}(x)$ – конвертує радіани у градуси;
 $\text{math.radians}(x)$ – конвертує градуси у радіани;
 $\text{math.cosh}(x)$ – обчислює гіперболічний косинус кута x (x вказується в радіанах);
 $\text{math.sinh}(x)$ – обчислює гіперболічний синус кута x (x вказується в радіанах);
 $\text{math.tanh}(x)$ – обчислює гіперболічний тангенс кута x (x вказується в радіанах);
 $\text{math.acosh}(x)$ – обчислює зворотній гіперболічний косинус від x , результат повертає в радіанах;
 $\text{math.asinh}(x)$ – обчислює зворотній гіперболічний синус від x , результат повертає в радіанах;
 $\text{math.atanh}(x)$ – обчислює розраховує зворотній гіперболічний тангенс від x , результат повертає в радіанах;
 $\text{math.erf}(x)$ – обчислює функцію помилок;
 $\text{math.erfc}(x)$ – обчислює додаткову функцію помилок $(1 - \text{math.erf}(x))$;
 $\text{math.gamma}(x)$ – обчислює гамма-функцію від x ;
 $\text{math.lgamma}(x)$ – обчислює натуральний логарифм гамма-функції від x .

Наведемо приклад використання деяких математичних функцій стандартного модулю Math:

```
from math import * # імпортувати усе з модуля math
x,y=1,1
acos(x) # арккосинус x
asin(x) # арксинус x
atan(x) # арктангенс x
atan2(y,x) # atan(y/x)
ceil(x) # найменше ціле, більше або рівне x
cos(x) # косинус x
cosh(x) # гіперболічний косинус x
e # константа e
exp(x) # експонента (e**x)
fabs(x) # абсолютне значення x
floor(x) # найбільше ціле, менше або рівне x
fmod(x,y) # остача від ділення x на y
frexp(x) # мантиса і порядок x як пара (m, i), де m - число з плаваючою
комкою, а i - ціле, таке, що x = m * 2.**i. Якщо x=0, то повертає (0,0),
інакше 0.5 <= abs(m) < 1.0
hypot(x,y) # sqrt(x*x + y*y)
ldexp(x,y) # x * (2**y)
log(x) # натуральний логарифм x
log10(x) # десятковий логарифм x
modf(x) # пара (y,q) - ціла та дробова частина x. Обидві частини мають
знак x pi # константа pi
pow(x,y) # x в степені y (або x**y)
sin(x) # синус x
sinh(x) # гіперболічний синус x
sqrt(x) # корінь квадратний від x
tan(x) # тангенс x
tanh(x) # гіперболічний тангенс x
```

Модуль `cmath` надає доступ до функцій, які можуть виконувати математичні дії над комплексними числами. Крім комплексних чисел, аргументами даних функцій можуть бути як цілі так і дійсні (з плаваючою крапкою) числа. Функції модуля `cmath` завжди повертають комплексне значення. Комплексні числа можуть бути представлені у прямокутних або у полярних координатах.

У модулі `cmath` визначено ті ж функції, що і у модулі `Math`, але адаптовані до дій над комплексними числами (за винятком функцій цілочислової математики, функцій округлення, функцій помилок, гамма функцій і деяких інших). Використання цих функцій відрізняється лише назвою модуля у вивозі функції. Наприклад, для виклику тригонометричних функцій комплексного аргументу необхідно викликати `cmath.cos(x)`,

`cmath.sin(x)` і т.д. Для переходу від одного представлення до іншого використовують спеціальні функції.

Для полегшення роботи з послідовностями існують спеціальні вбудовані функції [14]:

`filter()` – фільтрує послідовність за допомогою заданої функції;

`map()` – застосовує функцію для кожного елемента;

`reduce()` – застосовує до елементів функцію двох аргументів кумулятивно зліва направо;

`zip()` – об'єднує послідовності в список кортежів;

`enumerate()` – генерує пронумеровану послідовність;

`sorted()` – сортує послідовність.

Наведемо приклад використання цих спеціальних функцій із стандартного модулю `Math`:

```
from math import *

a=[1,2,3,4,5] # список
def fn1(x):
    return x!=3 # функція повертає істину, якщо x не 3
print(filter(fn1,a)) # відфільтрований функцією fn1 список
print(filter(lambda x:x!=3,a)) # або так #print [x for x in a if x!=3]

# або так:
def fn2(x):
    return x**2 # функція повертає квадрат числа
print(map(fn2,a)) # застосовує fn2 до кожного елемента

def fn3(x,y):
    return x+y # функція повертає суму двох чисел
print(map(fn3,a,[1,2,3,4,5])) # сума списків
print(map(lambda x,y:x+y,a,[1,2,3,4,5]))
# або так:
print([x+y for x,y in zip(a,[1,2,3,4,5])])

# підрахувати ((1+2)+3)+4)+5
print(zip([1,2,3],[4,5,6])) # об'єднати списки
print([i for i in enumerate("abc")]) # пронумерований список
print(sorted([(1,2),(2,1),(3,3)], key=lambda x: x[1])) # сортувати за
елементами з індексом 1
```

Модуль `time` надає засоби для обробки часу і дати [4]:

`time.asctime([t])` – перетворює кортеж або `struct_time` в рядок виду 'День тжня місяць число час рік' (наприклад 'Sun May 31 12:33:02 2020'), якщо аргумент не вказано, використовується поточний час;

time.clock() – повертає час, що минув з моменту першого виклику даної функції;

time.ctime ([сек]) – перетворює час, виражений в секундах з початку епохи комп'ютерної техніки ('Thu Jan 1 02:00:00 1970') в рядок виду 'День тжня місяць число час рік';

time.gmtime([сек]) – перетворює час, виражений в секундах з початку епохи комп'ютерної техніки ('Thu Jan 1 02:00:00 1970') у тип даних;

struct_time. time.mktime(t) – перетворює кортеж або *struct_time* в число секунд з початку епохи комп'ютерної техніки;

time.sleep(сек) – затримує виконання програми на задану кількість секунд;

time.strftime(формат, [t]) – перетворює кортеж або *struct_time* в рядок згідно заданого формату (див документацію на модуль);

time.strptime(рядок [формат]) – перетворює рядок, який представляє час відповідно до формату, повертає значення типу *struct_time*.

Наведемо приклад використання засбів для обробки часу і дати іункціями модулю *time*:

```
import datetime, time

d0=datetime.datetime.now() # поточна дата (datetime)
d1=datetime.datetime(2015,2,17,0,0,0,0) # задана дата (datetime)
print(d0.year, d0.month, d0.day, d0.hour, d0.minute, d0.second,
d0.microsecond, d0.tzinfo) # атрибути datetime

st=d0.timetuple() # об'єкт (time.struct_time)
print(time.mktime(st)) # кількість секунд з початку Епохи
print(datetime.datetime.strptime("Mon Apr 26 11:31:53 2010", "%a %b %d
%H:%M:%S %Y").strftime("%d%M%Y")) # конвертація з формату в формат
print(d0.isoweekday()) # ISO день тижня (1 - понеділок)
print(d1.replace(year=2014)) # дозволяє змінити певні атрибути дати #
тут місяців і років немає, бо вони можуть мати різну к-сть днів

td1=datetime.timedelta(weeks=1,days=2,hours=1)
print(td1.days, td1.seconds, td1.microseconds)
print(d1+td1) # додати до дати період (datetime)
print(d1+td1*2-abs(-td1)) # допустимі операції (datetime)
td2=d0-d1 # різниця дат (datetime)
print(d0>d1) # порівняння дат print td1>td2 # порівняння періодів
```

Модулі *datetime* та *time* містять методи для роботи з датою і часом: *date* (дата), *time* (час), *datetime* (дата і час), *timedelta* (період часу), *tzinfo*

(абстрактний клас для роботи з часовими поясами). Використання цих функцій дозволяє виконувати різноманітні математичні операції над значеннями дати і часу.

Бібліотека NumPy використовується для високопродуктивних операцій з багатовимірними масивами (у тому числі матрицями). Часто вона застосовується разом із бібліотекою SciPy та містить багато ефективних чисельних процедур для інтегрування, оптимізації, інтерполяції, статистики, обробки сигналів та іншого (<http://www.numpy.org>, <http://scipy.org>).

Розглянемо приклади базових операцій з масивами: створення, властивості, доступ до частин масиву (зрізи), зміна форми, арифметичні операції, математичні функції, способи індексації, збереження у файлах, створення масивів із різнотипними елементами [14].

Приклади визначення одновимірних та двовимірних масивів, матриць із випадковими значеннями:

```
import numpy as np
# створення масивів:
np.array([1.0,2.0,3.0,4.0]) # одновимірний масив
np.array([1,2,3,4], dtype=int) # одновимірний масив цілих чисел
np.array([[1.0,2],[3,4]]) # двовимірний масив дійсних чисел
print(np.array(range(6))) # одновимірний масив із прогресії
np.arange(6) # або так
print(np.linspace(start=0,stop=10,num=5)) # масив із рівномірно
розподіленими значеннями
np.zeros((2,2)) # двовимірний нульовий масив
np.ones(2) # одновимірний масив з одиниць
np.full(2, 1) # або так
np.identity(2) # одинична матриця
np.random.random(5) # масив із випадковими значеннями
np.random.normal(loc=5, scale=1, size=5) # масив із випадковими
значеннями (нормальний закон)
```

Приклади створення списків та доступ до частин масиву (зрізи), розбиття масивів, об'єднання і додавання масивів різного розміру:

```
import numpy as np
a=np.array([[1,2,3], [4,5,6]])
a.ndim # кількість вимірів масиву
a.shape # розмір кожного виміру
a.size # загальний розмір
a.dtype # тип даних
a.tolist() # перетворити у список
# зрізи над масивом у форматі: a[початок:кінець:крок]
print(a[0,1], a[0][1], a[0], a[-1], a[:,0], a[0:2:2,0:3:2])
```

```

a[0,0]=1.2 #змінити елемент з індексами 0,0
#Увага! a[0,0]==1 бо масив цілого типу
a[0]=np.array([1,2,3]) #змінити рядок з індексом 0
a0=a[0] # це не окрема копія першого рядка масиву а
a0[0]=2 #Увага! Масив а зміниться!
a0=a[0].copy() # це окрема копія першого рядка масиву а
print(a.reshape((3,2))) # повертає масив зі зміненою формою
a.shape=(3,2) # або змінити форму масиву
a.resize((3,3)) # змінити форму масиву і заповнити нові комірки нулями
a.transpose() # транспонувати (або а.Т)
np.ones(2)[: , np.newaxis] # перетворити в вектор-стовпчик
np.ones(2).reshape((2,1)) # або так
np.concatenate([a, a]) # об'єднати масиви по вертикалі
np.vstack([a, a]) # або так
np.concatenate([a, a],axis=1) # об'єднати масиви по горизонталі
np.hstack([a, a]) # або так
np.split(a, [1]) # розбити масиви по вертикалі
#див. також np.vsplit, np.hsplit
print(np.array([1,2])+np.array([[1,2],[3,4]])) # додавання масивів
різного розміру

```

Приклади арифметичних операцій над масивами, способи індексації, сортування і змінення форми масиву:

```

import numpy as np
# арифметичні операції над масивами
a+1 # додати 1 до кожного елемента
np.sum(a) # сума елементів
np.sum(a, axis=0) # суми в стовпцях
np.add.reduce(a, axis=0) # або так
np.cumsum(a) # накопичувальна сума
np.mean(a) # середнє
np.std(a) # стандартне відхилення
np.min(a), np.max(a) # мінімальне, максимальне
np.argmin(a) # індекс найменшого елемента
np.sort(np.array([3,2,7,1])) # сортувати
print(np.argsort(np.array([3,2,7,1]))) # індекси для сортування
a=np.array([1,2,3,4])
a[[1,2]] # масив елементів з індексами 1,2
a[np.array([1,2])] # або так
a[[1,2]]=[2,3] # можна також змінювати масив `a`
a[np.array([False, True, True, False])] # або так
np.array([[1,2],[4,5]])[1,[0,1]] # комбінована індексація
b=a<4 # масив із результатами логічного виразу (dtype=bool)
b=(a > 2) & (a < 4) #або складні логічні вирази
a[b] # масив елементів з індексами b
print(np.where(a<4)) # масив індексів, де виконується умова
np.any(a<4) # чи будь-який елемент
np.all(a<4) # чи усі елементи

```

Приклади зберігання у файл і читання з файлу, створення масивів із різнотипними елементами:

```

import numpy as np
# збереження у файлах
a.tofile("myfile") # зберегти у файл

```

```

a=np.fromfile("myfile", dtype=int) # прочитати з файлу
# або
np.save("myfile.npy",a) # зберегти у файл
a=np.load("myfile.npy") # прочитати з файлу
a = np.zeros(2, dtype=('i4,f4,a10')) # масив із різними типами
a[0]=(1, 10.0, 'A') # перший елемент
a[1]=(2, 20.0, 'B') # другий елемент
# або
a = np.zeros(2,
dtype={'names': ('i', 'x', 'name'), 'formats': ('i4', 'f4', 'a10')})
a[0]=(1, 10.0, 'A') # перший елемент
a[1]=(2, 20.0, 'B') # другий елемент
a['name'] # стовпчик 'name'
a['name']=['a','b'] # змінити значення стовпчика
print(a)
print(a[a['x'] < 20]) # ті елементи, де x<20

```

Повна назва бібліотеки NumPy – Numerical Python Extensions, або числові розширення Python. Код бібліотеки NumPy написано мовами C та Fortran, оскільки обчислення на компілюючих мовах відбуваються швидше і ефективніше, ніж на інтерпретуючих мовах, таких як мова Python. Можливості модулю NumPy дозволяють візуалізувати великі за розміром набори даних. Наприклад, бібліотека NumPy лежить в основі системи PyViz, яка включає десятки програм для візуалізації.

Розглянемо дві поширені задачі, які розв’язуються за допомогою бібліотек Math, NumPy та SciPy: рішення системи звичайних диференціальних рівнянь, рішення диференціального рівняння другого порядку, інтерполяція [14].

Рішення системи звичайних диференціальних рівнянь розглянемо на прикладі розв’язання простого диференційного рівняння першого порядку, яке описує струм у RLC-контурі, що у кардіології використовується як модель шунта з опором R (рис. 3.1):

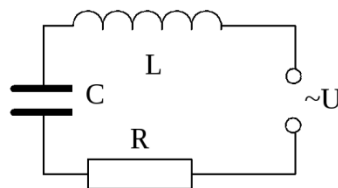


Рисунок 3.1 – Схема RLC-контур

$$C \cdot \frac{dU_c}{dt} = I_0 \cdot \cos\left(\omega \cdot t + \varphi_0 + \frac{\pi}{2}\right), \quad \omega_0 = \sqrt{\frac{1}{L \cdot C}},$$

де L, C – індуктивність і ємність контуру, ω_0 – власна частота коливань, I_0 – амплітудне значення струму.

Визначимо початкові умови: $U_c(0) = 0$, та врахуємо, що при $t \gg \tau$ складова $U_c(\tau) = U_0 \cdot e^{-\beta \cdot t} \cdot \cos(\omega \cdot t)$ прагне до нуля за рахунок перехідного процесу згасання коливань у контурі:

$$\tau = \frac{1}{\beta} = \frac{2 \cdot L}{R},$$

де β – коефіцієнт згасання вимушених коливань, R – опір у контурі.

Фаза напруги на конденсаторі залежить від співвідношення частоти джерела ω і частоти ω_0 :

$$\operatorname{tg}(\varphi) = 2 \cdot \beta \cdot \frac{\omega}{\omega_0^2 - \omega^2}.$$

При резонансі в RLC-контурі величина індуктивного реактивного опору дорівнює величині ємнісного реактивного опору:

$$\omega = \omega_0, \quad \operatorname{tg}(\varphi) \rightarrow \infty, \quad \varphi = \pi/2.$$

Опишемо задачу процесу згасання вимушених коливань у контурі на мові Python:

```
from scipy.integrate import odeint
from math import *
import numpy as np
import matplotlib.pyplot as plt

def carrent(y,t): # функція повертає похідну dy/dt в точці t
    R = 4          # опір, 4 Ом
    L = 8e-2       # індуктивність, 8мГн
    C = 2e-6       # ємність, 2мкФ
    I0 = 2e-2      # струм, 2мА
    B0 = R/(2*L)
    W0 = sqrt(1/(L*C))
    F0 = pi/2
    return -(I0/C)*exp(-B0*t)*cos(W0*t+F0) # значення правої частини
    рівняння

t = np.linspace(0, 0.1, 1000) # час
y = odeint(carrent, y0=0, t=t) # інтегрує диф. рівняння
plt.plot(t,y, 'b-') # рисує залежність y(t)
plt.xlabel('t'); plt.ylabel('I'); plt.grid(); plt.show()
```

Відображення процесу згасання вимушених коливань у RLC-контурі має вигляд (рис. 3.2):

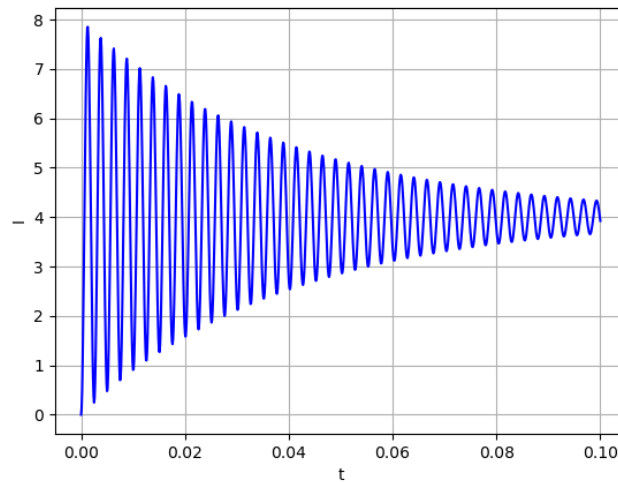


Рисунок 3.2 – Згасання вимушених коливань у RLC-контурі

Рішення диференціальних рівнянь другого порядку розглянемо на прикладі розв’язання однорідного ізотропного рівняння теплопровідності без зовнішніх джерел тепла, яке описує процес розвитку теплоперенесення у відкритому серці в умовах штучного кровообігу:

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) + \frac{1}{c \cdot \rho} \cdot f(x, y, t),$$

де α – температуропровідність, c – питома теплоємність, $f(x, y, t)$ – функція теплового джерела.

Оскільки у тканинах серця відсутні внутрішні джерела тепла, що впливають на процес теплоперенесення на поверхні міокарду, функція джерел $f(x, y, t) = 0$.

Припустимо, що зміна температури у часі є постійною величиною $\partial T / \partial t = \text{const}$, але не однакою за напрямом розповсюдження тепла на поверхні міокарда:

$$\alpha \frac{\partial^2 T}{\partial x^2} = \frac{\partial T}{\partial t} = 1 \text{ } ^\circ\text{C}, \quad \alpha \frac{\partial^2 T}{\partial y^2} = \frac{\partial T}{\partial t} = 7 \text{ } ^\circ\text{C}.$$

Гradient температур на поверхні серця $\partial T / \partial t \geq 7 \text{ }^{\circ}\text{C}$ вважається небезпечним для захисту серця в умовах штучного кровообігу. Тому оцінка однорідності розподілу температури під час кардіоплегії є дуже важливою.

Враховуючи припущення щодо члена $\partial T / \partial t$, можна записати таку систему диференціальних рівнянь для однорідного ізотропного рівняння теплопровідності:

$$\alpha \frac{\partial^2 T}{\partial x^2} = -1, \quad \alpha \frac{\partial^2 T}{\partial y^2} = -7.$$

де знак "—" свідчить про напрямок процесу, що пов'язаний з охолодження під час кардіоплегії.

Перетворимо її в систему диференціальних рівнянь першого порядку, визначивши початкові умови:

$$\begin{aligned} \frac{\partial T'}{\partial x} &= -1 \text{ }^{\circ}\text{C}, & \alpha \frac{\partial T}{\partial x} &= T', \\ \frac{\partial T'}{\partial y} &= -7 \text{ }^{\circ}\text{C}, & \alpha \frac{\partial T}{\partial y} &= T', \\ \frac{\partial T(0)}{\partial x} &= 5 \frac{^{\circ}\text{C}}{\text{хв.}}, & T(0, x) &= 36.6 \text{ }^{\circ}\text{C} \\ \frac{\partial T(0)}{\partial y} &= 5 \frac{^{\circ}\text{C}}{\text{хв.}}, & T(0, y) &= 36.6 \text{ }^{\circ}\text{C} \end{aligned}$$

У початкових умовах прийнято швидкість зміни температури $T' = 5 \text{ }^{\circ}\text{C} / \text{хв.}$, а початкова температура поверхні серця $T(0) = 36.6 \text{ }^{\circ}\text{C}$.

Опишемо задачу розповсюдження тепла на поверхні міокарда під час холодової кардіоплегії на мові Python:

```
from scipy.integrate import odeint
from math import *
import numpy as np
import matplotlib.pyplot as plt

def deriv(xy, t):
    x=xy[0] # переміщення по x
    x_=xy[1] # швидкість по x
    y=xy[2] # переміщення по y
    y_=xy[3] # швидкість по y
    return np.array([x_, -1, y_, -7]) # повертає значення функцій dx/dt,
dx'/dt, dy/dt, dy'/dt
t = np.linspace(0.0, 10.0, 100) # час
```

```

init = np.array([36.6, 5.0, 36.6, 5.0]) # початкові умови для x, x', y, y'
xy = odeint(deriv, init, t) # інтегруємо систему диф. рівнянь, отримуємо масив [x, x', y, y']
plt.plot(xy[:,0], xy[:,2]) # траєкторія
plt.xlabel("x"); plt.ylabel("y"); plt.grid(); plt.show()

```

Відображення розповсюдження тепла на поверхні міокарда під час холодової кардіоплегії має вигляд (рис. 3.3):

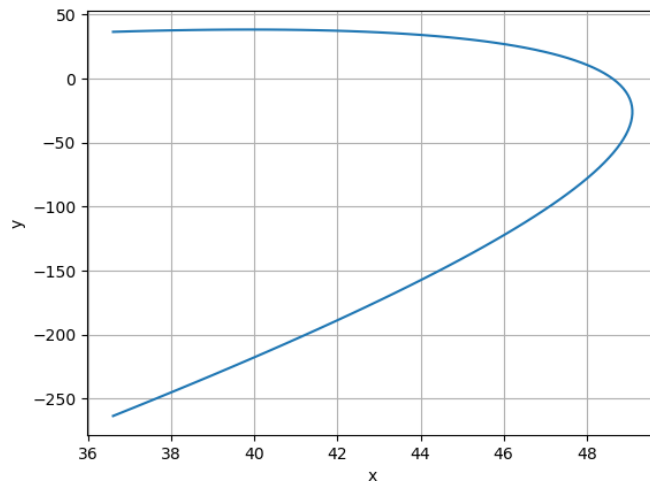


Рисунок 3.3 – Відображення розповсюдження тепла на поверхні міокарда

Проста модель розв'язання однорідного ізотропного рівняння теплопровідності показує, що наявність у часі градієнту температури $\partial T / \partial t \geq 7 \text{ } ^\circ\text{C}$ призводить до значної неоднорідності розповсюдження тепла на поверхні міокарда.

Приклад інтерполяції розглянемо на дискретному наборі експериментальних значень. Спосіб інтерполяції дозволяє знайти проміжні значення величини за її відомим дискретним набором значень.

Для інтерполяції і апроксимації сплайнами застосовують бібліотечні функції з модуля SciPy.interpolate: interp1d, interp2d, UnivariateSpline, SmoothBivariateSpline та інші. Сплайном є функція, область визначення якої розбита на частини, на кожній з яких функція є певним поліномом [14]:

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d, UnivariateSpline, interp2d

```



```

f=lambda x: x**x # функція
x = np.arange(4)
y = f(x)
print(x,y) # дискретний набір значень
y1 = interp1d(x, y, kind='linear') # лінійна інтерполяція
y2 = interp1d(x, y, kind='quadratic') # інтерполяція квадратичним
сплайном
y2 = UnivariateSpline(x, y, k=2, s=0) # або (s - коеф. згладжування)
kn=y2.get_knots() # вузли сплайна
print(y1(2.5), y2(2.5), f(2.5)) # інтерпольовані і дійсне значення в
точці x=2.5

x=np.linspace(0,x.max(),100)
plt.plot(x,f(x), 'k-', x, y1(x), 'b--', x, y2(x), 'r:') # графіки
plt.scatter(kn, y2(kn)) # вузли кв. сплайна
plt.xlabel('x'), plt.ylabel('y'), plt.grid(), plt.show()

f=lambda x,y: x**2+y**2 # функція двох змінних x=np.array([0,1,2])
y=np.array([0,1,2])
xx, yy = np.meshgrid(x,y) # сітка
z=f(xx,yy) # значення функції у вузлах сітки
z1 = interp2d(x, y, z, kind='linear') # двовимірна лінійна інтерполяція
даних x,y,z
print(z1(0.5,0.5), f(0.5,0.5)) # інтерпольоване і дійсне значення в
точці 0.5, 0.5

```

Візуалізація лінійної інтерполяції $y_1(x)$ і апроксимації квадратичним сплайном $y_2(x)$ простої демонстраційної функції $f(x)$ має вигляд (рис. 3.4):

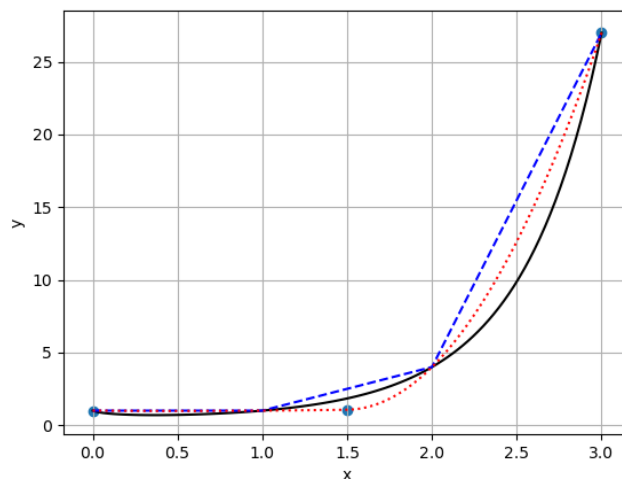


Рисунок 3.4 – Візуалізація інтерполяції і апроксимації

Отже, апроксимація кривої в Python є видом оптимізації, який знаходить оптимальний набір параметрів для певної функції, що підходить для набору спостережень.

3.2. Порядок роботи

Частина 1. Відображення математичної функції у графічному вікні бібліотеки PySimpleGUI.

(a) Створити структуру layout, яка містить шкалу прогресу ProgressBar і вивести зображення шкали у графічному вікні:

```
# layout the Window
layout = [[sg.Text('A custom progress meter')],
          [sg.ProgressBar(BAR_MAX, orientation='h', size=(20,20),
key='-PROG-')], [sg.Cancel()]]
# create the Window
window = sg.Window('Custom Progress Meter', layout)
```

Додати обробку події event під час натиснення на кнопку 'Cancel', яка очікує на дію timeout=10 хв. та виконує метод sg.WIN_CLOSED закриття графічного вікна:

```
event, values = window.read(timeout=10)
if event == 'Cancel' or event == sg.WIN_CLOSED:
    break
```

Програмний код відображення шкали прогресу ProgressBar у графічному вікні бібліотеки PySimpleGUI має вигляд:

```
import PySimpleGUI as sg

sg.theme('Dark Red')

BAR_MAX = 1000

# layout the Window
layout = [[sg.Text('A custom progress meter')],
          [sg.ProgressBar(BAR_MAX, orientation='h', size=(20,20),
key='-PROG-')],
          [sg.Cancel()]]

# create the Window
window = sg.Window('Custom Progress Meter', layout)
# loop that would normally do something useful
for i in range(1000):
    # check to see if the cancel button was clicked and exit loop if
    clicked
    event, values = window.read(timeout=10)
    if event == 'Cancel' or event == sg.WIN_CLOSED:
        break
    # update bar with loop value +1 so that bar eventually
    reaches the maximum
    window['-PROG-'].update(i+1)
```

```
# done with loop... need to destroy the window as it's still open
window.close()
```

Вивід шкали прогресу ProgressBar у графічному вікні PyCharm бібліотеки PySimpleGUI:

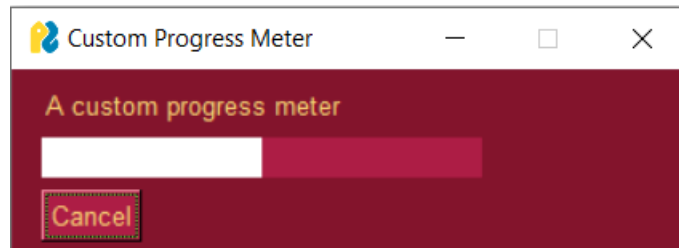


Рисунок 3.5 – Шкала прогресу ProgressBar

(b) Створити структуру layout, яка містить список Listbox і вивести зображення списку в графічному вікні:

```
# layout the Window
layout = [sg.vtop([sg.Listbox(list(range(10)), size=(5,5)),
sg.Multiline(size=(25,10))]),
[sg.Button('Ok'), sg.Button('Cancel')]]
# create the Window
window = sg.Window('Element Alignment', layout)
```

Додати обробку події event під час натиснення на кнопку 'Cancel', яка виконує метод sg.WIN_CLOSED закриття графічного вікна:

```
event, values = window.read()
if event == 'Cancel' or event == sg.WIN_CLOSED:
    break
```

Програмний код вивода списку Listbox у графічному вікні бібліотеки PySimpleGUI має вигляд:

```
import PySimpleGUI as sg

layout = [sg.vtop([sg.Listbox(list(range(10)), size=(5,5)),
sg.Multiline(size=(25,10))]),
[sg.Button('Ok'), sg.Button('Cancel')]]

window = sg.Window('Element Alignment', layout)

while True:
    event, values = window.read()
    if event in (sg.WIN_CLOSED, 'Cancel'):
        break
window.close()
```

Вивід списку Listbox у графічному вікні PyCharm бібліотеки PySimpleGUI (рис. 3.6):

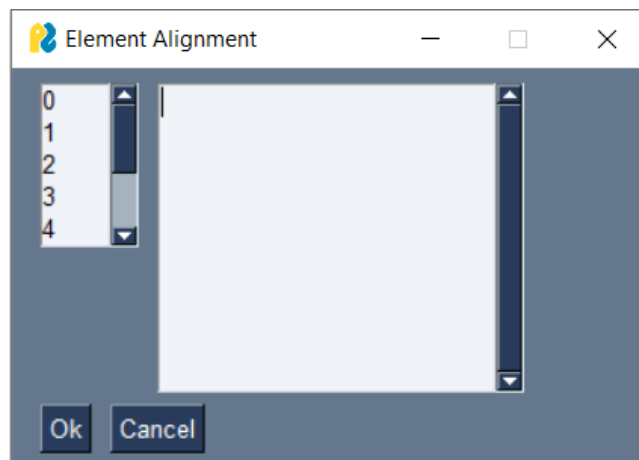


Рисунок 3.6 – Список Listbox

(с) Підключити бібліотеки Math та PySimpleGUI та створити структуру layout, яка містить графічне вікно Graph для виводу графіка математичної функції:

```
import math
import PySimpleGUI as sg

layout = [[sg.Graph(canvas_size=(400, 400), graph_bottom_left=(-105,-105), graph_top_right=(105,105), background_color='white',
key='graph', tooltip='This is a cool graph!')],]

window = sg.Window('Graph of Sine Function', layout,
grab_anywhere=True, finalize=True)
graph = window['graph']
```

Згенерувати просту математичну функцію за допомогою методів з бібліотеки Math, значення якої записати до векторів (x, y), що мають розмірність (-100, 100), та відобразити їх у графічному вікні за допомогою метода graph.DrawCircle(), з бібліотеки PySimpleGUI:

```
# Draw Graph
for x in range(-100,100):
    y = math.sin(x/20)*50
    graph.DrawCircle((x,y), 1, line_color='red', fill_color='red')
```

Програний код виводу графічного вікна Graph для відображення математичної функції:

```
import math
import PySimpleGUI as sg

layout = [[sg.Graph(canvas_size=(400, 400), graph_bottom_left=(-105, -105), graph_top_right=(105, 105), background_color='white', key='graph', tooltip='This is a cool graph!')],]

window = sg.Window('Graph of Sine Function', layout, grab_anywhere=True, finalize=True)
graph = window['graph']

# Draw axis
graph.DrawLine((-100, 0), (100, 0))
graph.DrawLine((0, -100), (0, 100))

for x in range(-100, 101, 20):
    graph.DrawLine((x, -3), (x, 3))
    if x != 0:
        graph.DrawText(x, (x, -10), color='green')

for y in range(-100, 101, 20):
    graph.DrawLine((-3, y), (3, y))
    if y != 0:
        graph.DrawText(y, (-10, y), color='blue')

# Draw Graph
for x in range(-100, 100):
    y = math.sin(x/20)*50
    graph.DrawCircle((x, y), 1, line_color='red', fill_color='red')

event, values = window.read()
```

Вивід графічного вікна Graph для відображення функції з бібліотеки PySimpleGUI (рис. 3.7):

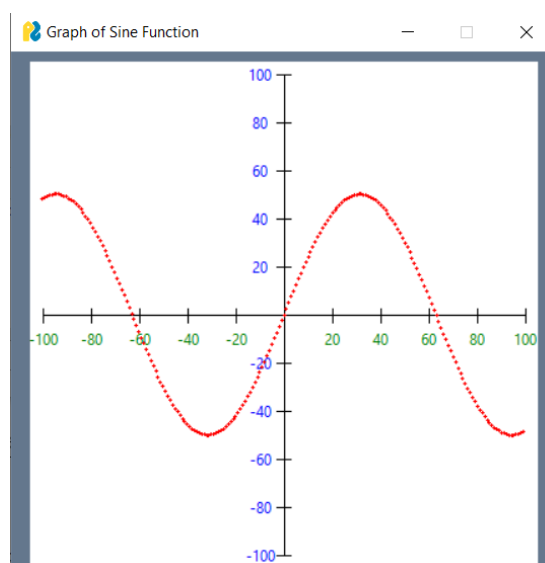


Рисунок 3.7 – Відображення функції у графічному вікні Graph

Частина 2. Рішення системи алгебраїчних рівнянь із спостереженням у часі адаптації рішення для модельної функції з використанням модулю роботи з часом Time.

Підключити бібліотеки NumPy і SciPy та реалізувати модельну функцію $f'(x^0)(x^{k+1} - x^k) + f(x^k) = 0, \quad k=0, \dots, n$, яка описує масив з $n = 10$ диференційних рівнянь:

```
from numpy import*
from scipy import optimize
import time

def f(x):
    n = 10
    f = zeros([n])
    t = zeros([n])
    ti = time.clock()
    for i in arange(0,n-1,1):
        f[i] = (3 + 2*x[i])*x[i] - x[i-1] - 2*x[i+1] - 2
    f[0] = (3 + 2*x[0])*x[0] - 2*x[1] - 3
    f[n-1] = (3 + 2*x[n-1])*x[n-1] - x[n-2] - 4
    t[i] = round(time.clock()-ti, 5)
    print('time = ', t[i], ' second, Solution = ', f[i], ' \n')
    return f
```

Програмно реалізувати метод Крилова-Боголюбова відшукування коренів нелінійних диференційних рівнянь на модельній функції $f'(x^0)(x^{k+1} - x^k) + f(x^k) = 0, \quad k=0, \dots, n$ за допомогою бібліотечної функції оптимізації Optimize.root:

```
def main():
    n = 10
    x0 = zeros([n])
    sol = optimize.root(f,x0, method='krylov')
```

У підсумку програма для тестування методу Крилова-Боголюбова відшукування коренів системи диференційних рівнянь на модельній функції має вигляд:

```
from numpy import*
from scipy import optimize
import time

def f(x):
    n = 10
    f = zeros([n])
```

```

t = zeros([n])
ti = time.clock()
for i in arange(0,n-1,1):
    f[i] = (3 + 2*x[i])*x[i] - x[i-1] - 2*x[i+1] - 2
f[0] = (3 + 2*x[0])*x[0] - 2*x[1] - 3
f[n-1] = (3 + 2*x[n-1])*x[n-1] - x[n-2] - 4
t[i] = round(time.clock()-ti, 5)
print('time = ', t[i], ' second, Solution = ', f[i], ' \n')
return f

def main():
    n = 10
    x0 = zeros([n])
    sol = optimize.root(f,x0, method='krylov')
    print('Solution:\n', sol.x)
    print('Krylov method iteration = ', sol.nit)
    print('Optimize root time', round(time.clock()-ti,3), 'seconds')

if __name__ == "__main__":
    main()

```

Зі збільшенням числа рівнянь помітна поява помилок у рішенні. При подальшому збільшенні n рішення стає неприйнятним, що вимагає автоматичної адаптацію кроку:

```

time = 0.0 second, Solution = -2.0
time = 0.001 second, Solution = -1.999999998026293
time = 0.002 second, Solution = -1.9999999981421202
...
time = 0.835 second, Solution = 1.213331266184389
time = 0.836 second, Solution = -0.3454996594622002

Solution:
[-1.54433496 -1.50795506 -0.22960679 -0.68288945 -1.80252485
 0.00756788 0.04772384 -0.92049982 -1.93313104 0.47029879]
Krylov method iteration = 1100
Optimize root time 0.836 seconds.

```

3.3. Завдання для самостійної роботи

Частина 1. Відображення математичної функції у графічному вікні бібліотеки PySimpleGUI.

Реалізувати послідовний вивід у графічне вікно бібліотеки PySimpleGUI списку Listbox та графічного вікна Graph для відображення математичної функції (рис. 3.8):

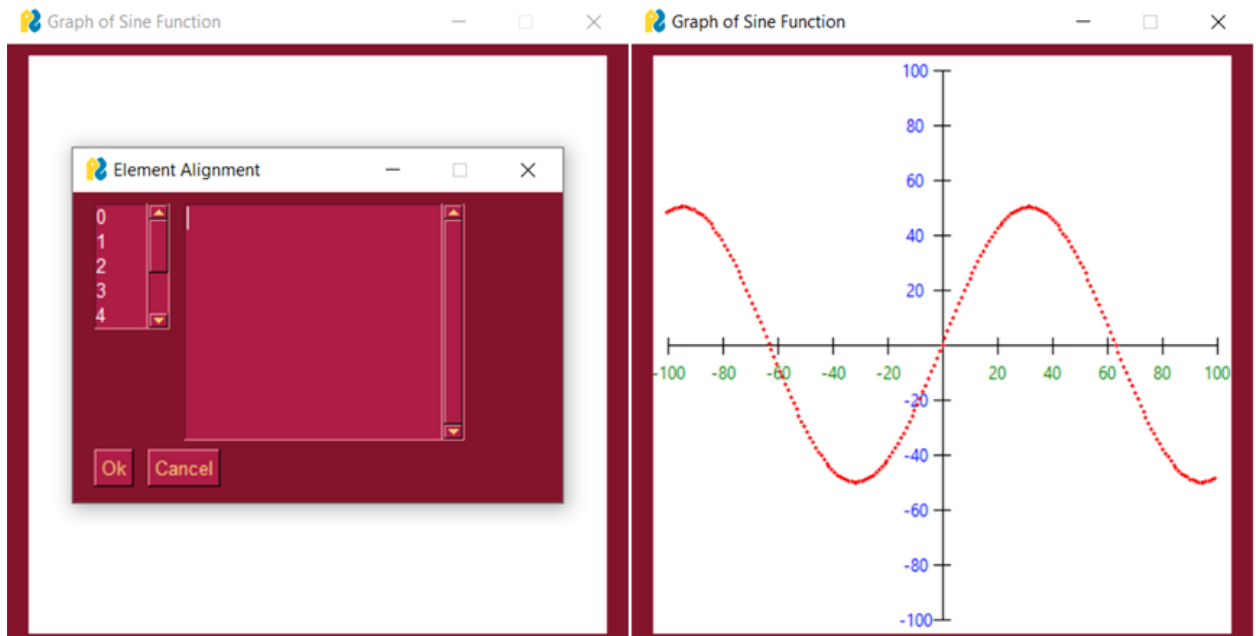


Рисунок 3.8 – Список Listbox та графічне вікно Graph

Під час виконання самостійного завдання у програмному коді реалізувати таку схему побудови графічного інтерфейсу (GUI):

```
# layout the Window
layout_1 = [[sg.Text('A custom progress meter')],
            [sg.ProgressBar(BAR_MAX, orientation='h', size=(20,20),
key='-PROG-')],
            [sg.Cancel()]]
layout_2 = [sg.vtop([sg.Listbox(list(range(10)), size=(5,5)),
sg.Multiline(size=(25,10))]),
            [sg.Button('Ok'), sg.Button('Cancel')]]
layout_3 = [[sg.Graph(canvas_size=(400, 400), graph_bottom_left=(-
105,-105), graph_top_right=(105,105),
background_color='white', key='graph', tooltip='This is a
cool graph!'))]]

# create the Window
window_1 = sg.Window('Custom Progress Meter', layout_1)
window_2 = sg.Window('Element Alignment', layout_2)
window_3 = sg.Window('Graph of Sine Function', layout_3,
grab_anywhere=True, finalize=True)
graph = window_3['graph']
```

Частина 2. Рішення системи алгебраїчних рівнянь із використанням модулю роботи з часом Time.

Реалізувати вивід у графічне вікно Graph бібліотеки PySimpleGUI відображення модельної функції $f'(x^0)(x^{k+1} - x^k) + f(x^k) = 0$, $k = 0, \dots, n$, яка описує масив з $n = 10$ диференціальних рівнянь:


```

import time
from numpy import *
from scipy import optimize
import PySimpleGUI as sg
import math

layout = [[sg.Graph(canvas_size=(400, 400), graph_bottom_left=(-105,-105), graph_top_right=(105,105), background_color='white', key='graph', tooltip='This is a cool graph!')],]
window = sg.Window('Graph of Sine Function', layout, grab_anywhere=True, finalize=True)
graph = window['graph']

def f(tx):
    n = 10
    f = zeros([n])
    t = zeros([n])
    ti = time.clock()
    for i in range(0, n - 1, 1):
        f[i] = (3 + 2 * tx[i]) * tx[i] - tx[i - 1] - 2 * tx[i + 1] -
2
    f[0] = (3 + 2 * tx[0]) * tx[0] - 2 * tx[1] - 3
    f[n - 1] = (3 + 2 * tx[n - 1]) * tx[n - 1] - tx[n - 2] - 4
    t[i] = round(time.clock() - ti, 5)
    print('time = ', t[i], ' second, Solution = ', f[i], ' \n')
    graph.DrawCircle((t[i]*5e+5, f[i]), 1, line_color='red', fill_color='red')
    return f

# Draw axis
graph.DrawLine((-100,0), (100,0))
graph.DrawLine((0,-100), (0,100))

for x in range(-100, 101, 20):
    graph.DrawLine((x,-3), (x,3))
    if x != 0:
        graph.DrawText( x, (x,-10), color='green')

for y in range(-100, 101, 20):
    graph.DrawLine((-3,y), (3,y))
    if y != 0:
        graph.DrawText( y, (-10,y), color='blue')

n = 10
x0 = zeros([n])
sol = optimize.root(f, x0, method='krylov')
print('Solution:\n', sol.x)
print('Function:\n', sol.fun)
print('Krylov method iteration = ', sol.nit)
#print('Optimize root time', round(time.clock() - ti, 3), 'seconds')

event, values = window.read()

```

Відображення модельної функції відшукування коренів нелінійних диференційних рівнянь у графічному вікні бібліотеки PySimpleGUI показує збіжність рішення ~65 мс. (рис. 3.9):

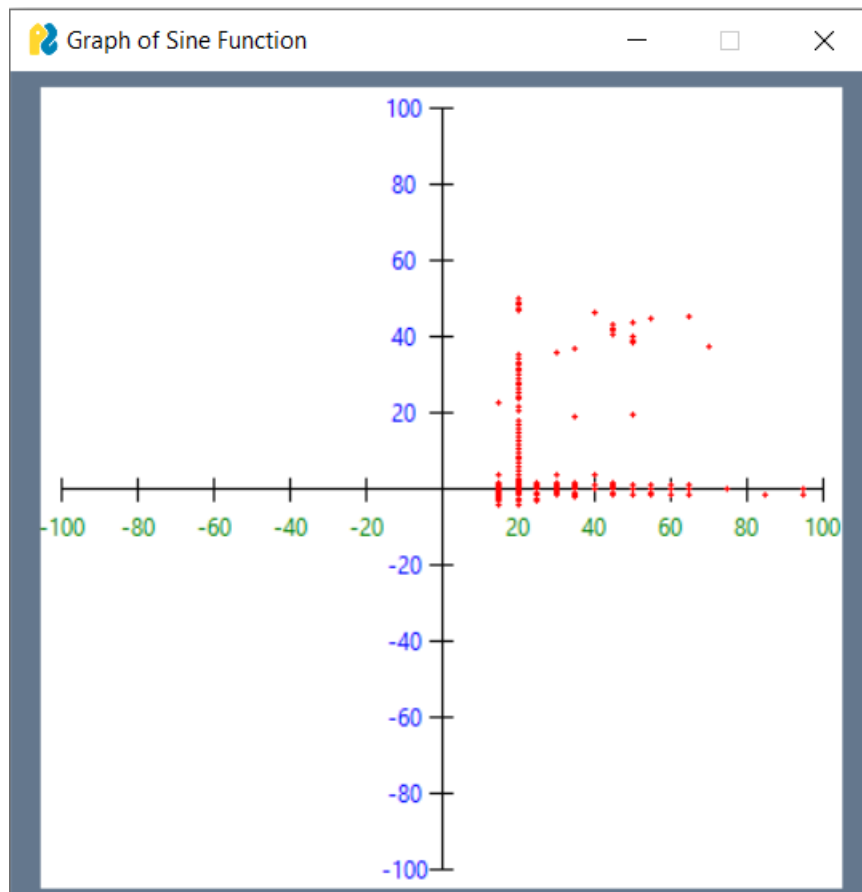


Рисунок 3.9 – Відображення модельної функції

Автоматична адаптація кроку під час відшукування коренів рівняння дає наступні значення функції оптимізації `Optimize.root`:

```
Solution:
[-1.54433496 -1.50795506 -0.22960679 -0.68288945 -1.80252485
 0.00756788
 0.04772384 -0.92049982 -1.93313104 0.47029879]
Function:
[ 0.15284619 0.0275403 0.29035215 0.71866414 -0.24162919 -
0.27010465
-0.01884159 0.75167862 -0.34549966 -0.21361068]
Krylov method iteration = 1100
```

3.4. Контрольні питання

1. Які можливості стандартної математичної бібліотеки `Math`?
2. Які засоби для обробки часу і дати має модуль `time`?
3. Які можливості стандартної бібліотеки `NumPy`?
4. Що таке інтерполяції і апроксимації даних?

ПРАКТИЧНА РОБОТА № 4

ПОБУДОВА ГРАФІКІВ ТА ДІАГРАМ

З ВКОРИСТАННЯМ ФУНКЦІЙ БІБЛІОТЕКИ MATPLOTLIB

Мета заняття: набути навичок дослідження математичних функцій із застосуванням функцій бібліотеки Matplotlib використовуючи мову програмування Python.

4.1. Основні теоретичні відомості

Для візуалізації при використанні SciPy часто використовують бібліотеку Matplotlib (<https://matplotlib.org/stable/gallery/index.html>). Переваги Matplotlib полягають у тому, що вона використовує Python, і це дає змогу застосовувати кожен зі стандартних або інших доступних бібліотек; вона поширюється за вільною ліцензією, що є важливим для вчених та студентів.

Однією з найбільших переваг Matplotlib є швидкість, з якою можна побудувати графік математичної функції. Розглянемо приклад візуалізації табличного графіку функції:

```
import matplotlib.pyplot as plt
plt.plot([3.3, 4.4, 4.5, 6.5], [3., 5., 6., 7.])
plt.show()
```

У першій інструкції було імпортовано основний модуль бібліотеки matplotlib.pyplot для побудови графіків під іменем plt. Після імпортування модуля можна користуватися його функціями. Тут застосовується функція plot(), яка будує графік, а потім функція show(), яка його показує. Аргументи для функції plot() – це послідовність x-значень та y-значень. Якщо серед аргументів будуть відсутні x-значення, то за замовчанням вони будуть заміщені списком x[0, 1, 2, 3].

Природно використовувати дві координати разом та замість списків – масиви. Наприклад, можна побудувати графік складної математичної функції $y(t) = t^2 \exp(-t^2)$. Програму наведено нижче [15]:

```
from numpy import *
import matplotlib.pyplot as plt

def f(t):
    return t**2*exp(-t**2)

t = linspace(0, 3, 51) # 51 точка між 0 і 3
y = f(t)

plt.plot(t, y)
plt.show()
```

Крім того, щоб побудувати криву $f(t)$, необхідно її назвати, позначити осі, вивести легенду, що особливо знадобиться, коли потрібно буде будувати декілька графіків. Можна також змінити вигляд самої кривої або відрізка часу на якому виводиться графік:

```
from numpy import *
import matplotlib.pyplot as plt

t = linspace(0, 3, 51)
y = t**2*exp(-t**2)

plt.plot(t, y, 'g--', label='t^2*exp(-t^2)')
plt.axis([0, 3, -0.05, 0.5]) # визначення [xmin, xmax, ymin, ymax]
plt.xlabel('t') # позначення осі абсцис
plt.ylabel('y') # позначення осі ординат
plt.title('Normal plot') # назва графіка
plt.legend() # вставка легенди (тексту в label)

plt.show()
```

Часто зустрічаються завдання, у яких окрім візуалізації функції, потрібно ще порівняти її з іншою функцією або навіть декількома функціями. Наприклад, можна порівняти вже відому функцію $y(t) = t^2 \exp(-t^2)$ з деякою схожою функцією $y(t) = t^4 \exp(-t^2)$. Для цього достатньо додати ще одну інструкцію plot():

```
from numpy import *
import matplotlib.pyplot as plt

t = linspace(0, 3, 51)
y1 = t**2*exp(-t**2)
y2 = t**4*exp(-t**2)
```

```
# декоративна частина
plt.plot(t, y1, 'g^--', label='t^2*exp(-t^2)')
plt.plot(t, y2, 'bo-', label='t^4*exp(-t^2)')
plt.xlabel('t')
plt.ylabel('y')
plt.title('Plotting two curves in the same plot')
plt.legend()
plt.show()
```

У Matplotlib є можливість керувати цим процесом використовуючи різний стиль ліній. Крім того, буває потрібно показувати не згладжену криву, а самі точки з цієї кривої. Для цього використовується синтаксис маркерів, у загальному – такий самий, що і для ліній [15]:

```
from numpy import *
import matplotlib.pyplot as plt

t = linspace(0, 3, 51)
y1 = t**2*exp(-t**2)
y2 = t**4*exp(-t**2)
y3 = t**6*exp(-t**2)

plt.plot(t, y1, 'g^', # маркери із зелених трикутників
t, y2, 'b--', # синя штрихова
t, y3, 'ro-') # червоні круглі маркери,
# з'єднані суцільною лінією
plt.xlabel('t')
plt.ylabel('y')
plt.title('Plotting with markers')
plt.legend(['t^2*exp(-t^2)',
't^4*exp(-t^2)',
't^6*exp(-t^2)'], # список легенди
loc='upper left') # позиція легенди

plt.show()
```

Візуалізація порівняння трьох графіків функцій має вигляд (рис. 4.1):

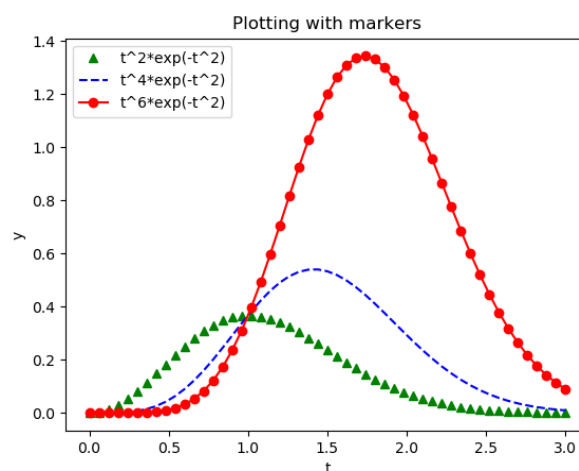


Рисунок 4.1 – Візуалізація порівняння трьох графіків функцій

Цей приклад (рис. 4.1) показує, як можна поєднувати відразу три графіки в одній інструкції. Крім того, видно, що можна не лише використовувати маркери (y1) або лінії (y2), але й поєднувати їх разом (y3).

Бібліотека Matplotlib дає можливість змінювати все у вікні графіка, і навіть написи за його межами. Самостійно Matplotlib підбирає лише осі. Керувати осями можна за допомогою функцій `xticks()` та `yticks()`, до яких передаються один або два списки значень:

```
import matplotlib.pyplot as plt
x = [5, 3, 7, 2, 4, 1]
plt.xticks(range(len(x)), ['a', 'b', 'c', 'd', 'e', 'f'])
plt.yticks(range(1, 8, 2))
plt.show()
```

Зберігати файли можна використовуючи панель під рисунком, але також можна досить легко запрограмувати автоматичне збереження результатів у файлі:

```
from numpy import *
import matplotlib.pyplot as plt

t = linspace(0, 3, 51)
y = t**2*exp(-t**2)

plt.plot(t, y)
plt.savefig('name_of_plot.png', dpi=200)
plt.show()
```

Файл зберігається в тому самому каталозі з іменем та розширенням, указаним у першому аргументі. Другий необов'язковий аргумент `dpi` дозволяє міняти роздільну здатність картинки, що зберігається у файл.

Модуль Matplotlib дає можливість будувати гістограми табличних функцій. Гістограма є спосіб графічного подання табличних даних, у якому кількісні співвідношення деякого показника мають вигляд прямокутників, площі яких пропорційні внеску. Для ілюстрації цього визначення та можливості отримати гістограму розглянемо таку програму:

```
from numpy import *
import matplotlib.pyplot as plt
import numpy as np
```

```
y = np.random.randn(1000)
plt.hist(y, 25)
plt.show()
```

Візуалізація гістограми для згенерованих випадкових значень функції $y()$ має вигляд (рис. 4.2):

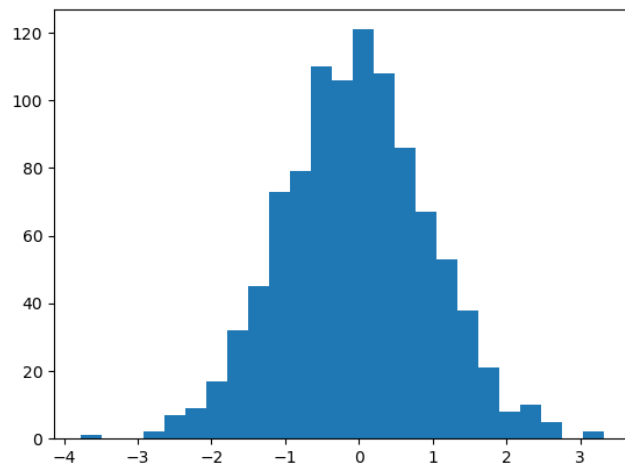


Рисунок 4.2 – Візуалізація гаусівського розподілу

В інструкції `np.random.randn(1000)` створюється масив із випадкових точок відповідно до гауссового розподілу. Як видно, на відміну від раніше використовуваної функції `plot()` для кривих, застосовується `hist()` (histogram). Першим аргументом вона приймає масив чисел, другим необов'язковим аргументом є кількість смуг, на які буде розбито масив. За замовчуванням це число дорівнює десяти, проте у прикладі задано – 25.

У практичній науці експеримент навіть за максимальної точності вимірювань завжди вносить похибки. Для того, щоб врахувати це, і указати можливий розкид навколо значення, яке вважається істинним, вводять діапазон помилок (error bars), який на кривій для отриманих точок показує своєрідний довірчий інтервал [15]:

```
from numpy import *
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(0, 4, 0.2)
```

```

y = np.exp(-x)
e1 = 0.1 * np.abs(np.random.randn(len(y)))

plt.errorbar(x, y, yerr=e1, fmt='.-')
plt.show()

```

Візуалізація довірчих інтервалів при визначенні експериментальної функції має вигляд (рис. 4.3):

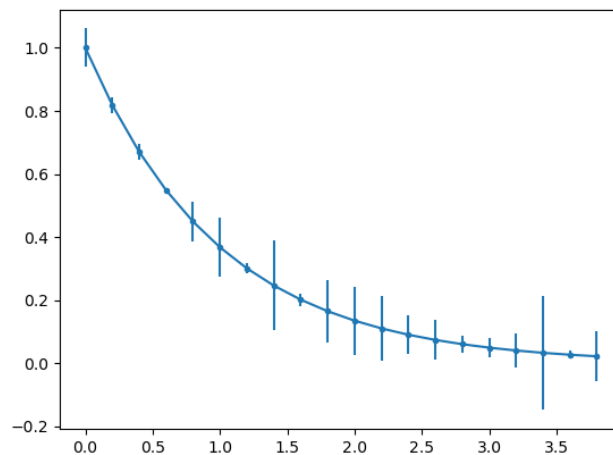


Рисунок 4.3 – Візуалізація довірчих інтервалів

За допомогою функції `arange()` створено масив точок x від 0 до 4 із кроком 0,2. Далі отримано масив точок y , відповідальних за значення функції $y(x)$. Наступна команда вносить випадкові помилки: визначається розмір масиву y , для нього встановлюється відповідний масив розподілених за гауссовим розподілом навколо нуля випадкових чисел, функція `abs()` бере від них модуль так, щоб усі числа у масиві були додатними, а також домножує їх на 0,1 таким чином, щоб отримані помилки не були занадто великими для отриманого графіка.

Далі одержана послідовність указується у функції `errorbar()` у вигляді аргументу `yerr` та послідовно накладається на отриманий графік $y(x)$, вигляд якого визначається параметром `fmt`. Якщо помилки у експерименті є не тільки для y , але і для x , то використовується еквівалентний аргумент `xerr`. Відповідні "зарубки" на графіку будуть мати вигляд хрестів.

Можливі також й несиметричні відхилення, які можна вказати в тій самій функції за допомогою списку з двох послідовностей: першої – для від'ємних відхилень, і другої – для додатних відхилень від істинних значень. Зробити це можна у такій формі для функції `errorbar()`:

```
from numpy import *
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(0, 4, 0.2)
y = np.exp(-x)

e1 = 0.1 * np.abs(np.random.randn(len(y)))
e2 = 0.1 * np.abs(np.random.randn(len(x)))

plt.errorbar(x, y, yerr=[e1,e2], fmt='.-')
plt.show()
```

Візуалізація довірчих інтервалів за координатами x та y при визначенні експериментальної функції має вигляд (рис. 4.4):

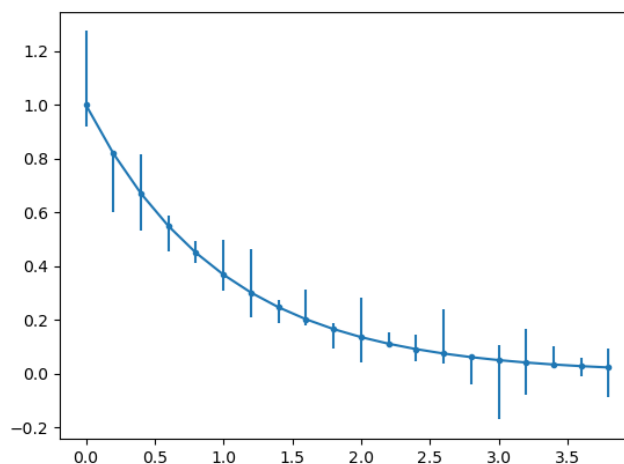


Рисунок 4.4 – Візуалізація довірчих інтервалів за координатами x та y

Переданий аргумент *fmt* може мати значення `None`, тоді на екран виводяться тільки довірчі інтервали (error bars), без графіка функції $y(x)$. Функція `errorbar()` також має інші параметри: *ecolor* та *elinewidth*, які визначають відповідно колір та товщину лінії, що показує інтервал; *capsize* визначає ширину обмежувальних "кришечок" (у пікселях).

Для визначення зв'язку між двома величинами, що дозволяє знайти більш точні межі вимірювань, використовують графік розсіювання. Відображення графіку розсіювання дозволяє зображувати одночасно дві множини даних, які не утворюють криву, а саме – двомірну множину точок, у якій кожна точка має дві координати.

Для створення таких графіків у модулі Matplotlib.pyplot є функція `scatter()`. Ця функція приймає дві послідовності та зображує їх на площині у вигляді маркерів (за замовчуванням, вони круглі та сині). Однак, їх можна змінювати за допомогою параметрів цієї функції [15]: *s* – визначає розмір маркерів, який може бути як одним числом для всіх, так і містити масив значень; *c* – визначає колір маркерів (колір може бути одним для всіх або вказуватись множиною); *marker* – визначає тип маркера.

```
from numpy import *
import matplotlib.pyplot as plt
import numpy as np

x = np.random.randn(1000)
y = np.random.randn(1000)

size = 50*np.random.randn(1000)
colors = np.random.rand(1000)

plt.scatter(x, y, s=size, c=colors)
plt.show()
```

Візуалізація двох послідовностей на площині має вигляд (рис. 4.5):

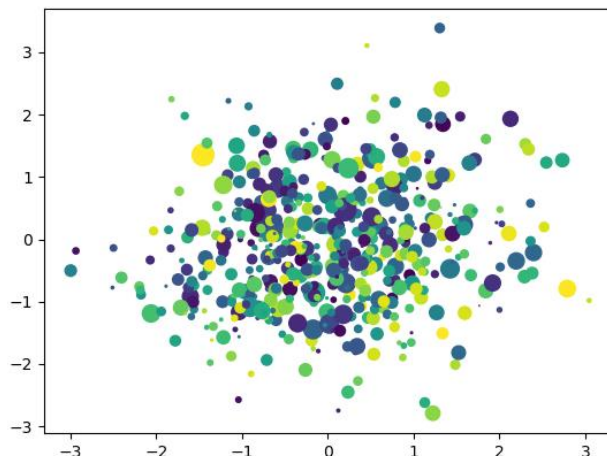


Рисунок 4.5 – Візуалізація двох послідовностей на площині

Для побудови тривимірних графіків у програмі PyCharm є можливість використати графічну бібліотеку Axes3D. Для підтримки бібліотеки Axes3D необхідно у програмі PyCharm перейти у меню Sketch → "+", знайти у вікні Available Packages пакет mplot3D-Drawer і встановити його натиснувши кнопку Install Packages (рис. 1.9).

У бібліотеці Axes3D є інструменти mplot3d для побудови різних типів графіків: лінійний 3D-графік, точковий 3D-графік, поверхня і каркасна поверхня. Розглянемо приклад побудови лінійного 3D-графіку:

```
import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d import Axes3D # import Axes3D

# ... your plotting in Axes3D
x = np.linspace(-np.pi, np.pi, 50)
y = x
z = np.cos(x)
fig = plt.figure()
ax = Axes3D(fig)
ax.plot(x, y, z, label='parametric curve')

# show()
plt.show()
```

Візуалізація лінійного 3D-графіку з використанням бібліотеки Axes3D (рис. 4.6):

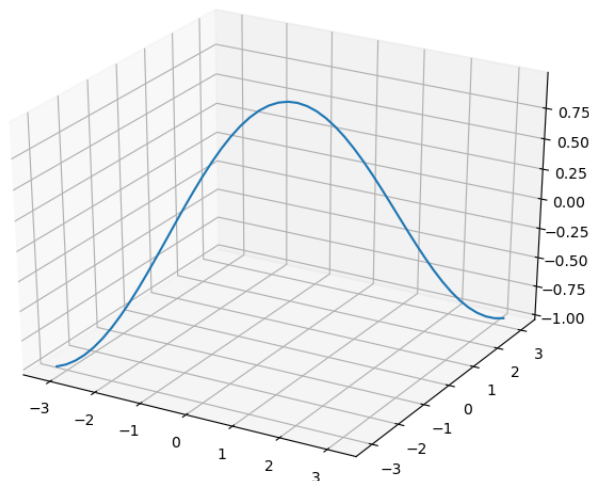


Рисунок 4.6 – Лінійний 3D-графік

Для побудови поверхні необхідно визначити функцію вздовж осі Z, наприклад, за координатою z можна задати функцію виду $z = \sin(\sqrt{x^2 + y^2})$,

яку відобразимо на 3D-графіку у діапазоні `set_zlim(-1.01, 1.01)`. У підсумку програмний код буде мати вигляд [5]:

```
import matplotlib.pyplot as plt
from matplotlib import cm
from matplotlib.ticker import LinearLocator, FormatStrFormatter
from mpl_toolkits.mplot3d import Axes3D # import Axes3D
import numpy as np

# ... your plotting in Axes3D
x = np.linspace(-np.pi, np.pi, 50)
y = x
z = np.cos(x)
fig = plt.figure()
ax = Axes3D(fig)

# Plot the surface.
X, Y = np.meshgrid(x, y)
R = np.sqrt(X**2 + Y**2)
Z = np.sin(R)
surf = ax.plot_surface(X, Y, Z, cmap=cm.coolwarm,
                      linewidth=0, antialiased=False)

# Customize the z axis.
ax.set_zlim(-1.01, 1.01)
ax.zaxis.set_major_locator(LinearLocator(10))
ax.zaxis.set_major_formatter(FormatStrFormatter('%.02f'))

# Add a color bar which maps values to colors.
fig.colorbar(surf, shrink=0.5, aspect=5)

# show()
plt.show()
```

Візуалізація поверхні на 3D-графіку з використанням бібліотеки `Axes3D` має вигляд (рис. 4.7):

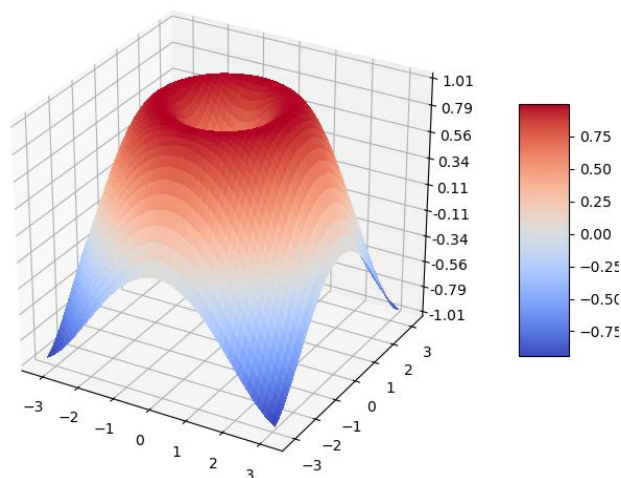


Рисунок 4.7 – Візуалізація поверхні на 3D-графіку

4.2. Порядок роботи

Частина 1. Використання бібліотеки Matplotlib для графічного виводу результатів математичних обчислень.

Підключіть бібліотеку Random та Matplotlib для генерації випадкових чисел `randint()` і графічного відображення результатів `plot()`:

```
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg,
FigureCanvasAgg
from matplotlib.figure import Figure
import matplotlib.backends.tkagg as tkagg

...
dpts = [randint(0, 10) for x in range(10000)]
...
for i in range(len(dpts)):
    ...
    ax.plot(range(20), dpts[i:i + 20], color='purple')
    graph.draw()
```

У підсумку програмний код генерації випадкових чисел функцією `randint()` із бібліотеки Random та відображення отриманих результатів у графічному вікні `Figure()` з бібліотеки Matplotlib:

```
from random import randint
import PySimpleGUI as sg
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg,
FigureCanvasAgg
from matplotlib.figure import Figure
import matplotlib.backends.tkagg as tkagg
import tkinter as Tk

fig = Figure()

ax = fig.add_subplot(111)
ax.set_xlabel("X axis")
ax.set_ylabel("Y axis")
ax.grid()

layout = [[sg.Text('Animated Matplotlib', size=(40, 1),
justification='center', font='Helvetica 20')],
          [sg.Canvas(size=(640, 480), key='canvas')],
          [sg.Button('Exit', size=(10, 2), pad=((280, 0), 3),
font='Helvetica 14')]]

# create the window and show it without the plot

window = sg.Window('Demo Application - Embedding Matplotlib In
```

```

PySimpleGUI', layout, finalize=True)
# needed to access the canvas element prior to reading the window

canvas_elem = window['canvas']

graph = FigureCanvasTkAgg(fig, master=canvas_elem.TKCanvas)
canvas = canvas_elem.TKCanvas

dpts = [randint(0, 10) for x in range(10000)]
# Our event loop
for i in range(len(dpts)):
    event, values = window.read(timeout=20)
    if event == 'Exit' or event == sg.WIN_CLOSED:
        exit(69)

    ax.cla()
    ax.grid()

    ax.plot(range(20), dpts[i:i + 20], color='purple')
    graph.draw()
    figure_x, figure_y, figure_w, figure_h = fig.bbox.bounds
    figure_w, figure_h = int(figure_w), int(figure_h)
    photo = Tk.PhotoImage(master=canvas, width=figure_w,
height=figure_h)

    canvas.create_image(640 / 2, 480 / 2, image=photo)

    figure_canvas_agg = FigureCanvasAgg(fig)
    figure_canvas_agg.draw()

    tkagg.blit(photo, figure_canvas_agg.get_renderer()._renderer,
colormode=2)

```

Відображення результатів генерації випадкових чисел функцією `randint()` із бібліотеки `Random` у графічному вікні `Figure()` з математичної бібліотеки `Matplotlib` (рис. 4.8):

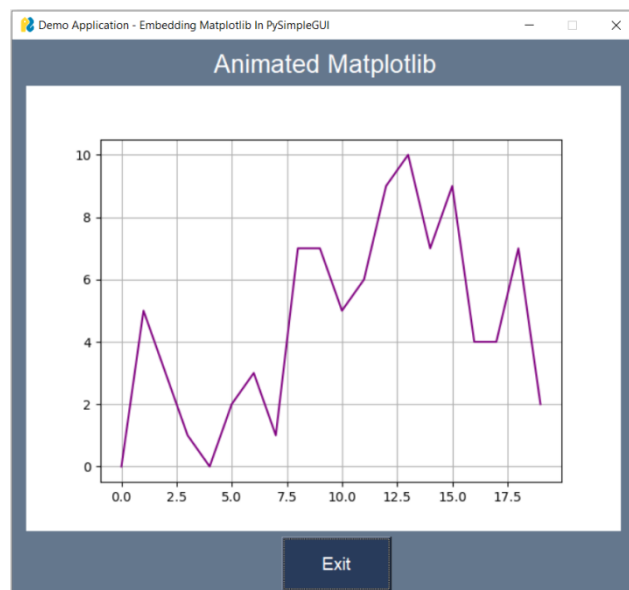


Рисунок 4.8 – Відображення результатів генерації випадкових чисел

Частина 2. Використання бібліотек Matplotlib та PyFirmata для графічного виводу результатів опитування аналогового давача, що підключено до контролера Arduino, у який завантажено скетч Firmata.

Завантажити на контролер Arduino код з підтримкою бібліотеки Firmata.h та Wire.h, який є у середовищі програмування Arduino IDE: File → Examples → Firmata → StandatdFirmata.ino.

Підключити контролер Arduino UNO до послідовного порту 'COM5' і приєднати до аналогового порту A0 давач, який має аналоговий вихід:

```
import pyfirmata
import numpy as np

board = pyfirmata.Arduino('COM5')
it = pyfirmata.util.Iterator(board)
it.start()
print("Communication Successfully started")

# Connect a USB Device to Serial Port
potentiometer = board.analog[0]
potentiometer.enable_reporting()
print("Connect a USB Device to Serial Port")

# Read data from Sensor on Serial Port
step = np.zeros(30)
data = np.zeros(30)
for t in range(30):
    step[t] = t
    data[t] = potentiometer.read()
    print(potentiometer.read())
```

Додати збереження даних у файл 'Write.csv' в форматі {'A1':"Time", 'B1':"Data"} через роздільник ';' з використанням функцій csv.DictWriter та csv.writer з бібліотеки CSV. Дані, що реєструються давачем read(), записуються до файлу 'Write.csv' за допомогою функцій column_stack() та writer.writerow() з бібліотеки CSV:

```
import csv

# CSV-файл
index = 0
url = 'Write.csv'

# Запись имен столбцов в файл из текущей директории
DataCSV = [{'A1' : "Time", 'B1' : "Data"}]
```

```

with open(url, 'w', newline='') as csvfile: fieldnames = ['A1', 'B1']
# Записує в myFile с разделителем '
writer = csv.DictWriter(csvfile, fieldnames=fieldnames,
delimiter=';', lineterminator='\n')
writer.writerow(DataCSV)

# Открываем файл для записи в текущей директории
csvFile = open(url, 'a', newline='')
# Записує в myFile с разделителем '
writer = csv.writer(csvFile, delimiter=';', lineterminator='\n')

with csvFile:

    while True:

        print("Reading data from sensor")
        step = np.zeros(30)
        data = np.zeros(30)
        for t in range(30):
            step[t] = t
            data[t] = potentiometer.read()
            data[t] = 50*data[t]
            print(potentiometer.read())

        xy = np.column_stack([step, data])
        writer.writerow(xy)
        index = index + 1
        print("Writing complete", index)

```

У підсумку програмний код опитування потенціометру, який підключено до аналогового виводу **A0** контролера Arduino, що з'єднаний з комп'ютером через послідовний порт **'COM5'**:

```

import csv
import PySimpleGUI as sg
import pyfirmata
import numpy as np
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg,
FigureCanvasAgg
from matplotlib.figure import Figure
import matplotlib.backends.tkagg as tkagg
import tkinter as Tk
import time
from random import randint

fig = Figure()

ax = fig.add_subplot(111)
ax.set_xlabel("X axis")
ax.set_ylabel("Y axis")
ax.grid()

layout = [[sg.Text('Animated Matplotlib', size=(40, 1),
justification='center', font='Helvetica 20')],
[sg.Canvas(size=(640, 480), key='canvas'),
[sg.Button('Exit', size=(10, 2), pad=((280, 0), 3),
font='Helvetica 14')]]]

```



```

# create the window and show it without the plot
window = sg.Window('Demo Application - Embedding Matplotlib In
PySimpleGUI', layout, finalize=True)

# needed to access the canvas element prior to reading the window
canvas_elem = window['canvas']

if __name__ == '__main__':
    board = pyfirmata.Arduino('COM5')
    print("Communication Successfully started")

    it = pyfirmata.util.Iterator(board)
    it.start()

    # Connect a USB Device to Serial Port
    potentiometer = board.analog[0]
    potentiometer.enable_reporting()
    print("Connect a USB Device to Serial Port")

    graph = FigureCanvasTkAgg(fig, master=canvas_elem.TKCanvas)
    canvas = canvas_elem.TKCanvas

    # CSV-файл
    index = 0
    url = 'Write.csv'

    # Запись имен столбцов в файл из текущей директории
    DataCSV = [{'A1' : "Time", 'B1' : "Data"}]
    with open(url, 'w', newline='') as csvfile:
        fieldnames = ['A1', 'B1']
        writer = csv.DictWriter(csvfile, fieldnames=fieldnames,
                                delimiter=';', lineterminator='\n') # Записывает в myFile с
                                # разделителем ';'
        writer.writerow(DataCSV)

    # Открываем файл для записи в текущей директории
    csvFile = open(url, 'a', newline='')
    writer = csv.writer(csvFile, delimiter=';', lineterminator='\n')
    # Записывает в myFile с разделителем ';'

    with csvFile:

        while True:

            print("Reading data from sensor")
            step = np.zeros(30)
            data = np.zeros(30)
            for t in range(30):
                step[t] = t
                data[t] = potentiometer.read()
                data[t] = 50*data[t]
                print(potentiometer.read())
                time.sleep(0.1)

            xy = np.column_stack([step, data])
            writer.writerow(xy)

            index = index + 1
            print("Writing complete", index)

            dpts = [data[1:30] for x in range(10000)]
            time.sleep(0.1)

```

```

# Our event loop
for i in range(len(data)):
    event, values = window.read(timeout=20)

    if event == 'Exit' or event == sg.WIN_CLOSED:
        exit(69)

    ax.cla()
    ax.grid()

    j = i + 10
    if j >= len(data):
        i = 0
        j = i + 10
    ax.plot(range(10), data[i:j], color='purple')
    time.sleep(0.01)

    graph.draw()
    figure_x, figure_y, figure_w, figure_h =
fig.bbox.bounds
    figure_w, figure_h = int(figure_w), int(figure_h)
    photo = Tk.PhotoImage(master=canvas, width=figure_w,
height=figure_h)

    canvas.create_image(640 / 2, 480 / 2, image=photo)

    figure_canvas_agg = FigureCanvasAgg(fig)
    figure_canvas_agg.draw()
    tkagg.blit(photo,
figure_canvas_agg.get_renderer()._renderer, colormode=2)

    # Закриваємо файл для записи в текущей директории
    csvFile.close()

```

Відображення результатів опитування давача з використанням бібліотеки PyFirmata у графічному вікні за допомогою модулів Figure, FigureCanvasTkAgg та FigureCanvasAgg з бібліотеки Matplotlib (рис. 4.9):

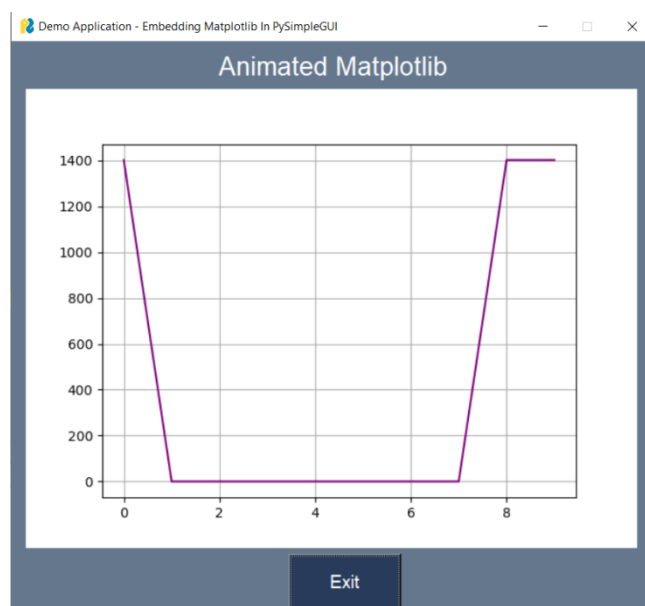


Рисунок 4.9 – Відображення результатів опитування давача

Детально параметри функцій бібліотеки Firmata можна переглянути у документації на сайті «Ardiuno.UA» <https://doc.arduino.ua/ru/prog/Firmata>.

4.3. Завдання для самостійної роботи

Завантажити на контролер Arduino код з підтримкою бібліотеки Firmata.h та Wire.h, який є у середовищі програмування Arduino IDE: File → Examples → Firmata → StandatdFirmata.ino.

Підключити контролер Arduino UNO до послідовного порту 'COM5' і приєднати до аналогових портів A0 та A1 давачі, які мають аналогові входи:

```
import pyfirmata

board = pyfirmata.Arduino('COM5')
it = pyfirmata.util.Iterator(board)
it.start()
print("Communication Successfully started")

# Connect a USB Device to Serial Port
potentiometer_1 = board.analog[0]
potentiometer_1.enable_reporting()

potentiometer_2 = board.analog[1]
potentiometer_2.enable_reporting()
```

Реалізувати послідовне опитування двох давачів, які підключено аналогових виводів контролеру Arduino, та відобразити отримані дані у графічному вікні з використанням функцій бібліотеки Matplotlib (рис. 4.10):

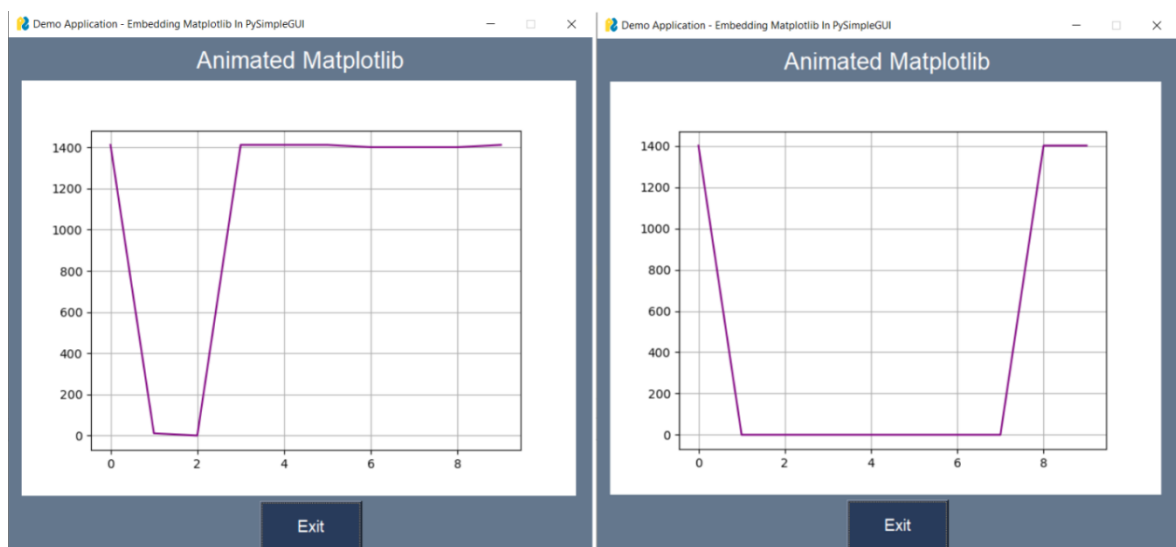


Рисунок 4.10 – Реалізувати послідовного опитування двох давачів

Під час виконання самостійного завдання у програмному коді реалізувати наступну схему опитування потенціометрів на аналогових виводах A0 і A1:

```
import math
from random import randint
...
# Connect a USB Device to Serial Port
potentiometer_1 = board.analog[0]
potentiometer_1.enable_reporting()

potentiometer_2 = board.analog[1]
potentiometer_2.enable_reporting()
...
sensor = math.fabs(randint(0, 1))
if sensor == 0:
    for t in range(30):
        step[t] = t
        data[t] = potentiometer_1.read()
        data[t] = 50*data[t]
        print(potentiometer_1.read())
        time.sleep(0.1)
if sensor == 1:
    for t in range(30):
        step[t] = t
        data[t] = potentiometer_1.read()
        data[t] = 50*data[t]
        print(potentiometer_1.read())
        time.sleep(0.1)
```

4.4. Контрольні питання

1. Які є можливості бібліотеки Matplotlib для побудови графіків?
2. Як порівняти декілька графіків використовуючи функції Matplotlib?
3. Що таке гістограма?
4. Як відобразити довірчі інтервали на графіку функції?
5. Які є можливості бібліотеки Axes3D для тривимірної графіки?
6. Як побудувати тривимірну поверхню використовуючи Axes3D?
7. Як зберігти дані у файл використовуючи функції бібліотеки CSV?
8. Як відобразити дані опитування давачів у реальному часі на графіку?

ПРАКТИЧНА РОБОТА № 5

РОБОТА ІЗ ЗОБРАЖЕННЯМИ В OpenCV

З ВИКОРИСТАННЯМ АРІ-ФУНКЦІЙ БІБЛІОТЕКИ CV2

Мета заняття: набути навичок оброблення зображень із застосуванням функцій бібліотеки OpenCV використовуючи мову програмування Python.

5.1. Основні теоретичні відомості

Однією із основних бібліотек для комп'ютерного зору є бібліотека OpenCV (Open Source Computer Vision Library), яка містить функцій та алгоритми обробки зображень і комп'ютерного зору, чисельні алгоритми загального призначення з відкритим кодом. Бібліотека надає засоби для обробки і аналізу вмісту зображень, у тому числі розпізнавання об'єктів на фотографіях (наприклад, осіб і фігур людей, тексту тощо), відстежування руху об'єктів, перетворення зображень, застосування методів машинного навчання і виявлення загальних елементів на різних зображеннях. Бібліотека розроблена Intel і підтримується Willow Garage та Itseez.

Бібліотека містить понад 2500 оптимізованих алгоритмів, серед яких повний набір як класичних так і практичних алгоритмів машинного навчання і комп'ютерного зору, які застосовують у таких сферах:

- Аналіз та обробка зображень;
- Системи з розпізнавання обличчя ;
- Ідентифікації об'єктів;
- Розпізнавання жестів на відео;
- Відстежування переміщення камери;
- Побудова 3D моделей об'єктів;
- Створення 3D хмар точок зі стерео камер;

- Склеювання зображень між собою для створення зображень сцени;
- Система взаємодії людини з комп'ютером;
- Пошуку схожих зображень із бази даних;
- Усування ефекту червоних очей при фотозйомці зі спалахом;
- Стеження за рухом очей;
- Трекінг відео;
- Аналіз руху;
- Ідентифікація об'єктів;
- Сегментація зображення;
- Розпізнавання елементів сцени і додавання маркерів для створення доповненої реальності та інших.

Розглянемо основні операції для роботи із зображеннями. Для зчитування зображень використовується [7]:

1. Функція ***cv2.imread***. Дана функція має два параметри: *path* – шлях до файлу, який необхідно відкрити; *flag* – опція, що вказує, яким чином слід відкрити файл. Функція *cv2.imread* повертає масив *numpy*. Якщо файл не буде знайдено, то буде повернено порожній масив.

Параметр *flag* може приймати наступні значення:

- ***cv2.IMREAD_COLOR***: завантажуване зображення розглядається як кольорове. Будь-яку прозорість зображення буде проігноровано. Це є значенням параметру за замовчуванням. Замість цього значення можна також передати значення 1.
- ***cv2.IMREAD_GRAYSCALE***: завантажуване зображення розглядається як кольорове. Альтернативним варіантом задання цього параметру є число 0.
- ***cv2.IMREAD_UNCHANGED***: показує, що завантажуване зображення містить альфа канал (тобто містить інформацію не тільки про значення кольорових каналів, наприклад інформацію про прозорі частини

зображення). Альтернативним варіантом задання цього параметру є число -1.

2. Функція **cv2.imwrite**. Дана функція використовується для збереження зображення і має два параметри: *path* – шлях до файлу, який необхідно зберігти; *img* – вихідне зображення; *flag* – опція, що вказує, яким чином слід відкрити файл.

Параметр *flag* може приймати наступні значення:

- **cv2.IMWRITE_JPEG_QUALITY**: вказує якість JPEG зображення, що зберігається як кольорове.

```
import cv2
import numpy
from matplotlib import pyplot as plt

img = cv2.imread('NTUU.jpg')
print (img.shape)

window_name = 'image'
cv2.imshow(window_name, img)
cv2.waitKey(0)

cv2.imwrite('NTUU.png', img, [cv2.IMWRITE_JPEG_QUALITY, 90])
img_new = cv2.imread('NTUU_grey.png')
cv2.waitKey(0)

cv2.destroyAllWindows()
```

Кольорові зображення в OpenCV завантажуються в режимі BGR. Оскільки зображення представляються у вигляді матриць (або масивів матриць, як у випадку із кольоровими зображеннями), то операції над матрицями можуть бути застосовані до зображень. Так, можна змінювати пікселі зображень, виділяти окремі області зображень і т.д. Також можна розділяти канали зображення, склеювати їх і т.д. Аналогічно можна змінювати значення пікселя та виділяти окремі області зображення.

```
import cv2
import numpy
from matplotlib import pyplot as plt

img = cv2.imread('NTUU.jpg')
px = img[300, 300]
print('Значення пікселя у форматі BGR:', px)
print('Значення блакитного каналу:', px[0])
```

```

img[100, 100] = [255, 255, 255]
px = img[100, 100]
print('Змінюємо колір окремого пікселя', px)

window_name = 'image'
img_scale = img[400:800, 400:800]
cv2.imshow(window_name, img_scale)
cv2.waitKey(0)

cv2.imwrite('NTUU_Scile.jpg', img_scale)
cv2.waitKey(0)

cv2.destroyAllWindows()

```

Порівняємо отримані зображення (рис. 5.1):



'NTUU.jpg'



'NTUU_Scile.jpg'

Рисунок 5.1 – Змінення начень пікселів зображення у форматі BGR

Над зображеннями також можна виконувати арифметичні операції. Якщо додати число до усіх елементів зображення, то збільшиться його інтенсивність (зображення стане світліше), якщо відняти – зменшиться. Самі зображення також можна додавати:

3. Функція ***cv2.addWeighted***. Дана функція додає зображення за формулою $img = \alpha \cdot img1 + \beta \cdot img2 + \gamma$.

```

import cv2
import numpy
from matplotlib import pyplot as plt

img_kpi = cv2.imread('NTUU.jpg')
img_cat = cv2.imread('MyCat.jpg')

img_scale_kpi = img_kpi[200:900, 200:900]
img_scale_cat = img_cat[000:700, 000:700]

```



```

window_name = 'image'
img_result = cv2.addWeighted(img_scale_kpi, .35, img_scale_cat, .95, 0)
cv2.imshow(window_name, img_result)
cv2.waitKey(0)

cv2.imwrite('NTUU_Cat.jpg', img_result)
cv2.waitKey(0)

cv2.destroyAllWindows()

```

Порівняємо отримані зображення (рис. 5.2):



img1 – 'NTUU.jpg'



img – 'NTUU_Cat.jpg'

Рисунок 5.2 – Додавання зображень: $img = \alpha \cdot img1 + \beta \cdot img2 + \gamma$

Над зображеннями можна також проводити побітові логічні операції та змінювати колірні моделі. Колірна модель є абстрактною моделлю опису представлення кольорів у вигляді кортежів (наборів) чисел, зазвичай з трьох або чотирьох значень, званих колірними компонентами або колірними координатами.

У OpenCV здебільшого використовують три колірні моделі опису: grayscale (чорно-білі зображення), BGR та HSV (або HSB). Розглянемо ці моделі детальніше [7]:

- Greyscale є моделлю, що зводить усю колірну інформацію до відтінків сірого, тобто до яскравості окремих частин зображення. Ця модель дуже корисна, коли інформація про яскравість є достатньою (наприклад при розпізнаванні облич). Зазвичай кожен піксель представляється 8-бітним числом, від 0 (чорне), до 255 (біле).

- BGR є моделлю, у якій кожен піксель представлено трійкою значень, кожне з яких відповідає за окремий канал: синій, зелений та червоний. В даній моделі трійка (0,0,0) позначає чорний колір, (255,0,0) – синій, (0,255,0) – зелений, (0,0,255) – червоний.
- HSV (також HSB) є моделлю, що заснована на трьох характеристиках кольору: колірному тоні (Hue), насиченості (Saturation) і значенні кольору (Value), який також називають яскравістю (Brightness). Тон змінюється у проміжку $[0,179]$, насиченість у $[0,255]$, значення кольору у проміжку $[0,255]$.

Різні програми використовують різні шкали, тому під час обробки зображень в OpenCV виникає необхідність нормалізації значень:

4. Функція **cv2.cvtColor**. Дана функція має два параметри: `input_image` – файл, який необхідно обробити; `flag` – опція, що визначає тип конвертації. Для переведення BGR у Grayscale використовується значення прапорця `cv2.COLOR_BGR2GRAY`. Аналогічно, для переведення BGR у HSV, використовується прапорець `cv2.COLOR_BGR2HSV`.

У OpenCV використовуються афінні перетворення зображень (affinis). Афінне перетворення вхідного відображення $f: \mathbb{R}^n \rightarrow \mathbb{R}^n$ можна записати у вигляді [7]:

$$f(x) = M \cdot x + v,$$

де M – невироджена матриця і $v \in \mathbb{R}^n$,

x – вихідний базис простору, v – новий базис простору.

Афінне відображення задовольняє наступним властивостям:

- Колінеарність: точки, які лежать на одній лінії до перетворення, лежатимуть на одній прямій і після перетворення;
- Паралельність: паралельні лінії залишатимуться паралельними і після трансформації;
- Опуклість: опукла множина залишиться опуклою після перетворення;

- Відношення відрізків, що лежать на паралельних прямих: відношення довжин відрізків, які лежать на двох паралельних прямих, не зміниться після перетворення.

Є три основні афінні перетворення: перенесення, поворот, масштабування. В OpenCV є дві функції для афінних перетворень:

5. Функція ***cv2.warpAffine***. Дана функція приймає матрицю розмірності 2×3 . Другим параметром функції є розмір перетвореного зображення, який задається у вигляді пари (width, height), де width – це кількість стовпців, height – кількість рядків у матриці зображення.

6. Функція ***cv2.warpPerspective***. Дана функція приймає матрицю розмірності 3×3 .

7. Функція ***cv2.getRotationMatrix2D***, яка використовується для знаходження матриці перетворення.

8. Функція ***cv2.resize***. Дана функція використовується для масштабування. Розмір зображення може бути заданим вручну, або за допомогою масштабуючого множника. При цьому використовуються різні методи інтерполяції, такі як: ***cv2.INTER_AREA*** для зменшення, ***cv2.INTER_CUBIC*** (повільний метод) та ***cv2.INTER_LINEAR*** для збільшення. За замовчуванням використовується метод масштабування ***cv2.INTER_LINEAR***.

Бібліотека OpenCV містить також методи фільтрації зображень в просторовій і частотній областях. При просторовій фільтрації яскравість (сигнал) кожної точки вихідного зображення, спотвореного шумом, замінюється деяким іншим значенням яскравості, яке в меншій мірі було спотворене. Результатом фільтрації є оцінка корисного сигналу зображення. Це досягається завдяки тому, що зображення часто являє собою двовимірну функцію просторових координат, яка змінюється по цих координатах повільніше, ніж шум, що також є двовимірною функцією. Це дозволяє при оцінці корисного сигналу в кожній точці зображення взяти до уваги сусідні точки, скориставшись певною подібністю сигналу.

В інших випадках, навпаки, ознакою корисного сигналу є різкі перепади яскравості. Однак, як правило, частота цих перепадів відносно невелика, так що на значних проміжках сигнал або постійний, або змінюється повільно. У цьому випадку властивості сигналу проявляються при спостереженні не тільки його окремої точки, але й при аналізі її умовної околиці.

Просторові фільтри можна поділити на такі два класи:

- Лінійні (усереднюючий фільтр, Лапласіан, Ліапласіан, Гаусіана і т.д.);
- Нелінійні (медіанний фільтр, фільтри "максимум" та "мінімум", фільтри Собеля, Прюїтта, Кенні).

Для фільтрації зображення використовується маска (яку ще називають фільтром), що зазвичай являє собою двовимірне квадратне вікно, яке рухається по зображенню, діючи тільки на один піксель за один прохід. Кожне число у фільтрі розглядається як коефіцієнт. Коефіцієнти фільтра визначають ефект відпросторової фільтрації та вигляд результуючого зображення (рис. 5.3) [7].

7	23	50	64	14
15	13	31	46	8
42	25	92	31	32
71	44	74	94	92
2	43	51	35	4

×

0	2	0
0	0	0
0	0	0

=

-	-	-	-	-
-	46	100	128	-
-	26	62	92	-
-	50	184	62	-
-	-	-	-	-

Рисунок 5.3 – Приклад просторової фільтрації

Лінійні фільтри являють собою певне узагальнення лінійних функцій, тому одним із найпоширеніших лінійних фільтрів є усереднюючий фільтр:

9. Функція ***cv2.filter2D***. Дана функція виконує лінійну просторову фільтрацію. За ковзаючу рамку (padding) у цій функції відповідає необов'язковий параметр `borderType`.

10. Функція ***cv2.blur*** або ***cv2.boxFilter***. Ці функції виконують усереднюючу фільтрацію.

11. Функція ***cv2.medianBlur***. Дана функція є медіанним фільтром, що ставить у відповідність кожному пікселю медіанне значення з його умовній околиці.

12. Функція ***cv2.erode***. Дана функція реалізує фільтр «мінімум», що замінює кожен піксель мінімальним значенням із його околиці.

13. Функція ***cv2.dilate***. Дана функція реалізує фільтр «максимум», який замінює кожен піксель максимальним значенням із його околиці.

Зміни інтенсивності сигналу можуть бути виміряні за допомогою перших та других похідних. Наявність змін інтенсивності між однією та іншою сторонами зображення дає можливість виділити контури, які являють собою множину точок зображення. Проблема пошуку фільтра, який є оптимальним за критеріями виділення, локалізації та мінімізації кількох відгуків одного краю, досі є актуальною.

Кенні (John F. Canny) ввів поняття Non-Maximum Suppression, яке означає, що пікселями границь оголошуються точки, в яких досягається локальний максимум градієнта у напрямку вектора градієнта. Детектор Canny має мінімально реагувати на контури, але при цьому ігнорувати хибні межі. Алгоритм складається із п'яти кроків [17]:

- Згладжування. Розмиття зображення для видалення шуму.
- Пошук рґадієнтів. Границі відмічаються там, де градієнт зображення набуває максимального значення.
- Приглушення не-максимумів. Тільки локальні максимуми відмічаються як справжні межі.
- Подвійна порогова фільтрація. Потенціальні межі відмічаються порогоми яскравості пікселів.
- Трасування області неоднозначності. Границі визначаються шляхом подавлення усіх країв, не зв'язаних із сильними границями.

Серед розроблених дотепер методів виявлення контурів алгоритм Кенні є один із найбільш строго визначених методів, який забезпечує добре та надійне виявлення. Методику виявлення контурів Кенні застосовують у різних системах комп'ютерного бачення [17].

14. Функція `cv2.Canny(image, grad1, grad2)`. Дана функція має два параметри: `image` – вихідне зображення, та `(grad1, grad2)` – зміни інтенсивності між однією та іншою сторонами зображення (рис. 5.4).

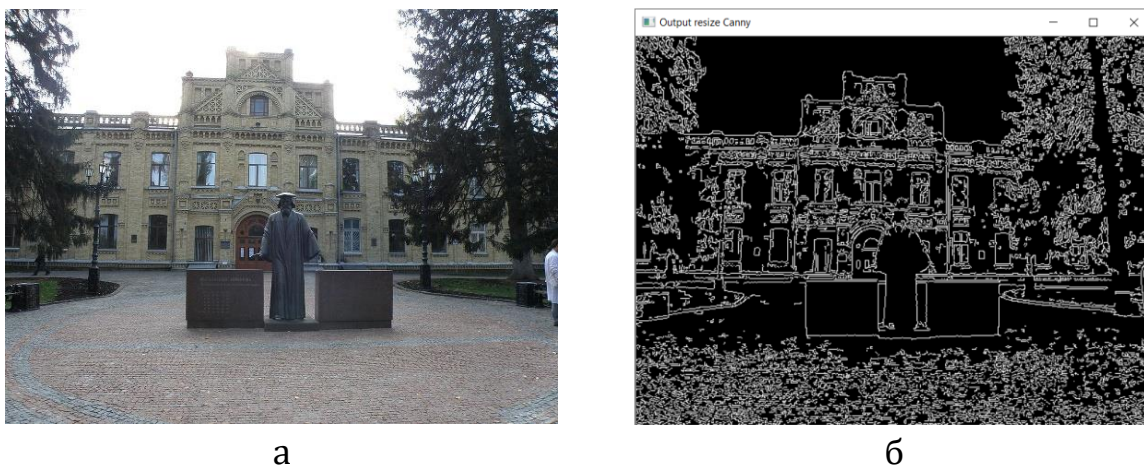


Рисунок 5.4 – Застосування фільтру Canny: *а* – вихідне зображення; *б* – зображення після фільтрації

Завдяки своїй оптимальності для відповідності критеріям виявлення контурів та простоті процесу втілення він став одним із найпопулярніших алгоритмів виявлення контурів. Детальніше про застосування методу Кенні на сайті «GeeksForGeeks.org» <https://www.geeksforgeeks.org/python-opencv-canny-function>.

5.2. Порядок роботи

Підключити до програми бібліотеку PySimpleGUI та зробити структуру для відображення тексту `Text()`, зображення `Image()`, віджету `Column()`, який забезпечує доступ до стовпців кадру даних:

```

import PySimpleGUI as sg

# For now will only show the name of the file that was chosen
images_col = [[sg.Text('You choose from the list:')],
               [sg.Text(size=(40,1), key='-TOUT-')],
               [sg.Image(key='-IMAGE-')]]

# ----- Full layout -----
layout = [[sg.Column(left_col, element_justification='c'),
           sg.VSeparator(), sg.Column(images_col, element_justification='c')]]

# ----- Create Window -----
window = sg.Window('Multiple Format Image Viewer',
                  layout, resizable=True)

```

Підключити до програми бібліотеки `os.Path` та `PIL.Image`, які використовуються для роботи із зображеннями:

```

import os.path
import PIL.Image

```

Визначити об'єктне посилання на вихідний файл `file_or_bytes` та результуючий файл `bio` з зображеннями у "PNG" форматі. Для відкриття фалу зображення використовується функція `PIL.Image.open()`, а для збереження зображення використовується функція `img.save()`:

```

def convert_to_bytes(file_or_bytes, resize=None):

    if isinstance(file_or_bytes, str):
        img = PIL.Image.open(file_or_bytes)
    else:
        try:
            img =
PIL.Image.open(io.BytesIO(base64.b64decode(file_or_bytes)))
        except Exception as e:
            dataBytesIO = io.BytesIO(file_or_bytes)
            img = PIL.Image.open(dataBytesIO)

    cur_width, cur_height = img.size
    if resize:
        new_width, new_height = resize
        scale = min(new_height/cur_height, new_width/cur_width)
        img = img.resize((int(cur_width*scale),
int(cur_height*scale)), PIL.Image.ANTIALIAS)
    bio = io.BytesIO()
    img.save(bio, format="PNG")
    del img
    return bio.getvalue()

```

У підсумку програмний код для зміни розміру відображення зображення за допомогою функцій `image.Open()` та `img.Resize()` з бібліотеки `PIL.Image`, системний шлях до якого вказується у функціях `listdir()` та `join()` з бібліотеки `os.Path`:

```
import PySimpleGUI as sg
import os.path
import PIL.Image
import io
import base64

def convert_to_bytes(file_or_bytes, resize=None):

    if isinstance(file_or_bytes, str):
        img = PIL.Image.open(file_or_bytes)
    else:
        try:
            img =
PIL.Image.open(io.BytesIO(base64.b64decode(file_or_bytes)))
        except Exception as e:
            dataBytesIO = io.BytesIO(file_or_bytes)
            img = PIL.Image.open(dataBytesIO)

    cur_width, cur_height = img.size
    if resize:
        new_width, new_height = resize
        scale = min(new_height/cur_height, new_width/cur_width)
        img = img.resize((int(cur_width*scale),
int(cur_height*scale)), PIL.Image.ANTIALIAS)
    bio = io.BytesIO()
    img.save(bio, format="PNG")
    del img
    return bio.getvalue()

# ----- Define Layout -----
# First the window layout...2 columns

left_col = [[sg.Text('Folder'), sg.In(size=(25,1), enable_events=True
, key='-FOLDER-'), sg.FolderBrowse()],
            [sg.Listbox(values=[], enable_events=True,
size=(40,20), key='-FILE LIST-')],
            [sg.Text('Resize to'), sg.In(key='-W-', size=(5,1)),
sg.In(key='-H-', size=(5,1))]]

# For now will only show the name of the file that was chosen
images_col = [[sg.Text('You choose from the list:'),
                [sg.Text(size=(40,1), key='-TOUT-'),
                [sg.Image(key='-IMAGE-')]]]

# ----- Full layout -----
layout = [[sg.Column(left_col, element_justification='c'),
sg.VSeparator(), sg.Column(images_col, element_justification='c')]]

# ----- Create Window -----
window = sg.Window('Multiple Format Image Viewer',
layout, resizable=True)
```



```

# ----- Run the Event Loop -----
while True:
    event, values = window.read()
    if event in (sg.WIN_CLOSED, 'Exit'):
        break
    if event == sg.WIN_CLOSED or event == 'Exit':
        break
    if event == '-FOLDER-':
        # Folder name was
        # filled in, make a list of files in the folder
        folder = values['-FOLDER-']
        try:
            file_list = os.listdir(folder)
            # get list of
            # files in folder
        except:
            file_list = []
            fnames = [f for f in file_list if os.path.isfile(
                os.path.join(folder, f)) and f.lower().endswith((".png",
                ".jpg", ".jpeg", ".tiff", ".bmp"))]
            window['-FILE LIST-'].update(fnames)
        elif event == '-FILE LIST-':
            # A file was chosen from the
            # listbox
            try:
                filename = os.path.join(values['-FOLDER-'], values['-FILE
                LIST-'][0])
                window['-TOUT-'].update(filename)
                if values['-W-'] and values['-H-']:
                    new_size = int(values['-W-']), int(values['-H-'])
                else:
                    new_size = None
                window['-IMAGE-'].update(data=convert_to_bytes(filename,
                resize=new_size))
            except Exception as E:
                print(E.decode("utf-8"))
                pass
            # something weird happened making the full
            # filename
# ----- Close & Exit -----
window.close()

```

Після оброблення зображення воно зберігається у заданому форматі `format="PNG"` за допомогою функції `img.save()` з бібліотеки `PIL.Image`.

Відображення процедури зміни розміру зображення від `size=(100,100)` до `size=(400,400)` із використанням функції `img.Resize()` з бібліотеки `PIL.Image` (рис. 5.5):

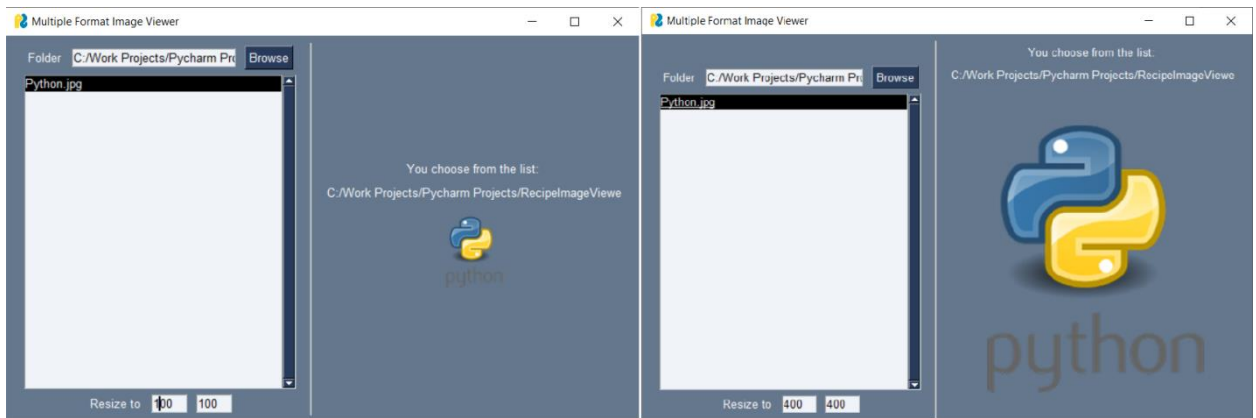


Рисунок 5.5 – Зміни розміру зображення від (100, 100) до (400, 400)

5.3. Завдання для самостійної роботи

Підключити бібліотеку cv2 (OpenCV) для обробки зображень методами комп'ютерного зору:

```
import cv2
import os.path
import PIL.Image
```

Створити відображення заданого зображення із використанням функцій `matchShapes` і `drawContours` для створення плавних переходів кольорів на контурах зображення, які входять до бібліотеки cv2 (OpenCV):

```
filename = os.path.join(values['-FOLDER-'], values['-FILE LIST-'][0])

# Read image from disk.
img = cv2.imread(filename)

# Get number of pixel horizontally and vertically.
(height, width) = img.shape[:2]

# Specify the size of image along with interpolation methods.
# cv2.INTER_AREA is used for shrinking, whereas cv2.INTER_CUBIC
# is used for zooming.
res = cv2.resize(img, (int(width / 2), int(height / 2)),
interpolation=cv2.INTER_CUBIC)

# Write image back to disk.
cv2.imwrite('result_resize.jpg', res)

# Contour shapes of images
img1 = cv2.imread('result_resize.jpg')
img2 = cv2.imread(filename)

ref_contour = get_ref_contour(img1)
input_contours = get_all_contours(img2)
```

```

closest_contour = input_contours[0]
min_dist = -sys.maxsize - 1
for contour in input_contours:
    ret = cv2.matchShapes(ref_contour, contour, 1, 0.0)
    if ret < min_dist:
        min_dist = ret
        closest_contour = contour

cv2.drawContours(img2, [closest_contour], -1, (0, 0, 0), 3)
cv2.imwrite('result_shape.jpg', img2)
cv2.imshow('Output contour Shapes', img2)

```

Відображення зображення після оброблення його методами бібліотеки cv2 (OpenCV) має вигляд (рис. 5.6):

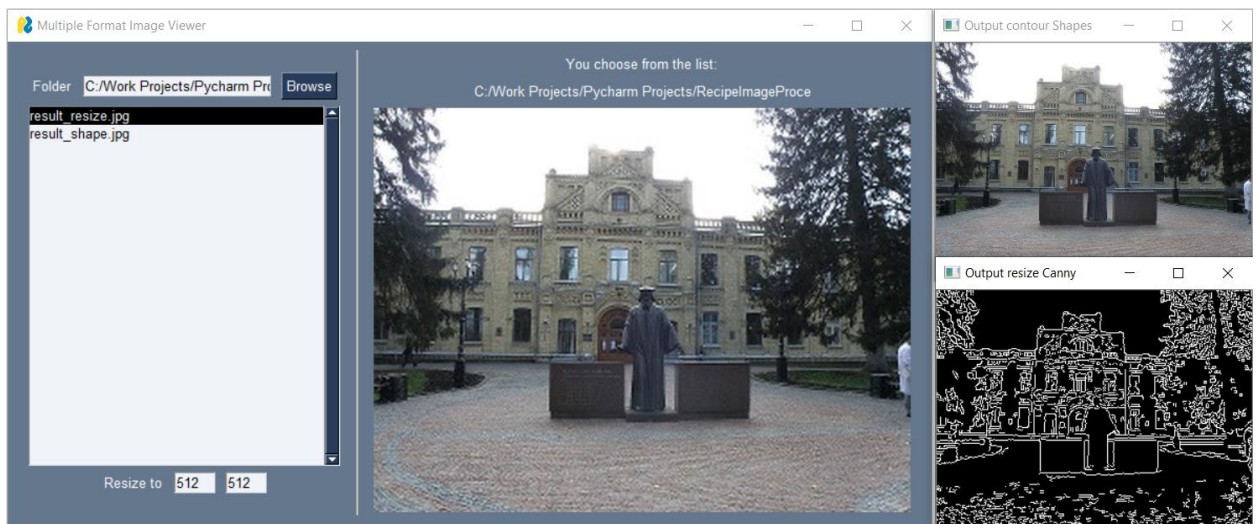


Рисунок 5.6 – Відображення зображення після оброблення

Для отриманого зображення застосувати фільтр Канні з використанням функції Canny() із бібліотеки комп'ютерного зору cv2 і структури програмного коду, в якому використовуються бібліотеки PIL.Image, io, cv2:

```

import sys
import PySimpleGUI as sg
import os.path
import PIL.Image
import io
import base64
import cv2

def get_ref_contour(img):
    ref_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    ret, thresh = cv2.threshold(ref_gray, 127, 255, 0)
    contours, hierarchy = cv2.findContours(thresh, 1, 2)

```

```

    for contour in contours:
        area = cv2.contourArea(contour)
        img_area = img.shape[0] * img.shape[1]
        if 0.05 < area/float(img_area) < 0.8:
            return contour

def get_all_contours(img):
    ref_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    ret, thresh = cv2.threshold(ref_gray, 127, 255, 0)
    contours, hierarchy = cv2.findContours(thresh, 1, 2)
    return contours

def convert_to_bytes(file_or_bytes, resize=None):

    if isinstance(file_or_bytes, str):
        img = PIL.Image.open(file_or_bytes)
    else:
        try:
            img =
PIL.Image.open(io.BytesIO(base64.b64decode(file_or_bytes)))
        except Exception as e:
            dataBytesIO = io.BytesIO(file_or_bytes)
            img = PIL.Image.open(dataBytesIO)

    cur_width, cur_height = img.size
    if resize:
        new_width, new_height = resize
        scale = min(new_height/cur_height, new_width/cur_width)
        img = img.resize((int(cur_width*scale), int(cur_height*scale)),
PIL.Image.ANTIALIAS)
    bio = io.BytesIO()
    img.save(bio, format="PNG")
    del img
    return bio.getvalue()

# ----- Define Layout -----

# First the window layout...2 columns

left_col = [[sg.Text('Folder'), sg.In(size=(25,1), enable_events=True
, key='-FOLDER-'), sg.FolderBrowse()],
            [sg.Listbox(values=[], enable_events=True,
size=(40,20), key='-FILE LIST-')],
            [sg.Text('Resize to'), sg.In(key='-W-', size=(5,1)),
sg.In(key='-H-', size=(5,1))]]

# For now will only show the name of the file that was chosen
images_col = [[sg.Text('You choose from the list:'),
            [sg.Text(size=(40,1), key='-TOUT-')],
            [sg.Image(key='-IMAGE-')]]

# ----- Full layout -----
layout = [[sg.Column(left_col, element_justification='c'),
sg.VSeparator(), sg.Column(images_col, element_justification='c')]]

# ----- Create Window -----
window = sg.Window('Multiple Format Image Viewer',
layout, resizable=True)

# ----- Run the Event Loop -----
while True:
    event, values = window.read()
    if event in (sg.WIN_CLOSED, 'Exit'):

```

```

        break
    if event == sg.WIN_CLOSED or event == 'Exit':
        break
    if event == '-FOLDER-':
        # Folder name was
        # filled in, make a list of files in the folder
        folder = values['-FOLDER-']
        try:
            file_list = os.listdir(folder)
            # get list of files
            in folder
        except:
            file_list = []
            fnames = [f for f in file_list if os.path.isfile(
                os.path.join(folder, f)) and f.lower().endswith((".png",
                ".jpg", ".jpeg", ".tiff", ".bmp"))]
            window['-FILE LIST-'].update(fnames)
        elif event == '-FILE LIST-':
            # A file was chosen from the listbox
            try:
                filename = os.path.join(values['-FOLDER-'], values['-FILE
                LIST-'][0])

                # Read image from disk.
                img = cv2.imread(filename)

                # Get number of pixel horizontally and vertically.
                (height, width) = img.shape[:2]

                # Specify the size of image along with interpolation
                methods.
                # cv2.INTER_AREA is used for shrinking, whereas
                cv2.INTER_CUBIC
                # is used for zooming.
                res = cv2.resize(img, (int(width / 2), int(height / 2)),
                interpolation=cv2.INTER_CUBIC)

                # Write image back to disk.
                cv2.imwrite('result_resize.jpg', res)

                # Canny edge detection.
                edges = cv2.Canny(res, 100, 200)

                # Show image after Canny.
                cv2.imshow('Output resize Canny', edges)

                # Contour shapes of images
                img1 = cv2.imread('result_resize.jpg')
                img2 = cv2.imread(filename)

                ref_contour = get_ref_contour(img1)
                input_contours = get_all_contours(img2)

                closest_contour = input_contours[0]
                min_dist = -sys.maxsize - 1
                for contour in input_contours:
                    ret = cv2.matchShapes(ref_contour, contour, 1, 0.0)
                    if ret < min_dist:
                        min_dist = ret
                        closest_contour = contour

                cv2.drawContours(img2, [closest_contour], -1, (0, 0, 0), 3)
                cv2.imwrite('result_shape.jpg', img2)
                cv2.imshow('Output contour Shapes', img2)

                window['-TOUT-'].update(filename)

```

```

    if values['-W-'] and values['-H-']:
        new_size = int(values['-W-']), int(values['-H-'])
    else:
        new_size = None
    window['-IMAGE-'].update(data=convert_to_bytes(filename,
resize=new_size))
    except Exception as E:
        print(E.decode("utf-8"))
    pass          # something weird happened making the full
filename
# ----- Close & Exit -----
window.close()

```

5.4. Контрольні питання

1. Які методи та алгоритми охоплює бібліотека OpenCV?
2. Які є основні операції для роботи із зображеннями?
3. У чому сутність зваженого додавання зображень?
4. Які є афінні перетворення зображень?
5. У чому сутність фільтрації у просторовій області?
6. Які є методи виявлення контурів на зображеннях?

ПРАКТИЧНА РОБОТА № 6

ВІЗУАЛІЗАЦІЯ І ФІЛЬТРАЦІЯ СИГНАЛІВ З ВИКОРИСТАННЯМ ФУНКЦІЙ МОДУЛЮ SIGNAL БІБЛІОТЕКИ SCIPY

Мета заняття: набути навичок проектування аналогових та цифрових фільтрів і реалізації дискретної фільтрації сигналів використовуючи мову програмування Python.

6.1. Основні теоретичні відомості

Основна властивість аналогових і цифрових фільтрів є їх частотна та фазова характеристики. Вони показують, як фільтр впливає на амплітуду і фазу різних гармонік оброблюваного сигналу.

Апроксимація фільтрів – апроксимація форми амплітудно-частотної характеристики (АЧХ) фільтрів. Апроксимація фільтру полягає у підборі аналітичного опису передавальної функції (характеристичної функції), яка задовольняє умовам фільтрації та легко реалізується у вигляді конкретного схемотехнічного блоку або чисельного алгоритму.

За формою графіку модуля комплексної частотної характеристики (за видом АЧХ) фільтри ділять на чотири основні типи [8]:

- фільтр нижніх частот (ФНЧ, low-pass filter) – має порівняно великий коефіцієнт передачі на частотах, які знаходяться в околі нульової частоти, і порівняно низький – на інших частотах. Тому такі фільтри зберігають незмінною величину спектральних складових вхідного сигналу на нижніх частотах. ФНЧ пропускають низькочастотні сигнали та не пропускають високочастотні;

- фільтр верхніх частот (ФВЧ, high-pass filter) – має високий коефіцієнт передачі на частотах, які більші за частоту зрізу, і низький – на частотах від нуля до частоти зрізу f_p . Отже, вони пропускають без змін частину спектру вхідного сигналу, яка знаходиться на частотах від частоти зрізу f_p до $+\infty$ і не

пропускають складові спектру сигналу від 0 до f_p . ФВЧ пропускають високочастотні сигнали;

– смуговий фільтр (СФ, band-pass filter) – має великий коефіцієнт передачі лише у певній смузі частот між частотами f_{p1} та f_{p2} . Поза цим діапазоном частот коефіцієнт передачі малий. Отже, на вихід такого фільтра пройдуть лише ті спектральні складові сигналу, які знаходяться у смузі пропускання фільтра, і не пройдуть ті сигнали, які лежать від нуля до f_{p1} та від f_{p2} до $+\infty$;

– загороджувальний фільтр (ЗФ, band-stop filter) – є дуальним до смугового: великий коефіцієнт передачі у нього поза діапазоном частот від f_{p1} до f_{p2} . Починаючи від нульової частоти і до f_{p1} , а також від f_{p2} до $+\infty$ коефіцієнт передачі великий. Тому у разі проходження через такий фільтр сигналу, із його спектру видаляються складові, що лежать між f_{p1} та f_{p2} , а всі інші складові пройдуть на вихід фільтра без змін. Іноді загороджувальний фільтр називають режекторним фільтром. Якщо смуга загороджувального фільтра порівняно вузька, такий фільтр іноді називають фільтром-пробкою (notch-filter). Зокрема, такі фільтри використовують для позбавлення від мережевої перешкоди, коли треба вирізати зі спектру сигналу лише спектральну складову в околі частоти 50 Гц.

Для неперервної системи комплексна частотна характеристика (КЧХ) може бути записана в операторній формі через оператор Лапласа [8]:

$$K(p) = \frac{Y(p)}{X(p)} = \frac{a_0 + a_1 p + a_2 p^2 + \dots + a_m p^m}{b_0 + b_1 p + b_2 p^2 + \dots + b_m p^m},$$

де a, b – коефіцієнти передавальної функції фільтра.

Сукупність значень нулів у чисельнику функції називають нулями передавальної функції, сукупність значень нулів знаменника – полюсами. Вираз для комплексної частотної характеристики $K(p)$ можна представляти у вигляді добутків множників не вище другого порядку:

$$H(p) = k_1(p) \cdot k_2(p) \cdot \dots \cdot k_n(p),$$

де k_n – передавальна функція деякої простої схемної ланки.

Кожний з таких множників є передавальною функцією деякої простої схемної ланки, тобто фільтр будь-якого порядку можна отримати за послідовного з'єднання фільтрів першого та другого порядку. Задача апроксимації полягає у знаходженні кількості нулів та полюсів передавальної функції, а також всіх коефіцієнтів передавальної функції.

Для частотного фільтрування сигналів можна застосувати пряме і обернене дискретне перетворення Фур'є. Перетворення Фур'є розкладає функцію на осциляторні функції, тобто подає сигнал у вигляді суми гармонійних коливань. Для дискретних функцій застосовують дискретне перетворення Фур'є. Обернене дискретне перетворення Фур'є повертає сигнал за спектром його частот, що дає змогу реалізувати смугову фільтрацію сигналів.

На мові Python є модуль SciPy.fftpack, що використовує швидкі алгоритми прямого і оберненого дискретного перетворення Фур'є (FFT). Розглянемо фільтрацію низькочастотного періодичного сигналу частотою 2 Гц, який зашумлений мережевою завадою 50 Гц [14]:

```
import numpy as np
from scipy.fftpack import fft, rfft, irfft, fftfreq
import matplotlib.pyplot as plt

N = 800 # кількість точок
T = 1.0/400.0 # проміжки часу, с
A1= 10.0; A2= 1.0 # амплітуди, мВ
f1= 2.0; f2= 50.0 # лінійні частоти, Гц
omega1=f1*np.pi; omega2=f2*np.pi # циклічні частоти

x = np.linspace(0.0, N*T, N) # час
y = A1*np.cos(omega1*x) + A2*np.cos(omega2*x) # сигнал (лінійна
комбінація синусоїдальних сигналів)
plt.plot(x,y,'r') # нарисувати сигнал
plt.xlabel('t');plt.ylabel('y');plt.grid();plt.show()
print ("Рисунок - Сигнал")
yf = fft(y) # розрахувати спектр частот для сигналів змінних у часі за
допомогою дискретного перетворення Фур'є. Декомпозиція сигналу на
частоти і амплітуди.
```

```

xf = np.linspace(0.0, 1.0/(2.0*T), N//2) # частоти
yf_ = 2.0/N * np.abs(yf[0:N//2]) # амплітуди
plt.figure()
plt.plot(xf, yf_, 'k') # малювати спектр
plt.xlabel('f'); plt.ylabel('A'); plt.grid(); plt.show()
print ("Рисунок - Амплітудно-частотна характеристика сигналу")

time = x # час, с
signal = y # сигнал, мВ
W = fftfreq(signal.size, d=time[1]-time[0]) # частоти
f_signal = rfft(signal) # спектр (дискретне перетворення Фур'є для
дійсних)
cut_f_signal = f_signal.copy() # копія сигналу
cut_f_signal[W>49] = 0 # фільтруємо сигнал (відкидаємо частоти > 49
Гц)
cut_signal = irfft(cut_f_signal) # відфільтрований сигнал (обернене
дискретне перетворення Фур'є для дійсних)

plt.subplot(121); plt.plot(time, signal);
plt.xlabel('t'); plt.ylabel('y');
plt.subplot(122); plt.plot(time, cut_signal);
plt.xlabel('t')
plt.show()
print ("Рисунок - Початковий і відфільтрований сигнал")

plt.figure()
plt.subplot(121); plt.plot(W, f_signal);
plt.xlim(0, 10); plt.xlabel('f'); plt.ylabel('A');
plt.subplot(122); plt.plot(W, cut_f_signal);
plt.xlim(0, 10); plt.xlabel('f')
plt.show()
print ("Рисунок - Спектр початкового і відфільтрованого сигналу")

```

Відобразимо результати фільтрації сигналу (рис. 6.1 і 6.2): вихідний зашумлений сигнал $y(t)$, амплітудно-частотна характеристика сигналу $A(f)$, відфільтрований сигнал $\hat{y}(t)$, спектр початкового та відфільтрованого сигналу $\hat{A}(f)$.

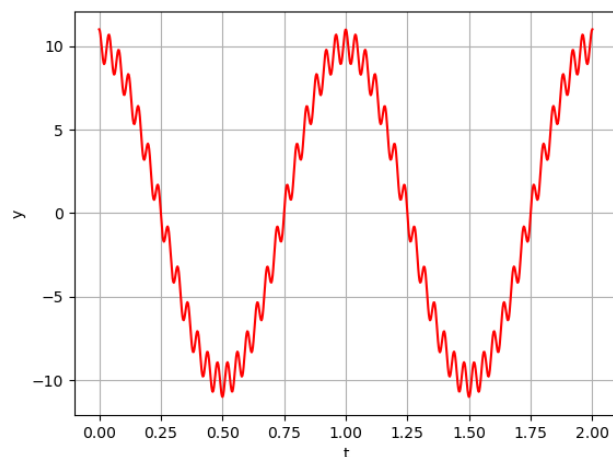


Рисунок 6.1 – Вихідний зашумлений сигнал

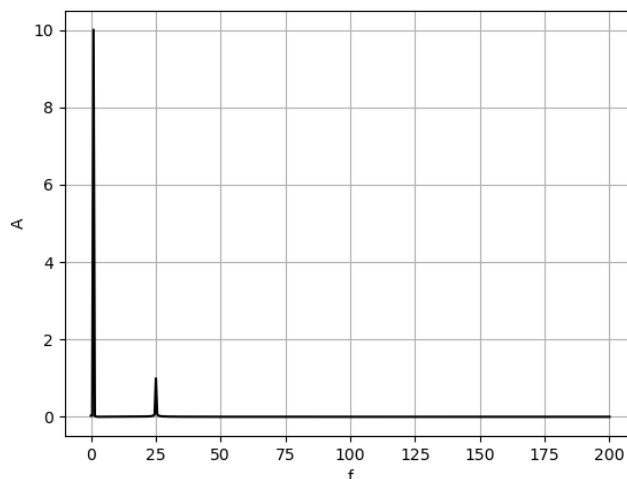


Рисунок 6.2 – Амплітудно-частотна характеристика сигналу

За допомогою модулю `SciPy.fftpack` обчислюється швидко перетворення Фур'є, шляхом передачі одновимірний масиву `signal` у функцію `fft(signal)` чи `rfft(signal)` типу `numpy`, а повернення перетвореного масиву виконанням функції `irfft(signal)`. Відсічка частот вище 49 Гц, що містять заваду здійснюється послідовним виконанням функцій `W = fftfreq(signal.size, time)` та `signal[(W>49)] = 0` (рис. 6.3 і 6.4) [14].

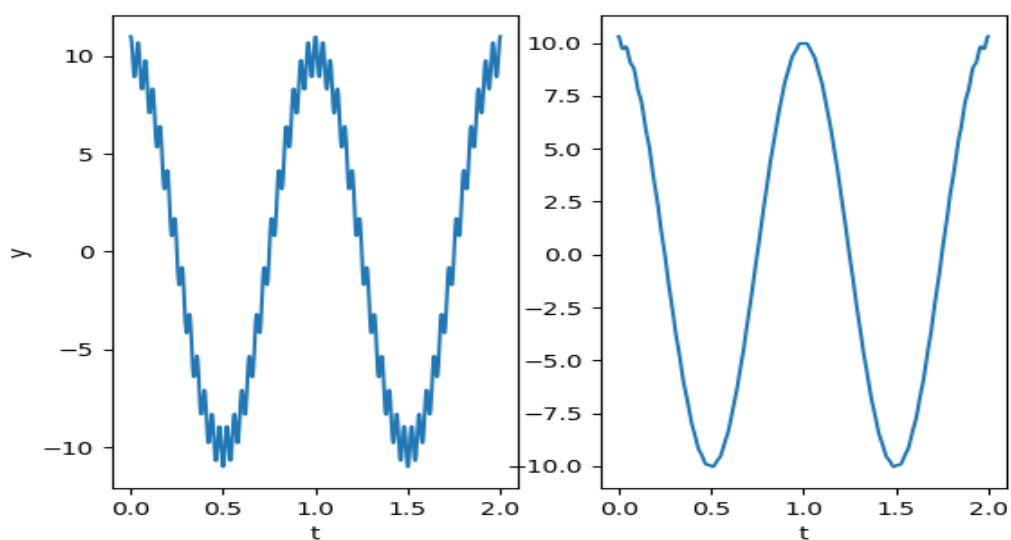


Рисунок 6.3 – Початковий і відфільтрований сигнал

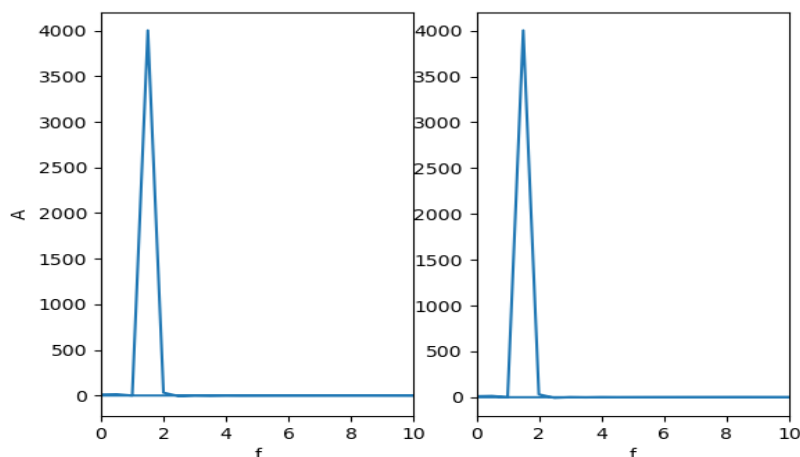


Рисунок 6.4 – Спектр початкового і відфільтрованого сигналу

Потрібно враховувати, що за принципом роботи більшості фільтрів нижніх частот, межа від’ємних частот може бути -3 кГц, що дещо зсуває смугу фільтрації.

6.2. Порядок роботи

Сигнал електрокардіограми завантажується з аудіо файлу 'ecg.wav' та піддається подвійній фільтрації за допомогою цифрових фільтрів Баттерворта, і результат у вигляді сигналу, який очищеного від високочастотних завад, відображається у графічному вікні.

Для використання методів фільтрації використовуються функції Signal із бібліотеки SciPy:

(a) Цифровий або аналоговий фільтр Баттерворта N-го порядку можна реалізувати з використанням функції `signal.butter()`, яка створює фільтр Баттерворта N-го порядку та повертає коефіцієнти фільтра; та функції `signal.sosfilt()`, яка фільтрує дані вздовж одного виміру за допомогою каскадних секцій другого порядку;

(b) Цифровий або аналоговий смуговий фільтр Баттерворта N-го порядку можна реалізувати з використанням функції `signal.buttord()`, яка повертає порядок найнижчого фільтра, що у смузі пропускання забезпечує

втрати не більш ніж gPass дБ та у смузі ослаблення забезпечує відсічку щонайменше gStop дБ; і функції `signal.filtfilt()`, яка двічі застосовує лінійний цифровий фільтр, один раз вперед та один раз назад.

(с) Цифровий або аналоговий смуговий фільтр Баттерворта N-го порядку можна реалізувати з використанням функції `signal.butter()`, яка створює цифровий або аналоговий фільтр Баттерворта N-го порядку та поверніть коефіцієнти фільтра; та функції `signal.filtfilt()`, яка двічі застосовує лінійний цифровий фільтр, один раз вперед та один раз назад.

Детально параметри функцій `Signal` із бібліотеки `SciPy` можна переглянути у документації на сайті «Фундаментальні алгоритми для наукових обчислень на Python» <https://docs.scipy.org/doc/scipy-1.4.0/reference/signal.html>.

Програмний код, який реалізує фільтрацію зашумленого сигналу електрокардіограми трьома способами (а, б, с) використовує стандартні функції `Signal` із бібліотеки `SciPy`:

```
from scipy import signal
import wave, struct, math, random
import matplotlib.pyplot as plt
import numpy as np

dataRate = 44100.0 # hertz
duration = 1.0 # seconds
frequency = 440.0 # hertz

# vFollowing code reads some of the parameters of WAV file
obj = wave.open('ecg.wav', 'r')

# read ECG data from the WAV file
length = obj.getnframes()
sampleRate = obj.getframerate()

data = np.zeros(length)
for i in range(0, length):
    ecg_data = obj.readframes(1)
    io_data = struct.unpack("<h", ecg_data)
    data[i] = int(io_data[0])
time = np.arange(len(data))/sampleRate
print( "Number of channels ", obj.getnchannels())
print( "Sample width      ", obj.getsampwidth())
print( "Frame rate        ", sampleRate)
print( "Number of frames   ", length)

# apply a 3-pole lowpass filter at 0.1x Nyquist frequency
t = np.linspace(0, 1, 1000, False) # 1 second
sig = np.sin(2*np.pi*10*t) + np.sin(2*np.pi*20*t)
sos = signal.butter(10, 15, 'hp', fs=1000, output='sos')
```

```

filtered = signal.sosfilt(sos, sig)
# Plot the filtered of signal
plt.plot(t[:], sig[:], label="Signal channel")
plt.plot(t[:], filtered[:], label="Filter channel")
plt.legend()
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.show()

# apply a 3-pole lowpass filter at 0.1 frequency
# for bandpass:
wp = 0.1
ws = 3.0
# N = order
N, wn = signal.buttord(wp, ws, 30, 100)
# apply a 3-pole lowpass filter at Nyquist frequency
b, a = signal.butter(N, wn, btype='high')
fir_one = signal.filtfilt(b, a, data)

# Plot the waveform
plt.plot(time, data[:], label="ECG channel")
plt.plot(time, fir_one[:], label="Filter channel")
plt.legend()
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.show()

# apply a 3-pole lowpass filter at Nyquist frequency
b, a = signal.butter(3, 0.1)
fir_two = signal.filtfilt(b, a, fir_one)

# Plot the waveform
plt.plot(time, data[:], label="ECG channel")
plt.plot(time, fir_two[:], label="Filter channel")
plt.legend()
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.show()

# Following code creates a WAV file
# with random short integer bytes of 99999 seconds duration.
obj = wave.open('sound.wav', 'w')
obj.setnchannels(1) # mono
obj.setsampwidth(2)
obj.setframerate(dataRate)

for i in range(99999):
    value = random.randint(-32767, 32767)
    data = struct.pack('<h', value)
    obj.writeframesraw( data )

obj.close()

```

У наведеному прикладі задано втрати не більш ніж $g_{\text{Pass}} = 30$ дБ та у смузі ослаблення забезпечує відсічку щонайменше $g_{\text{Stop}} = 100$ дБ, а крайові частоти смуги пропускання мають значення: $w_p = 0,1$ та $w_s = 3,0$, частота Найквіста $f_s = 1000$. Смугу пропускання фільтр Баттерворта можна також

задати наступним чином: $w_p = [0,1, 0,5]$, $w_s = [0,5, 3,0]$. Комбінований фільтр Баттерворта створений з використанням функції `signal.filtfilt()` має нульову фазу та порядок фільтра, який вдвічі перевищує вихідний.

Відображення повернутих функцією `signal.butter(10, 15, 'hp', fs=1000, output='sos')` коефіцієнтів фільтра Баттерворта, який реалізовано функцією `signal.sosfilt(sos, signal)` має вигляд (рис. 6.5):

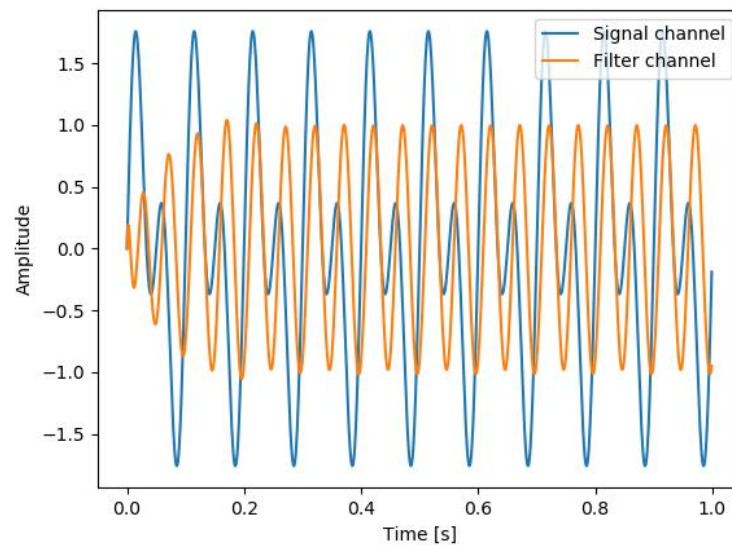


Рисунок 6.5 – Коефіцієнти фільтра Баттерворта

За варіантом (b) значення порядку N і смуга пропускання w_p цифрового фільтра розраховуються з використанням функції `N, Wn = signal.buttord(wp, ws, 30, 100)` для смугового фільтра Баттерворта. Застосування до зашумленого сигналу електрокардіограми функцій фільтрації `b,a = signal.butter(N, Wn, btype='high')` і `signal.filtfilt(b, a, data)` дає наступне відображення обробленого сигналу (рис. 6.6):

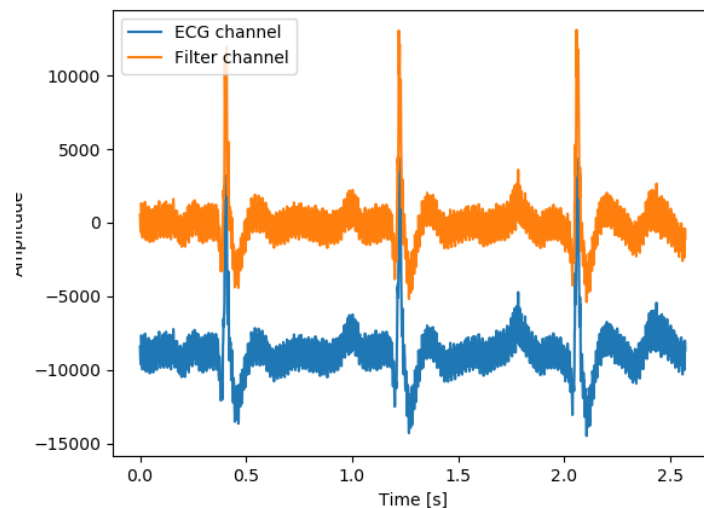


Рисунок 6.6 – Зашумлений сигнал електрокардіограми

За варіантом (с) коефіцієнти цифрового фільтра розраховуються з використанням функції `b,a = signal.butter(3, 0.1)` для смугового фільтра, для якої порядок фільтра $N = 3$ і критична частота або частоти $W_n = 0.1$ Гц. Функція фільтрації `signal.filtfilt(b, a, fir_butter)` має параметр `fir_butter`, що визначається за варіантом проектування (b) із функції `fir_butter = signal.filtfilt(b, a, data)`. Застосування варіанта реалізації фільтра (с) до зашумленого сигналу електрокардіограми дає наступне відображення обробленого сигналу (рис. 6.7):

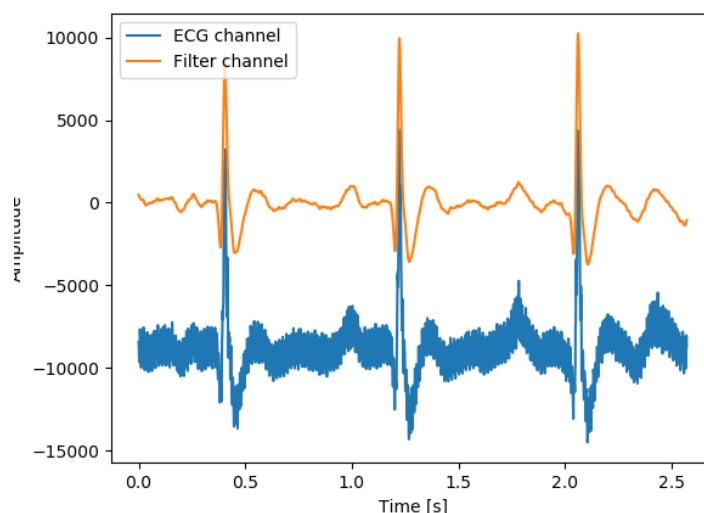


Рисунок 6.7 – Відфільтрований сигнал електрокардіограми

Отже, подвійна фільтрація зашумленого сигналу електрокардіограми з використанням фільтра Баттерворта (b, c) дає змогу очистити сигнал від високочастотних завад у смузі пропускання.

6.3. Завдання для самостійної роботи

На виході сенсору реєструється гармонійний сигнал $\sin(x)$, який зашумлений завадою, що має щільність ймовірності шуму рівну щільності ймовірності нормального розподілу, також відомого як гауссівський шум. Гауссівський шум отримано з використанням функції `gauss(0.0, sigma)` із бібліотеки `Random`:

```
from math import sin
from random import gauss

def generate_sensor_data(n, sigma=0.01):
    """
    Generate data from a fake sensor. These data are generated
    according to a sine function, with zero-mean
    Gaussian noise, with standard deviation sigma.
    """
    data = []
    for i in range(n):
        data.append(sin(i * 0.01) + gauss(0.0, sigma))
    return data

if __name__ == '__main__':
    # Print out a few of the numbers
    for d in generate_sensor_data(10):
        print(d)
```

Необхідно реалізувати цифровий фільтр `mean_filter()` із смугою змінної ширини, який буде змінювати кожне значення вихідного сигналу на середнє значення з підмножини, що має змінну ширину та визначається амплітудами сигналу за деякий час.

Отже параметрами функції фільтра `mean_filter(data, w=3)` є значення: `data[start : end]` – вектор чисел, `w = 3` – додатне та непарне число, яке визначає шириною підмножини. Визначення функції фільтра може мати вигляд:

```
import csv
from sensor import generate_sensor_data
from amp_filter import apply_amp_filter
from statistics import mean
```

```

def mean_filter(data, w=3):
    """
    Replace each reading with mean of a variable width subset in
    which reading is the center.
    :param data: A list of numbers
    :param w: A positive and odd number which is the subset width
    :return: A new list of numbers
    """

    filtered = []
    final_start = int(len(data) - w + 1)
    end = w

    for number in range(final_start):
        start = number
        subset = data[start:end]
        new_datum = mean(subset)
        filtered.append(new_datum)
        end += 1

    if w > len(data) or w < 0 or w % 2 == 0:
        raise ValueError

    return filtered

if __name__ == '__main__':

    sensor_data = generate_sensor_data(1000)
    amp_filter_data = apply_amp_filter(sensor_data)
    mean_data = mean_filter(amp_filter_data)
    all_data = zip(sensor_data[1:1000], amp_filter_data[1:1000],
mean_data)

    with open('mean_filter.csv', 'w') as f:
        csv_file = csv.writer(f)
        csv_file.writerows(all_data)

    test_data = (0, 1, 2, 3, 4, 5, 6, 7, 8)

    for i in range(-3, 11):
        try:
            new_data = mean_filter(test_data, i)
            print('For test_data and w =', i, 'output:', new_data)
        except ValueError:
            print('Exception thrown for: test_data and w =', i)

```

Числова підмножина, яка визначається амплітудами вихідного сигналу та має змінну ширину, розраховується за допомогою функції `apply_amp_filter(data, cutoff=0.8)`, у якій параметр `cutoff=0.8` визначає мінімальний поріг значень амплітуд сигналу, що увійдуть до множини:

```

def apply_amp_filter(data, cutoff=0.8):
    """
    Apply an amplitude filter to an iterable, returning a list.
    """

```

```

# Make an empty list
filtered = []

# Add one element at a time to the list. There are a lot of ways
to do this; this is just one of them.
for datum in data:
    datum = min(datum, cutoff)
    filtered.append(datum)

# Return the new list
return filtered

```

Нарешті можемо зібрати цифровий фільтр у вигляді послідовного застосування функцій: генерації зашумленого гармонійного сигналу `sensor_data = generate_sensor_data(1000)`; визначення значень амплітуд сигналу, що увійдуть до медіанної підмножини `amp_filter_data = apply_amp_filter(sensor_data)`; і самого медіанного фільтру `mean_data = mean_filter(amp_filter_data)`. Отже, основний виконавчий модуль буде мати такий вигляд:

```

import csv
from sensor import generate_sensor_data
from amp_filter import apply_amp_filter

if __name__ == '__main__':

    # Generate 1000 points of sensor data, then run them through the
    amp filter
    # then zip to get first column as sensor data and second column
    as (amp) filtered sensor data.
    sensor_data = generate_sensor_data(1000)
    amp_filter_data = apply_amp_filter(sensor_data)
    data = zip(sensor_data, amp_filter_data)

    with open('filter.csv', 'w') as f:
        csv_file = csv.writer(f)
        csv_file.writerows(data)

    # An alternative to 'csv_file.writerows(data)' would have
    been:
    # for row in data:
    #     csv_file.writerow(row)

```

Наприкінці програмного коду, відфільтровані дані записуються до файлу `'filter.csv'` із використанням функцій `csv.writer()` і `writerows(data)` з бібліотеки CSV.

Під час процедури фільтрації відбувається процес звуження ширини медіанної підмножини:

```
For test_data and w = 1 output: [0, 1, 2, 3, 4, 5, 6, 7, 8]
Exception thrown for: test_data and w = 2
For test_data and w = 3 output: [1, 2, 3, 4, 5, 6, 7]
Exception thrown for: test_data and w = 4
For test_data and w = 5 output: [2, 3, 4, 5, 6]
Exception thrown for: test_data and w = 6
For test_data and w = 7 output: [3, 4, 5]
Exception thrown for: test_data and w = 8
For test_data and w = 9 output: [4]
```

6.4. Контрольні питання

1. Що таке апроксимація фільтрів?
2. Що таке комплексна частотна характеристика?
3. Для чого застосовується дискретне перетворення Фур'є?
4. Як реалізувати фільтр Баттерворта N-го порядку?
5. За якими параметрами задається смуга пропускання фільтр Баттерворта?
6. Що таке гауссівський шум?

ПРАКТИЧНА РОБОТА № 7

КЛАСИФІКАЦІЯ ДАНИХ ЗА ДОПОМОГОЮ НЕЙРОМЕРЕЖ З ВИКОРИСТАННЯМ ФУНКЦІЙ БІБЛІОТЕКИ SKLEARN ТА SCIPY

Мета заняття: отримати навички застосування нейронних мереж для класифікації даних використовуючи мову програмування Python.

7.1. Основні теоретичні відомості

Основним інструментом для класифікації та розпізнавання об'єктів, облич на фотографіях, розпізнавання мови є згортова нейронна мережа (Convolutional Neural Network – CNN). Реалізовано безліч варіантів застосування CNN, такі як Deep Convolutional Neural Network (DCNN), Region-CNN (R-CNN), Fully Convolutional Neural Networks (FCNN), Mask R-CNN та інші.

В основі згорткових шарів нейронних мереж лежить операція згортки. Згортка – це процес додавання кожного елемента зображення до сусідніх фремів зображення, зважених ядром. Важливо зауважити, що виконувана матрична операція згортки є не звичайним множенням, хоча і позначається знаком " * ". Архітектура згорткової нейромережі містить згоркові шари та агрегувальні шари або шари субдискретизації, які є одним з основних структурних елементів згорткових нейронних мереж (рис. 7.1) [7].

Наприклад, якщо є дві 3x3 матриці, перша – ядро, друга – фрагмент зображення, процес згортки полягатиме у транспонуванні рядків та стовпчиків ядра з наступним множенням і додаванням. Центральний елемент отриманого зображення з координатами [2, 2] буде зваженою комбінацією всіх елементів матриці зображення, з вагами взятими з ядра:

$$\left(\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} * \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \right) [2, 2] = (i \cdot 1) + (h \cdot 2) + (g \cdot 3) + (f \cdot 4) + (e \cdot 5) + (d \cdot 6) + (c \cdot 7) + (b \cdot 8) + (a \cdot 9).$$

Такі шари можуть бути як глобальними, так і локальними, тобто розповсюджуватись тільки на окремі групи вхідних даних, які будуть рецептивними полями для нейронів поточного шару (рис. 7.1).

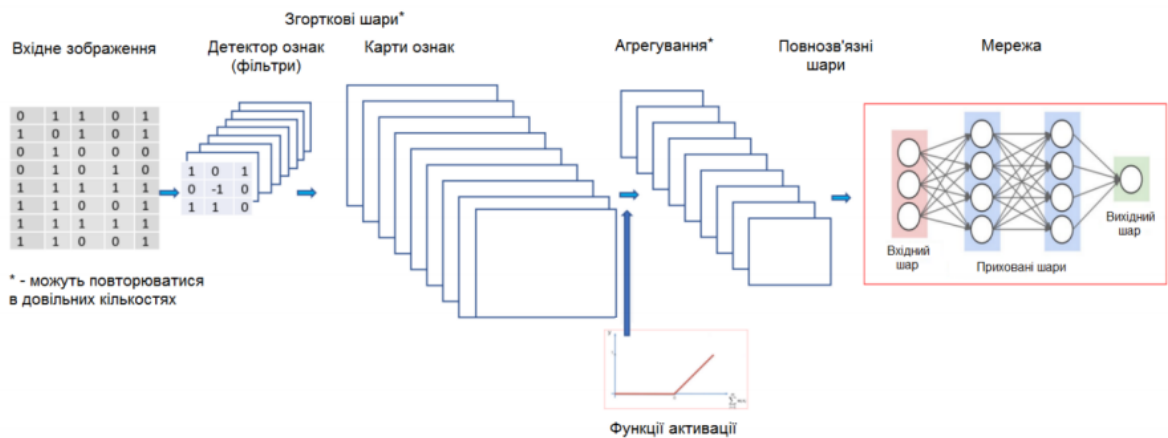


Рисунок 7.1 – Архітектура згорткової неймережі

Значення кожного пікселя у вихідному зображенні, які складають суму вхідних ознак, дорівнює сумі добутків значень у матриці згортки та відповідних пікселів вхідного зображення, які формують карту ознак у загортковому шарі (рис. 7.2) [7].



Рисунок 7.2 – Згортковий шар

Головною задачею агрегувальних шарів є зменшення розмірності даних з одночасним збереженням найважливіших характеристик шляхом формування залежності між кількома елементами (нейронами) з попереднього шару з одним елементом даного шару. Тому при побудові

мережі вони зазвичай використовуються з певною періодичністю між згортковими шарами. Агрегувальні шари бувають трьох підтипів: мінімізаційні (minimal), усереднювальні (average), максимізаційні (maximal). Найбільш розповсюджено використання максимізаційних шарів (рис. 7.3).

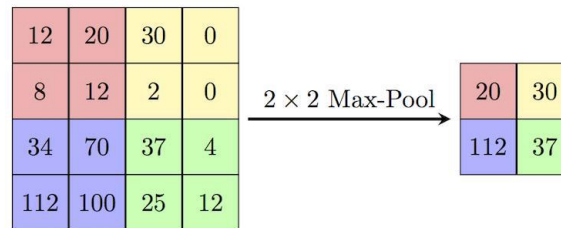


Рисунок 7.3 – Агрегувальний шар

Для поєднання нейронів попереднього шару з нейронами наступного шару використовуються повнозв'язні рівні (fully-connected layers), які мають поєднані зв'язки з нейронами шарів. Використання таких рівнів на початкових та прихованих рівнях мережі невиправдане, адже воно ускладнює модель і навіть може ігнорувати знайдені раніше за допомогою згорток та агрегації ознаки та закономірності. Проте, повнозв'язні рівні зазвичай використовуються на передостанньому кроці роботи мережі для підготовки знаходження результатів на виході мережі.

Найпростіший видом нейронної мережі є персептрон (Perceptron). В основі такої нейромережі лежить математична модель сприйняття інформації мозком, що складається із сенсорів, асоціативних та реагуючих елементів. Розглянемо нейронну мережу прямого поширення (MLP) на основі багатошарових персептронів. Мережу MLP можна розглядати як узагальнення лінійних моделей, яка виконує кілька етапів обробки даних.

Наприклад, у лінійній регресії прогноз отримують за допомогою наступної формули [9]:

$$\hat{y} = w[0] * x[0] + w[1] * x[1] + \dots + w[p] * x[p] + b,$$

де $\hat{y}$ – зважена сума вхідних ознак $x[0] \dots x[p]$,

$w[0] \dots w[p]$ – вагові коефіцієнти, що зважують вхідні ознаки та обчислюються під час навчання мережі.

Зв'язок між вхідними ознаками та сумою вихідних ознак можна представити графічно (рис. 7.4) [9].

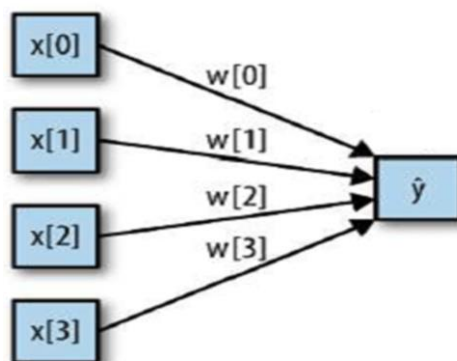


Рисунок 7.4 – Візуалізація логістичної регресії, в якій вхідні ознаки і прогнози показані в вигляді вузлів, а коефіцієнти – у вигляді сполучень між вузлами

У нейромережі MLP процес обчислення зважених сум повторюється декілька разів. Спочатку обчислюються приховані елементи (hidden units), які являють собою проміжний етап обробки. Вони знову об'єднуються за допомогою зважених сум для отримання кінцевого результату (рис. 7.5.) [9].

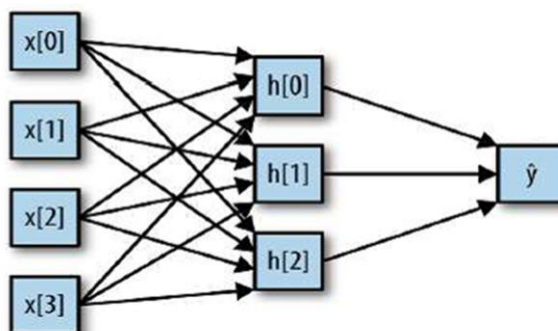


Рисунок 7.5 – Візуалізація багатошарового персептрона з одним прихованим шаром

Під час обчислення зваженої суми входів для кожного елемента прихованого шару, до неї застосовується функція активації – зазвичай використовуються нелінійні функції випрямлений лінійний елемент (rectified linear unit або relu) або гіперболічний тангенс (hyperbolic tangent або tanh). Як результат отримуємо виходи нейронів прихованого шару. Ці проміжні

виходи можуть вважатися нелінійними перетвореннями і комбінаціями первинних входів. Вони стають входами вихідного шару. Знову обчислюється зважена сума входів, застосовується функція активації і отримуються підсумкові значення цільової змінної.

Функції активації relu та $\tanh$ показані на рис. 7.6. Функція активації relu відсікає значення нижче нуля, в той час як функція $\tanh$ приймає значення від -1 до 1 (відповідно для мінімального і максимального значень входів мережі).

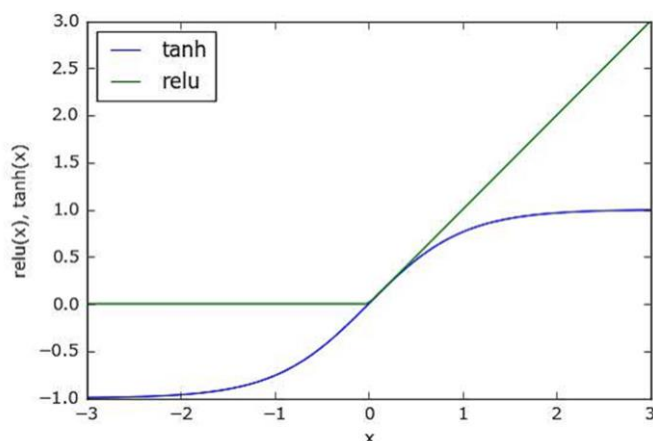


Рисунок 7.6 – Функція активації:

гіперболічний тангенс ($\tanh$), випрямлений лінійний елемент (relu)

Вагові коефіцієнти прихованих елементів та зважена сума вхідних ознак на основі функції активації $\tanh$ [9]:

$$h[0] = \tanh(w[0,0] * x[0] + w[1,0] * x[1] + w[2,0] * x[2] + w[3,0] * x[3])$$

$$h[1] = \tanh(w[0,0] * x[0] + w[1,0] * x[1] + w[2,0] * x[2] + w[3,0] * x[3])$$

$$h[2] = \tanh(w[0,0] * x[0] + w[1,0] * x[1] + w[2,0] * x[2] + w[3,0] * x[3])$$

$$\hat{y} = v[0] * h[0] + v[1] * h[1] + v[2] * h[2]$$

де h – проміжні обчислення,

w – ваги між входом x і прихованим шаром h ,

v – вагові коефіцієнти між прихованим шаром h і виходом $\hat{y}$.

Значення x є вхідними ознаками, ваги v і w обчислюються на основі даних і функції активації, а $u^{\wedge}$ є обчислений вихід, у вигляді зваженої суми вхідних ознак.

Важливою властивістю нейронних мереж є те, що їх ваги задаються випадковим чином перед початком навчання, а випадкова ініціалізація впливає на процес навчання моделі. Це означає, що навіть при використанні одних і тих же параметрів можна отримати дуже різні моделі, задаючи різні стартові значення генератора псевдовипадкових чисел. Одна з базових переваг нейронних мереж полягає в тому, що вони здатні обробляти інформацію, що міститься у великих обсягах даних, і будувати неймовірно складні моделі. При наявності достатнього часу обчислень, даних і ретельного налаштування параметрів нейронні мережі часто перевершують інші алгоритми машинного навчання. Але нейронні мережі, особливо великі нейронні мережі, як правило, вимагають тривалого часу навчання.

На мові Python є бібліотеки машинного навчання SciPy.special та PyTorch, які особливо корисні для розробки систем штучного інтелекту, обробки природної мови та аналізу часових рядів. Наприклад, бібліотека PyTorch розроблена групою дослідження штучного інтелекту компанії Facebook та є вільним та відкритим програмним забезпеченням.

Детально з особливостями PyTorch можна ознайомитись в офіційній документації на сайті «PyTorch.org» <https://pytorch.org/docs/stable/index.html>.

7.2. Порядок роботи

Підключити бібліотеку SciPy.special, що містить спеціальні функції, можливості та поняття з математичної фізики:

```
from scipy.stats import truncnorm
...
```

Структура нейронної мережі описується у класі `class NeuralNetwork`: вхідні вузли мережі `no_of_in_nodes`, вихідні вузли мережі `no_of_out_nodes`, вузли скритих шарів мережі `no_of_hidden_nodes`, вагові коефіцієнти `create_weight_matrices()`, логістична або порогова функція `learning_rate`.

```
def __init__(self,
              no_of_in_nodes,
              no_of_out_nodes,
              no_of_hidden_nodes,
              learning_rate):
    self.no_of_in_nodes = no_of_in_nodes
    self.no_of_out_nodes = no_of_out_nodes
    self.no_of_hidden_nodes = no_of_hidden_nodes
    self.learning_rate = learning_rate
    self.create_weight_matrices()
...

simple_network = NeuralNetwork(no_of_in_nodes=2,
                              no_of_out_nodes=3,
                              no_of_hidden_nodes=5,
                              learning_rate=0.3)
```

Функція вагових коефіцієнтів нейронної мережі включає метод `truncated_normal` із бібліотеки `SciPy.special`, який виводить випадкові значення з усіченого нормального розподілу для визначення вагових коефіцієнтів мережі:

```
from scipy.stats import truncnorm
...

def create_weight_matrices(self):
    """ A method to initialize the weight matrices of the neural
    network """
    rad = 1 / np.sqrt(self.no_of_in_nodes)
    X = truncated_normal(mean=0, sd=1, low=-rad, upp=rad)
    self.weights_in_hidden = X.rvs((self.no_of_hidden_nodes,
                                    self.no_of_in_nodes))
    rad = 1 / np.sqrt(self.no_of_hidden_nodes)
    X = truncated_normal(mean=0, sd=1, low=-rad, upp=rad)
    self.weights_hidden_out = X.rvs((self.no_of_out_nodes,
                                    self.no_of_hidden_nodes))
```

Для навчання нейронної мережі застосовується тренувальна функція `train(self, input_vector, target_vector)` з тіла якої викликаються функції активації `activation_function()` для обчислення вагових коефіцієнтів у скритому шарі мережі на основі вектору даних на вхідних вузлах

`input_vector()` і на основі вектору даних на вихідних вузлах `output_vector_hidden()`. Вектор вагових коефіцієнтів `weights_hidden_out()` визначається з урахуванням значень порогової функції `learning_rate()`:

```
from scipy.special import expit as activation_function
...
def train(self, input_vector, target_vector):
    """
    input_vector and target_vector can be tuples, lists or ndarrays
    """
    # make sure that the vectors have the right shape
    input_vector = np.array(input_vector)
    input_vector = input_vector.reshape(input_vector.size, 1)
    target_vector =
np.array(target_vector).reshape(target_vector.size, 1)

    output_vector_hidden = activation_function(self.weights_in_hidden
@ input_vector)
    output_vector_network =
activation_function(self.weights_hidden_out @ output_vector_hidden)

    output_error = target_vector - output_vector_network
    tmp = output_error * output_vector_network * (1.0 -
output_vector_network)
    self.weights_hidden_out += self.learning_rate * (tmp @
output_vector_hidden.T)

    # calculate hidden errors:
    hidden_errors = self.weights_hidden_out.T @ output_error
    # update the weights:
    tmp = hidden_errors * output_vector_hidden * (1.0 -
output_vector_hidden)
    self.weights_in_hidden += self.learning_rate * (tmp @
input_vector.T)
```

Для здійснення класифікації фактичних даних нейронною мережею реалізовано функцію `run(self, input_vector)` з тіла якої також викликаються функції активації `activation_function()` для перевизначення вагових коефіцієнтів у скритому шарі мережі на основі вхідного вектору даних `input_vector()` та на основі вектору вагових коефіцієнтів мережі `weights_hidden_out()`:

```
def run(self, input_vector):
    """
    running the network with an input vector 'input_vector'.
    'input_vector' can be tuple, list or ndarray
    """
    # make sure that input_vector is a column vector:
    input_vector = np.array(input_vector)
    input_vector = input_vector.reshape(input_vector.size, 1)
```

```

        input4hidden = activation_function(self.weights_in_hidden @
input_vector)
        output_vector_network =
activation_function(self.weights_hidden_out @ input4hidden)
        return output_vector_network

```

Для оцінювання результативності класифікації даних нейронною мережею реалізовано функцію `evaluate(self, data, labels)`, яка підраховує, як часто фактичний результат відповідає цільовому результату:

```

def evaluate(self, data, labels):
    """
    Counts how often the actual result corresponds to the
    target result.
    A result is considered to be correct, if the index of
    the maximal value corresponds to the index with the "1"
    in the one-hot representation,
    e.g.
    res = [0.1, 0.132, 0.875]
    labels[i] = [0, 0, 1]
    """
    corrects, wrongs = 0, 0
    for i in range(len(data)):
        res = self.run(data[i])
        res_max = res.argmax()
        if res_max == labels[i].argmax():
            corrects += 1
        else:
            wrongs += 1
    return corrects, wrongs

```

Результат вважається прийнятним, якщо максимальне значення функції `evaluate()` відповідає індексу з номером «1», наприклад:

```

res = [0.1, 0.132, 0.875]
labels[i] = [0, 0, 1]

```

У підсумку програмний код реалізації нейронної мережі на основі методу Backpropagation забезпечує навчання, тренування, класифікацію фактичних даних, і оцінювання роботи мережі:

```

# Backpropagation in Neural Networks
import numpy as np
from scipy.special import expit as activation_function
from scipy.stats import truncnorm
import matplotlib.pyplot as plt

```

```

from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split

# create the test data
n_samples = 500
blob_centers = ([2, 6], [6, 2], [7, 7])
n_classes = len(blob_centers)
data, labels = make_blobs(n_samples=n_samples,
                          centers=blob_centers,
                          random_state=7)

colours = ('green', 'red', "yellow")
fig, ax = plt.subplots()

# visualize the previously created data
for n_class in range(n_classes):
    ax.scatter(data[labels==n_class][:, 0],
              data[labels==n_class][:, 1],
              c=colours[n_class],
              s=40,
              label=str(n_class))

plt.show()

# One-Hot Representation
labels = np.arange(n_classes) == labels.reshape(labels.size, 1)
labels = labels.astype(np.float64)
print('train data ', labels[:7])

# to create a train and a test data set:
res = train_test_split(data, labels,
                      train_size=0.8,
                      test_size=0.2,
                      random_state=42)
train_data, test_data, train_labels, test_labels = res
print('test data ', train_labels[:10])

# create a neural network with two input nodes, and three output
nodes
def truncated_normal(mean=0, sd=1, low=0, upp=10):
    return truncnorm(
        (low - mean) / sd, (upp - mean) / sd, loc=mean, scale=sd)

class NeuralNetwork:

    def __init__(self,
                 no_of_in_nodes,
                 no_of_out_nodes,
                 no_of_hidden_nodes,
                 learning_rate):
        self.no_of_in_nodes = no_of_in_nodes
        self.no_of_out_nodes = no_of_out_nodes
        self.no_of_hidden_nodes = no_of_hidden_nodes
        self.learning_rate = learning_rate
        self.create_weight_matrices()

    def create_weight_matrices(self):
        """ A method to initialize the weight matrices of the neural
        network"""
        rad = 1 / np.sqrt(self.no_of_in_nodes)
        X = truncated_normal(mean=0, sd=1, low=-rad, upp=rad)
        self.weights_in_hidden = X.rvs((self.no_of_hidden_nodes,
                                         self.no_of_in_nodes))
        rad = 1 / np.sqrt(self.no_of_hidden_nodes)

```

```

X = truncated_normal(mean=0, sd=1, low=-rad, upp=rad)
self.weights_hidden_out = X.rvs((self.no_of_out_nodes,
                                   self.no_of_hidden_nodes))

def train(self, input_vector, target_vector):
    """
    input_vector and target_vector can be tuples, lists or
ndarrays
    """
    # make sure that the vectors have the right shape
    input_vector = np.array(input_vector)
    input_vector = input_vector.reshape(input_vector.size, 1)
    target_vector =
np.array(target_vector).reshape(target_vector.size, 1)

    output_vector_hidden =
activation_function(self.weights_in_hidden @ input_vector)
    output_vector_network =
activation_function(self.weights_hidden_out @ output_vector_hidden)

    output_error = target_vector - output_vector_network
    tmp = output_error * output_vector_network * (1.0 -
output_vector_network)
    self.weights_hidden_out += self.learning_rate * (tmp @
output_vector_hidden.T)

    # calculate hidden errors:
    hidden_errors = self.weights_hidden_out.T @ output_error
    # update the weights:
    tmp = hidden_errors * output_vector_hidden * (1.0 -
output_vector_hidden)
    self.weights_in_hidden += self.learning_rate * (tmp @
input_vector.T)

def run(self, input_vector):
    """
    running the network with an input vector 'input_vector'.
    'input_vector' can be tuple, list or ndarray
    """
    # make sure that input vector is a column vector:
    input_vector = np.array(input_vector)
    input_vector = input_vector.reshape(input_vector.size, 1)
    input4hidden = activation_function(self.weights_in_hidden @
input_vector)
    output_vector_network =
activation_function(self.weights_hidden_out @ input4hidden)
    return output_vector_network

def evaluate(self, data, labels):
    """
    Counts how often the actual result corresponds to the
target result.
    A result is considered to be correct, if the index of
the maximal value corresponds to the index with the "1"
in the one-hot representation,
e.g.
res = [0.1, 0.132, 0.875]
labels[i] = [0, 0, 1]
    """
    corrects, wrongs = 0, 0
    for i in range(len(data)):
        res = self.run(data[i])
        res_max = res.argmax()

```

```

        if res_max == labels[i].argmax():
            corrects += 1
        else:
            wrongs += 1
    return corrects, wrongs

# Use one output node for each class
simple_network = NeuralNetwork(no_of_in_nodes=2,
                               no_of_out_nodes=3,
                               no_of_hidden_nodes=5,
                               learning_rate=0.3)

# consists in training our network with the data
for i in range(len(train_data)):
    simple_network.train(train_data[i], train_labels[i])

# use the evaluate function:
print('evaluate ', simple_network.evaluate(train_data, train_labels))

```

Візуалізацію процесу класифікації даних реалізовано з використанням функцій `make_blobs()` та `plt.subplots()` із бібліотеки `Sklearn.datasets` та `Matplotlib.pyplot`, відповідно (рис. 7.7):

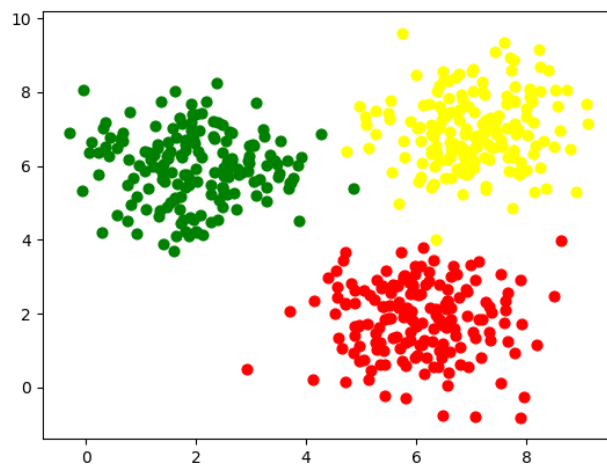


Рисунок 7.7 – Візуалізація процесу класифікації даних за методом Backpropagation

Результат виконання програми з нейронною мережею за методом Backpropagation є таким:

```

train data  [[0. 0. 1.]
             [0. 0. 1.]
             [0. 1. 0.]
             [1. 0. 0.]
             [0. 0. 1.]
             [1. 0. 0.]
             [0. 1. 0.]]
test data   [[0. 0. 1.]

```



```

[0. 1. 0.]
[1. 0. 0.]
[0. 0. 1.]
[0. 0. 1.]
[1. 0. 0.]
[0. 1. 0.]
[1. 0. 0.]
[1. 0. 0.]
[0. 0. 1.]]
evaluate (392, 8)

```

7.3. Завдання для самостійної роботи

Реалізувати нейронну мережу за методом Backpropagation з вузлами зміщення **Bias**. Структуру такої нейронної мережі можна також описати у класі `class NeuralNetwork`: вхідні вузли мережі `no_of_in_nodes`, вихідні вузли мережі `no_of_out_nodes`, вузли скритих шарів мережі `no_of_hidden_nodes`, вагові коефіцієнти `create_weight_matrices()`, логістична або порогова функція `learning_rate`, вузли зміщення `bias`.

```

def __init__(self,
               no_of_in_nodes,
               no_of_out_nodes,
               no_of_hidden_nodes,
               learning_rate,
               bias=None):
    self.no_of_in_nodes = no_of_in_nodes
    self.no_of_hidden_nodes = no_of_hidden_nodes
    self.no_of_out_nodes = no_of_out_nodes
    self.learning_rate = learning_rate
    self.bias = bias
    self.create_weight_matrices()

```

Вузли зміщення **Bias** можна описати у тілі функції вагових коефіцієнтів нейронної мережі `create_weight_matrices(self)`, яка включає метод `truncated_normal` із бібліотеки `SciPy.special`, що виводить випадкові значення з усіченого нормального розподілу для визначення вагових коефіцієнтів мережі:

```

def create_weight_matrices(self):
    """ A method to initialize the weight matrices of the neural
    network with optional bias nodes"""
    bias_node = 1 if self.bias else 0
    rad = 1 / np.sqrt(self.no_of_in_nodes + bias_node)
    X = truncated_normal(mean=0, sd=1, low=-rad, upp=rad)
    self.weights_in_hidden = X.rvs((self.no_of_hidden_nodes,

```

```

                                self.no_of_in_nodes + bias_node))
rad = 1 / np.sqrt(self.no_of_hidden_nodes + bias_node)
X = truncated normal(mean=0, sd=1, low=-rad, upp=rad)
self.weights_hidden_out = X.rvs((self.no_of_out_nodes,
                                self.no_of_hidden_nodes +
bias_node))

```

Остаточний вигляд програмного коду, який наведено для самостійної перевірки результатів навчання, може бути такий:

```

# Neural Network with Bias Nodes
import numpy as np
from scipy.stats import truncnorm
from scipy.special import expit as activation_function

import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

from sklearn.model_selection import train_test_split

# create the test data
n_samples = 500
blob_centers = ([2, 6], [6, 2], [7, 7])
n_classes = len(blob_centers)
data, labels = make_blobs(n_samples=n_samples,
                           centers=blob_centers,
                           random_state=7)

colours = ('green', 'red', "yellow")
fig, ax = plt.subplots()

# visualize the previously created data
for n_class in range(n_classes):
    ax.scatter(data[labels==n_class][:, 0],
               data[labels==n_class][:, 1],
               c=colours[n_class],
               s=40,
               label=str(n_class))

plt.show()

# One-Hot Representation
labels = np.arange(n_classes) == labels.reshape(labels.size, 1)
labels = labels.astype(np.float64)
print('train data ', labels[:7])

# to create a train and a test data set:
res = train_test_split(data, labels,
                        train_size=0.8,
                        test_size=0.2,
                        random_state=42)
train_data, test_data, train_labels, test_labels = res
print('test data ', train_labels[:10])

# Create Neural Network with Bias Nodes
def truncated_normal(mean=0, sd=1, low=0, upp=10):
    return truncnorm(
        (low - mean) / sd, (upp - mean) / sd, loc=mean, scale=sd)

class NeuralNetwork:

    def __init__(self,

```

```

        no_of_in_nodes,
        no_of_out_nodes,
        no_of_hidden_nodes,
        learning_rate,
        bias=None):
    self.no_of_in_nodes = no_of_in_nodes
    self.no_of_hidden_nodes = no_of_hidden_nodes
    self.no_of_out_nodes = no_of_out_nodes
    self.learning_rate = learning_rate
    self.bias = bias
    self.create_weight_matrices()

    def create_weight_matrices(self):
        """ A method to initialize the weight matrices of the neural
        network with optional bias nodes"""
        bias_node = 1 if self.bias else 0
        rad = 1 / np.sqrt(self.no_of_in_nodes + bias_node)
        X = truncated_normal(mean=0, sd=1, low=-rad, upp=rad)
        self.weights_in_hidden = X.rvs((self.no_of_hidden_nodes,
                                         self.no_of_in_nodes +
bias_node))
        rad = 1 / np.sqrt(self.no_of_hidden_nodes + bias_node)
        X = truncated_normal(mean=0, sd=1, low=-rad, upp=rad)
        self.weights_hidden_out = X.rvs((self.no_of_out_nodes,
                                         self.no_of_hidden_nodes +
bias_node))

    def train(self, input_vector, target_vector):
        """ input_vector and target_vector can be tuple, list or
        ndarray """

        # make sure that the vectors have the right shap
        input_vector = np.array(input_vector)
        input_vector = input_vector.reshape(input_vector.size, 1)
        if self.bias:
            # adding bias node to the end of the input vector
            input_vector = np.concatenate((input_vector,
[[self.bias]]))
        target_vector =
np.array(target_vector).reshape(target_vector.size, 1)

        output_vector_hidden =
activation_function(self.weights_in_hidden @ input_vector)
        if self.bias:
            output_vector_hidden =
np.concatenate((output_vector_hidden, [[self.bias]]))
        output_vector_network =
activation_function(self.weights_hidden_out @ output_vector_hidden)

        output_error = target_vector - output_vector_network
        # update the weights:
        tmp = output_error * output_vector_network * (1.0 -
output_vector_network)
        self.weights_hidden_out += self.learning_rate * (tmp @
output_vector_hidden.T)

        # calculate hidden errors:
        hidden_errors = self.weights_hidden_out.T @ output_error
        # update the weights:
        tmp = hidden_errors * output_vector_hidden * (1.0 -
output_vector_hidden)
        if self.bias:
            x = (tmp @ input_vector.T)[:,-1, :] # last row cut off,

```

```

        else:
            x = tmp @ input_vector.T
            self.weights_in_hidden += self.learning_rate * x

    def run(self, input_vector):
        """
        running the network with an input vector 'input_vector'.
        'input_vector' can be tuple, list or ndarray
        """
        # make sure that input_vector is a column vector:
        input_vector = np.array(input_vector)
        input_vector = input_vector.reshape(input_vector.size, 1)
        if self.bias:
            # adding bias node to the end of the input_vector
            input_vector = np.concatenate((input_vector, [[1]]))
        input4hidden = activation_function(self.weights_in_hidden @
input_vector)
        if self.bias:
            input4hidden = np.concatenate((input4hidden, [[1]]))
        output_vector_network =
activation_function(self.weights_hidden_out @ input4hidden)
        return output_vector_network

    def evaluate(self, data, labels):
        corrects, wrongs = 0, 0
        for i in range(len(data)):
            res = self.run(data[i])
            res_max = res.argmax()
            if res_max == labels[i].argmax():
                corrects += 1
            else:
                wrongs += 1
        return corrects, wrongs

# Use one output node for each class
simple_network = NeuralNetwork(no_of_in_nodes=2,
                               no_of_out_nodes=3,
                               no_of_hidden_nodes=5,
                               learning_rate=0.1,
                               bias=1)

# consists in training our network with the data
for i in range(len(train_data)):
    simple_network.train(train_data[i], train_labels[i])

# use the evaluate function:
print('evaluate ', simple_network.evaluate(train_data, train_labels))

```

Результат візуалізації процесу класифікації даних нейронною мережею за методом Backpropagation з вузлами зміщення Bias (рис. 7.8):

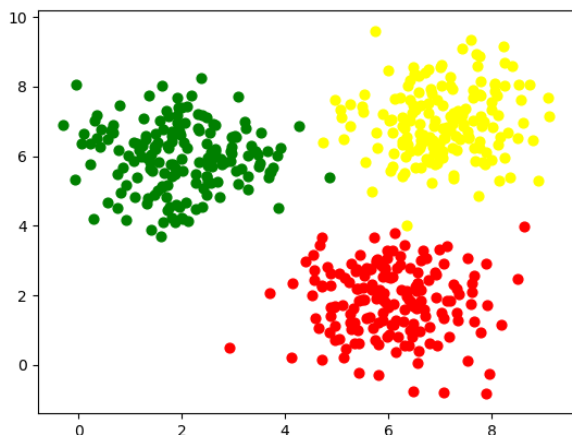


Рисунок 7.8 – Візуалізація процесу класифікації даних за методом Backpropagation з вузлами зміщення Bias

Результат оцінювання результатів класифікації даних:

```
evaluate (390, 10)
```

7.4. Контрольні питання

1. Що таке нейронна мережа?
2. У чому особливість згорткової нейронної мережі (CNN)?
3. У чому особливість нейронної мережі прямого поширення (MLP)?
4. У чому особливість методу Backpropagation?
5. У чому особливість методу Backpropagation з вузлами зміщення Bias?
6. Які параметри впливають на точність класифікації даних?

ПРАКТИЧНА РОБОТА № 8

МАТЕМАТИЧНА СТАТИСТИКА З ВИКОРИСТАННЯМ ФУНКЦІЙ МОДУЛЮ STATS БІБЛІОТЕКИ SCIPY

Мета заняття: отримати навички статистичного аналізу даних використовуючи мову програмування Python.

8.1. Основні теоретичні відомості

Описову статистику можна застосовувати до одного або кількох наборів даних або змінних. Коли описують та обчислюють характеристики однієї змінної, то виконують одномірний аналіз. Якщо шукають статистичні зв'язки між парою змінних, роблять двомірний аналіз. Аналогічним чином, багатовимірний аналіз пов'язаний із декількома змінними одночасно.

Функції опису із бібліотеки SciPy автоматично обчислюють основну статистику для всіх безперервних змінних [10]:

- кількість одиниць змінної (the count of that variable);
- середнє значення (the mean);
- стандартне відхилення (the standard deviation – std);
- мінімальне значення (the minimum value);
- міжквартильний діапазон IQR (the interquartile range: 25%, 50%, 75%);
- максимальне значення (the maximum value).

Підрахунок значень – це хороший спосіб зрозуміти, скільки одиниць кожної характеристики або змінної мають місце. Цей метод "value_counts" працює лише на одновимірних масивах Series Pandas, які містять дані будь-якого типу (цілі числа, рядки, числа з плаваючою точкою, об'єкти python тощо), але не працює на фреймах даних Pandas.

Кореляція – міра ступеня взаємозалежності між змінними, яка встановлює ймовірність причинно-наслідкового зв'язку: зв'язок між

причиною і наслідком між двома змінними. Кореляція не означає прямого причинно-наслідкового зв'язку. Наприклад, кореляція Пірсона вимірює лінійну залежність між двома змінними X і Y , де отриманий коефіцієнт є значенням від -1 до 1 включно, де: 1 – ідеальна позитивна лінійна кореляція; 0 – відсутність лінійної кореляції, коли дві змінні, швидше за все, не впливають одна на одну; -1 – ідеальна негативна лінійна кореляція.

Кореляція Пірсона – це стандартний метод функції «corr». Найбільш загальновідомою мірою залежності між двома величинами є коефіцієнт кореляції Пірсона PPMCC (Pearson product-moment correlation coefficient), який зазвичай називають коефіцієнт кореляції (the correlation coefficient). Його отримують взяттям відношення коваріації двох досліджуваних змінних чисельного набору даних, унормованої квадратним коренем їхніх дисперсій. Математично, коваріацію цих двох змінних просто ділять на добуток їхніх стандартних відхилень. Для обчислення коефіцієнту кореляції Пірсона використовується формула [10]:

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}},$$

де r – коефіцієнт кореляції,

x_i – значення x -ої змінної у вибірці $x[]$, y_i – значення y -ої змінної у вибірці $y[]$,

$\bar{x}$ – середнє значення змінної x , $\bar{y}$ – середнє значення змінної y .

Коефіцієнт кореляції Пірсона намагається встановити лінію, яка найкраще наближається до набору даних із двох змінних, викладаючи очікувані значення, а отриманий коефіцієнт кореляції вказує, наскільки далеким від очікуваних значень є фактичний набір експериментальних даних. Залежно від знаку коефіцієнта кореляції Пірсона можна отримати як від'ємну, так і додатну кореляцію, якщо якийсь зв'язок між змінними нашого набору даних існує.

Коефіцієнти рангової кореляції, такі як коефіцієнт рангової кореляції Спірмена та коефіцієнт рангової кореляції Кендалла (τ), визначають, до якої

міри в разі збільшення однієї змінної інша змінна схильна збільшуватися, не вимагаючи, щоби це збільшення було подано лінійною залежністю. Якщо за збільшення однієї змінної інша зменшується, то коефіцієнти рангової кореляції будуть від'ємними. Ці коефіцієнти рангової кореляції часто розглядають як альтернативи коефіцієнту Пірсона, та використовують або для зменшення кількості обчислень, або для того, щоби зробити коефіцієнт менш чутливим до не нормальності в розподілах. Оскільки коефіцієнти рангової кореляції вимірюють інший тип зв'язку, ніж коефіцієнт кореляції Пірсона, їх найкраще розглядати як показники іншого типу зв'язку, а не як альтернативну міру генерального коефіцієнту кореляції.

P-value – це значення ймовірності того, що кореляція між двома змінними є статистично значущою. Зазвичай у медицині вибирається рівень значущості 0,05, який означає впевненість на 95%, що кореляція між змінними є значущою. Цю інформацію можна отримати за допомогою функцій модуля Stats у бібліотеці SciPy.

Наприклад, якщо ми стверджуємо одну гіпотезу і нашою метою є статистична перевірка чи є ця гіпотеза не хибною, але не маємо наміру, одночасно з тим, досліджувати іншу гіпотезу, тоді така перевірка називається перевіркою значимості. Статистична гіпотеза, яка стосується лише числових значень невідомих параметрів певного розподілу називається параметричною гіпотезою. P-value застосовується у контексті перевірки нульової гіпотези для надання кількісної оцінки поняттю статистичної значущості доведення. Зауважте, що статистична значущість результату не означає те, що результат має наукову значимість. По суті, твердження вважається правильним, якщо його протилежне твердження є неймовірним.

Методи перевірки статистичних гіпотез називаються статистичними тестами. Тести, що перевіряють параметричні гіпотези називаються параметричними тестами. Існують три різні перевірки статистичних гіпотез, які використовуються в різних ситуаціях: Т-тест, Z-тест і F-тест.

T-тест використовується, коли стандартне відхилення сукупності невідоме, а розмір вибірки невеликий (зазвичай менш ніж 30 значень). Він передбачає, що дані мають нормальний розподіл. T-критерій зазвичай використовується для порівняння середніх значень двох вибірок або для перевірки того, чи середнє значення однієї вибірки значно відрізняється від гіпотетичного значення.

Z-тест використовується, якщо відоме стандартне відхилення сукупності та розмір вибірки великий (зазвичай більш ніж 30 значень). Подібно до T-критерію, Z-тест використовується для порівняння середніх значень, але він передбачає, що дані мають нормальний розподіл.

F-тест використовується для порівняння дисперсій двох вибірок. Його зазвичай використовують у контексті дисперсійного аналізу, щоб перевірити, чи рівні дисперсії кількох груп даних. F-тест також використовується для перевірки загальної значущості моделі лінійної регресії.

Вибір між цими тестами залежить від конкретних характеристик даних і питання дослідження, яке розглядається.

Дисперсійний аналіз – це статистичний метод, який використовується для перевірки того, чи існують суттєві відмінності між середніми показниками двох або більше груп. Результат дисперсійного аналізу повертає два параметри: оцінка F-test та P-value. Оскільки F-test передбачає, що середні значення для всіх груп однакові, тому оцінка обчислює, наскільки фактичні середні відхиляються від припущення, і повідомляє це як оцінку F-test. Більший отриманий бал означає, що між середніми є більша різниця. Оцінка P-value вказує, наскільки статистично значуще наше обчислене значення. Наприклад, якщо змінна значення P-value сильно корелює зі змінною, яка аналізується, можна очікувати, що дисперсійний аналіз поверне значну оцінку F-тесту та невелике значення P.

Дисперсійний аналіз особливо ефективний при вивченні кількох факторів. Наприклад, в будь-якому експерименті середні значення досліджуваних величин змінюються у зв'язку зі зміною основних факторів

(кількісних та якісних), що визначають умови досліджу, а також і випадкових факторів. Дослідження впливу тих чи інших факторів на мінливість середніх є задачею дисперсійного аналізу. При дисперсійному аналізі кожне спостереження служить для одночасної оцінки всіх факторів та їх взаємодії.

Дисперсійний аналіз дає можливість виділити та оцінити окремі фактори, що викликають зміну досліджуваної випадкової величини. При цьому проводиться розклад сумарної вибіркової дисперсії на складові, обумовлені незалежними факторами. Кожна з цих складових є оцінкою дисперсії генеральної сукупності. Щоб дати оцінку дієвості впливу даного фактору, необхідно оцінити значимість відповідної вибіркової дисперсії у порівнянні з дисперсією відтворення, яка обумовлена випадковими факторами. Перевірка значимості оцінок дисперсії проводиться за допомогою критерію Фішера [10]. Наприклад, якщо розрахункове значення критерію Фішера виявиться меншим за табличне, то немає підстав вважати значимим вплив досліджуваного фактору. Коли ж розрахункове значення критерію Фішера виявиться більшим за табличне, то цей фактор впливає на зміни середніх.

Для описової статистики можна застосувати візуальний підхід, що ілюструє дані за допомогою діаграм, графіків, гістограм та інших графічних образів. Модуль Stats у бібліотеці SciPy дає змогу реалізувати візуальний підхід та зробити кількісний аналіз для визначення загальних чисельних статистичних показників. Модуль містить велику кількість дискретних і неперервних розподілів імовірності та бібліотеку статистичних функцій.

Наприклад, у модулі SciPy.Stats є функції для роботи з випадковими величинами (BV) (рис. 8.1 і 8.2) [14]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

dist = stats.norm(loc=15, scale=1) # BV з нормальним розподілом
X = np.linspace(10,20,100) # область значень BV
Xs=dist.rvs(size=1000) # випадкова вибірка значень BV розміром 1000
print (Xs.mean(), Xs.std(), Xs.var()) # середнє, середньоквадратичне
відхилення, дисперсія
plt.hist(Xs, bins=20, normed=True, color='y') # гістограма вибірки
```

```

plt.plot(X, dist.pdf(X), 'k') # функція густини ймовірності розподілу ВВ
plt.plot(X, dist.cdf(X), 'k--') # функція розподілу ВВ (ймовірність того,
що ВВ буде мати значення менше або рівне x)
print (dist.cdf(15)-dist.cdf(0)) # ймовірність попадання в інтервал
значень (0,15)
plt.xlabel('x'); plt.ylabel('y, Y'); plt.grid(); plt.show()
print ("Рисунок - Гістограма, функція густини ймовірностей y(x) (-) та
функція розподілу Y(x) (--)" )
plt.figure()

Y=np.linspace(0,1,100) # область значень функції розподілу
plt.plot(Y, dist.ppf(Y), 'k') # квантільна функція (інверсна до cdf)
print (dist.ppf(0.5)) # значення ВВ, якому відповідає cdf=0.5
cdf1=stats.norm().cdf(3) # 0.998650101968
cdf2=stats.norm().cdf(-3) # 0.00134989803163
print (cdf1-cdf2) # ймовірність попадання в інтервал (-3*std,+3*std) #
квантілі стандартного нормального розподілу:
print (stats.norm().ppf(cdf1)) # з рівнем cdf1
print (stats.norm().ppf(0.5)) # з рівнем 0.5
plt.xlabel('Y'); plt.ylabel('x'); plt.grid(); plt.show()
print ("Рисунок - Квантільна функція x(Y)" )

```

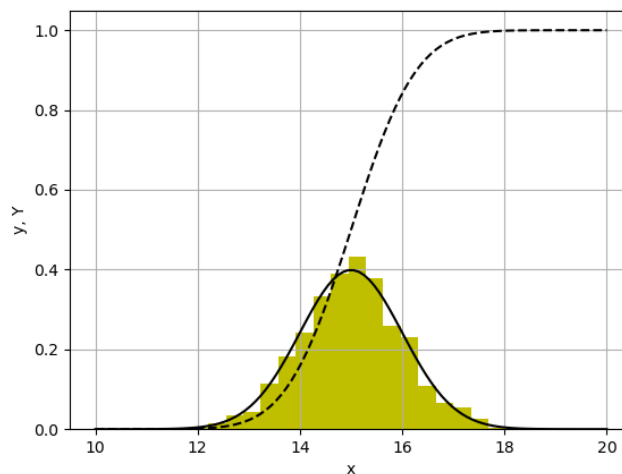


Рисунок 8.1 – Гістограма: (-) функція густини ймовірностей $y(x)$;
 (--) функція розподілу $Y(x)$

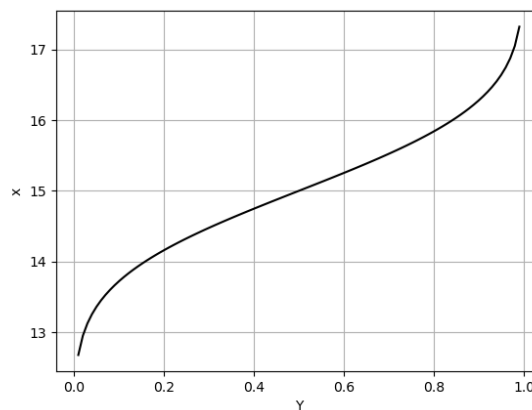


Рисунок 8.2 – Квантільна функція $x(Y)$

Для випадкових величин з нормальним розподілом густини ймовірностей $y(x)$, якому відповідає гістограма на рис. 8.1, отримуємо такі статистичні параметри [14]:

```
# значення BB, якому відповідає cdf=0.5
15.0
# ймовірність попадання в інтервал (-3*std,+3*std)
0.9973002039367398
# квантілі стандартного нормального розподілу:
2.9999999999999982 # з рівнем cdf1
0.0 # з рівнем 0.5
```

Розподіл густини ймовірностей (рис. 8.1) є законом, який описує область значень випадкової величини та ймовірності їхньої появи. Квантільна функція (рис. 8.2) виводить значення випадкової величини так, що її ймовірність менша або дорівнює вхідному значенню ймовірності.

8.2. Порядок роботи

Час операції на відкритому серці в умовах штучного кровообігу становить у середньому 45 хв. ± 10 хв. в залежності від тривалості процесу охолодження та зігрівання пацієнта. Визначимо вірогідність тривалості операції більш ніж 50 хв. та 60 хв. за умови, якщо у 10% випадків затримка охолодження або зігрівання пацієнта становитиме до 5 хв. Припустимо, що тривалість операцій має нормальне розподілення, тоді для функції розподілення густини вірогідності можна застосувати гістограму нормального закону розподілення `stats.norm()` із бібліотеки SciPy:

```
from scipy import stats
...
# prepare data
norm_rv = stats.norm(loc=45, scale=10)
```

У цьому програмному коді параметр `loc=45` визначає середню тривалість операції, а параметр `scale=10` є параметром масштабу, який визначає стандартне відхилення в нормальному розподілі.

Для порівняльного оцінювання сформуємо два статистично пов'язані вектори, які містять дані з середньою тривалістю операції в умовах штучного кровообігу. Вектори з даними `data_1` та `data_2` згенеруємо з використанням методу `randint()` із бібліотеки `Random`:

```
data_test = np.trunc(norm_rv.rvs(512))
data_1 = data_test + [randint(1, 10) for x in range(512)]
data_2 = data_test + [randint(1, 10) for x in range(512)]
```

Визначимо для тестових даних `data_test`, що є основою формування статистично пов'язаних векторів, основні статистичні параметри: середнє значення `mean()`, медіану `median()`, моду `mode()`, стандартне відхилення від середнього значення `std()`, дисперсію елементів масиву `var()`. Зазначені статистичні функції знаходяться у бібліотеці `NumPy`:

```
import numpy as np
...
# statistical parameters
mean = np.mean(data_test)
median = np.median(data_test)
mode = stats.mode(data_test)
std_dev = np.std(data_test)
variance = np.var(data_test)
```

З використанням методів з бібліотеки `Seaborn` побудуємо гістограми нормального розподілення для векторів з даними `data_1` та `data_2`:

```
import seaborn as sns
...
# diagram of data-test plot
fig = plt.figure(1)
sns.histplot(x=data_1, discrete=True);

fig = plt.figure(2)
sns.histplot(x=data_2, discrete=True);
```

Гістограми нормального розподілення для векторів з даними `data_1` та `data_2` мають вигляд (рис. 8.3):

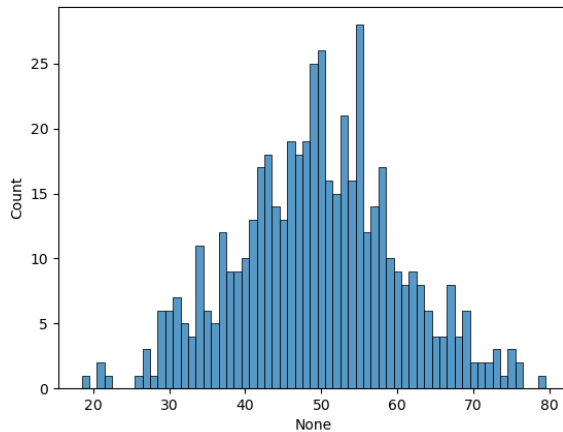
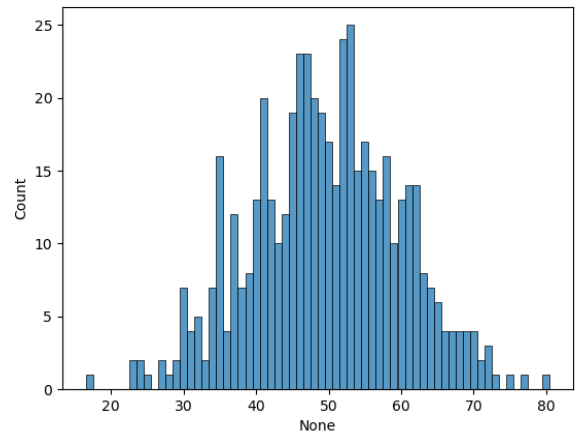
Вектор з даними `data_1`Вектор з даними `data_2`

Рисунок 8.3 – Гістограми нормального розподілення для векторів з даними

Для порівняння середніх значень двох груп даних `data_1` та `data_2` використаємо T-test, який допомагає визначити, чи є значна різниця між групами даних. Цей статистичний тест є одним із найчастіше використовуваних для аналізу даних, його метод `ttest_ind()` знаходяться у бібліотеці SciPy:

```
from scipy import stats
...
# t-test
t_stat, p_value = stats.ttest_ind(data_1, data_2)
print(f"T-statistic: {t_stat}, P-value: {p_value}")
```

Для визначення наскільки значення у вибірці відхиляються від середнього значення застосуємо тест Z-score (або стандартне відхилення). Результат тесту Z-score показує, наскільки далеко чи близько перебуває значення від середнього, що дає змогу стандартизувати дані:

```
# z-score
mean = np.mean(data_test)
std_dev = np.std(data_test)

z_scores = [(x - mean) / std_dev for x in data_1]
anomalies_1 = [x for x in z_scores if abs(x) > 2]
print(f"Anomalies for data 1: {anomalies_1}")
```

```

z_scores = [(x - mean) / std_dev for x in data_2]
anomalies_2 = [x for x in z_scores if abs(x) > 2]
print(f"Anomalies for data 2: {anomalies_2}")

```

У цьому коді обчислюється середнє і стандартне відхилення для вибірки даних `data_1` та `data_2`, а потім розраховуються значення `z_scores` для кожної групи даних, що дозволяє визначити, які значення у вибірках `anomalies_1` та `anomalies_2` є аномальними.

Приналежність вибірки `data_1` і `data_2` до одного класу даних, що описують тривалість однієї і тієї ж операції, можна побачити побудувавши точковий графік:

```

# diagram of scatter plot
fig = plt.figure()
ax = fig.add_axes([0,0,1,1])
ax.scatter(data_test, data_1, color='r')
ax.scatter(data_test, data_2, color='b')
ax.set_xlabel('Grades Range')
ax.set_ylabel('Grades Scored')
ax.set_title('scatter plot')

```

Точковий графік, який використовує функцію `scatter()` з бібліотеки Matplotlib має вигляд (рис. 8.4):

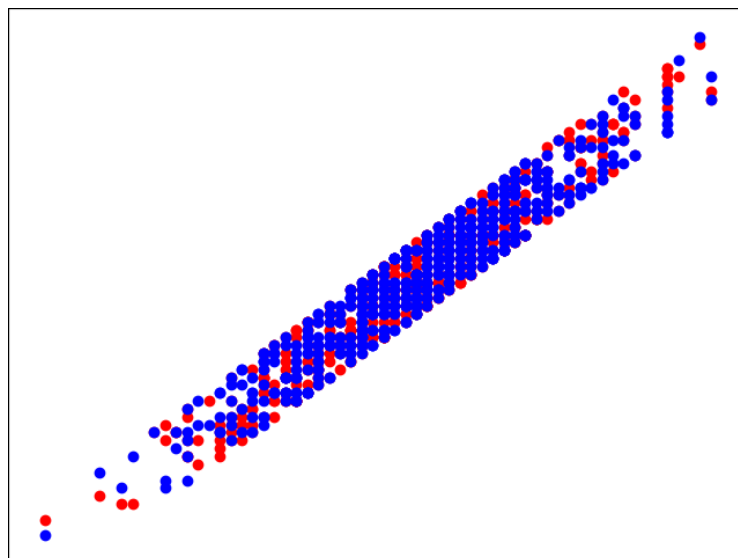


Рисунок 8.4 – Точковий графік приналежності даних до вибірки

Тепер розрахуємо вірогідність тривалості операції за умови, якщо у 10% випадків маємо затримку охолодження або зігрівання пацієнта становитиме до 5 хв. Оскільки усі дані в групах `data_1` та `data_2` мають у розподілі однакову ймовірність, можна вважати, що кожную вибірку описує рівномірний розподіл, а результат лежить між певними межами. Межі результату визначаються параметрами функції `stats.uniform(loc=0, scale=4)` із бібліотеки SciPy:

```
from scipy import stats
...
# probable event
unif_time = stats.uniform(loc=0, scale=5)
event_data = unif_time.rvs(size=3)
print("Probable event:", event_data)
```

У цьому коді обчислюється ймовірність випадку затримки охолодження або зігрівання пацієнта тривалістю від нуля до 5 хв., якщо частота повторення такої події не перевищує `size=3` разів. Якщо поширити таку статистику на `N = 10000` випадків, то можна отримати гістограму з розподілом вірогідності затримки операції:

```
from scipy import stats
...
# probable event
unif_time = stats.uniform(loc=0, scale=5)

N = 10000
event_samples = [unif_time.rvs(size=10).sum() for i in range(10000)]
sns.histplot(x=event_samples, discrete=True)
```

Параметр `size=10` у статистичній функції `unif_time.rvs()` визначає відсоток випадків, за яких можлива затримка на час `scale=5` хв. (рис. 8.5):

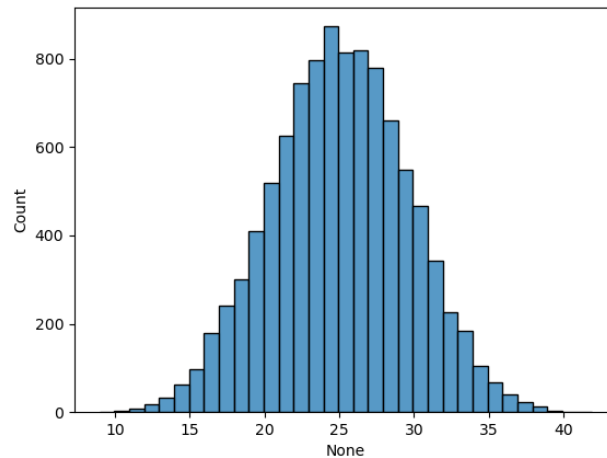


Рисунок 8.5 – Гістограма з розподілом вірогідності затримки операції

Нарешті за допомогою функції `np.histogram()` з бібліотеки NumPy відобразимо гістограму, у якій висота прямокутників показує не кількість входжень кожного значення до вибірки, а ймовірність їх появи у вибірці:

```
import numpy as np
...
N = 10000
np.random.seed(69)
values = np.trunc(norm_rv.rvs(N))

fig, ax = plt.subplots()
v_le_51 = np.histogram(values, np.arange(10, 52))
v_ge_50 = np.histogram(values, np.arange(50, 52))
ax.bar(np.arange(10, 51), v_le_51[0]/N, color='0.8')
ax.bar(np.arange(50, 52), v_ge_50[0]/N)
p = np.sum(v_ge_50[0]/N)
ax.set_title('P(Y>50min) = {:.3}'.format(p))
ax.vlines(50, 0, 0.08, color='k', lw=1);
```

У цьому коді для визначення кількості прямокутників на гістограмі `np.histogram()` використовується генератор псевдовипадкових чисел `np.random.seed(69)`, який має початкове значення для створення випадкового числа. Гістограма показує скільки разів у відсотках затримка операції вийде за межі 50 хв., для чого значення у межах `np.arange(10, 52)` виділяються кольором `color='0.8'`, а інші значення `np.arange(50, 52)` відображаються кольором за замовчанням (рис. 8.6):

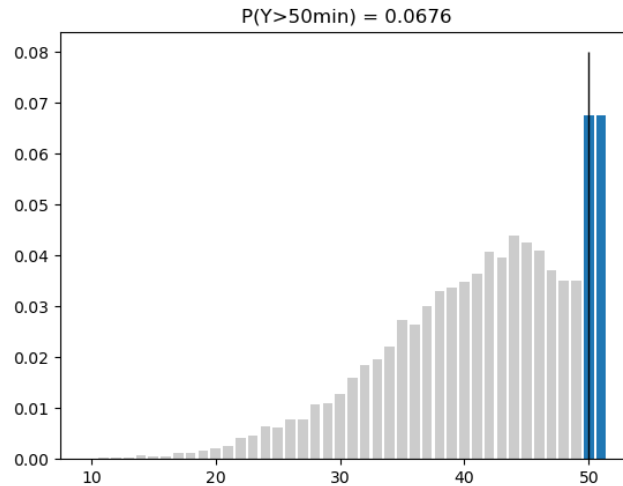


Рисунок 8.6 – Гістограма, що показує скільки разів затримка операції вийде за межі 50 хв.

Відповідно будується гістограма, яка показує скільки разів у відсотках затримка операції вийде за межі 60 хв., для чого значення у межах `pr.arange(10, 62)` виділяються кольором `color='0.8'`, а інші значення `pr.arange(60, 62)` відображаються кольором за замовчанням (рис. 8.7):

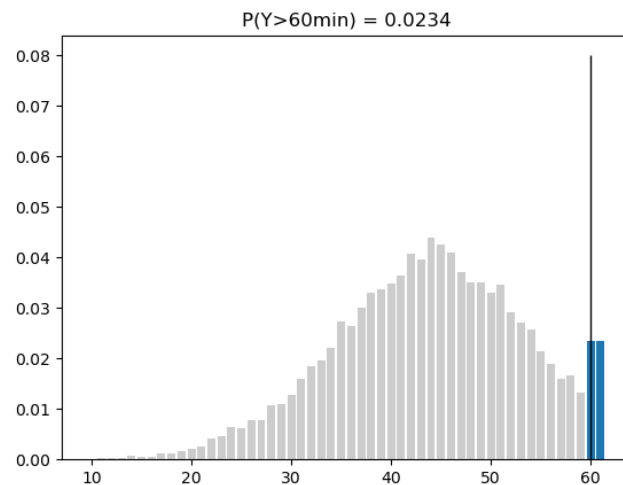


Рисунок 8.7 – Гістограма, що показує скільки разів затримка операції вийде за межі 60 хв.

У підсумку програмний код для визначення вірогідності тривалості операції понад встановлений час, має вигляд:

```

import numpy as np
from numpy.random import seed
from scipy import stats
from random import randint
import matplotlib.pyplot as plt
import seaborn as sns

# prepare data
norm_rv = stats.norm(loc=45, scale=10)
data_test = np.trunc(norm_rv.rvs(512))

data_1 = data_test + [randint(1, 10) for x in range(512)]
data_2 = data_test + [randint(1, 10) for x in range(512)]

mean = np.mean(data_test)
median = np.median(data_test)
mode = stats.mode(data_test)
std_dev = np.std(data_test)
variance = np.var(data_test)

print("Mean:", mean)
print("Median:", median)
print("Mode:", mode.mode[0], "Count:", mode.count[0])
print("Standard Deviation:", std_dev)
print("Variance:", variance)

# diagram of data-test plot
fig = plt.figure(1)
sns.histplot(x=data_1, discrete=True);

fig = plt.figure(2)
sns.histplot(x=data_2, discrete=True);

# t-test
t_stat, p_value = stats.ttest_ind(data_1, data_2)
print(f"T-statistic: {t_stat}, P-value: {p_value}")

# z-score
z_scores = [(x - mean) / std_dev for x in data_1]
anomalies_1 = [x for x in z_scores if abs(x) > 2]
print(f"Anomalies for data 1: {anomalies_1}")

z_scores = [(x - mean) / std_dev for x in data_2]
anomalies_2 = [x for x in z_scores if abs(x) > 2]
print(f"Anomalies for data 2: {anomalies_2}")

# diagram of scatter plot
fig = plt.figure(3)
ax = fig.add_axes([0,0,1,1])
ax.scatter(data_test, data_1, color='r')
ax.scatter(data_test, data_2, color='b')
ax.set_xlabel('Grades Range')
ax.set_ylabel('Grades Scored')
ax.set_title('scatter plot')

# probable event
unif_time = stats.uniform(loc=0, scale=5)
event_data = unif_time.rvs(size=3)
print("Probable event:", event_data)

fig = plt.figure(5)

```

```

N = 10000
event_samples = [unif_time.rvs(size=10).sum() for i in range(10000)]
sns.histplot(x=event_samples, discrete=True)

np.random.seed(69)
values = np.trunc(norm_rv.rvs(N))

fig, ax = plt.subplots()
v_le_51 = np.histogram(values, np.arange(10, 52))
v_ge_50 = np.histogram(values, np.arange(50, 52))
ax.bar(np.arange(10, 51), v_le_51[0]/N, color='0.8')
ax.bar(np.arange(50, 52), v_ge_50[0]/N)
p = np.sum(v_ge_50[0]/N)
ax.set_title('P(Y>50min) = {:.3}'.format(p))
ax.vlines(50, 0, 0.08, color='k', lw=1);

fig, ax = plt.subplots()
v_le_61 = np.histogram(values, np.arange(10, 62))
v_ge_60 = np.histogram(values, np.arange(60, 62))
ax.bar(np.arange(10, 61), v_le_61[0]/N, color='0.8')
ax.bar(np.arange(60, 62), v_ge_60[0]/N)
p = np.sum(v_ge_60[0]/N)
ax.set_title('P(Y>60min) = {:.3}'.format(p))
ax.vlines(60, 0, 0.08, color='k', lw=1);

plt.show()

```

Результат обчислень основних статистичних параметрів такий:

```

Mean: 44.07421875
Median: 44.0
Mode: 39.0 Count: 26
Standard Deviation: 10.205809819761901
Variance: 104.15855407714844
T-statistic: -0.18526293670460992, P-value: 0.853059566202617
Anomalies for data 1: [2.148362710771297, 2.3443295213743442, ...
32.2463461160728206, 2.2463461160728206]
Anomalies for data 2: [2.2463461160728206, 2.0503793054697734, ...
2.5402963319773915, 2.148362710771297, 2.3443295213743442]
Probable event: [3.3929548 2.05995659 1.06231319]

```

8.3. Завдання для самостійної роботи

Реалізувати для визначення вірогідності тривалості операції інші функції густини розподілу: бета `stats.beta()`, гамма `stats.gamma()` і трикутного `stats.triang()` розподілу. Для побудови цих функцій можна використати такі функції з бібліотеки Seaborn:

```

norm_rv = stats.norm(loc=30, scale=5)
beta_rv = stats.beta(a=5, b=5, loc=14, scale=32)
gamma_rv = stats.gamma(a = 20, loc = 7, scale=1.2)
tri_rv = stats.triang(c=0.5, loc=17, scale=26)

```

```

x = np.linspace(10, 50, 300)

sns.lineplot(x = x, y = norm_rv.pdf(x), color='r', label='norm')
sns.lineplot(x = x, y = beta_rv.pdf(x), color='g', label='beta')
sns.lineplot(x = x, y = gamma_rv.pdf(x), color='k', label='gamma')
sns.lineplot(x = x, y = tri_rv.pdf(x), color='b', label='triang')

sns.histplot(x=data_test, discrete=True, stat='probability',
alpha=0.2);

```

Отримати за допомогою бібліотеки Seaborn гістограми густини розподілу для наведених функцій (рис. 8.8):

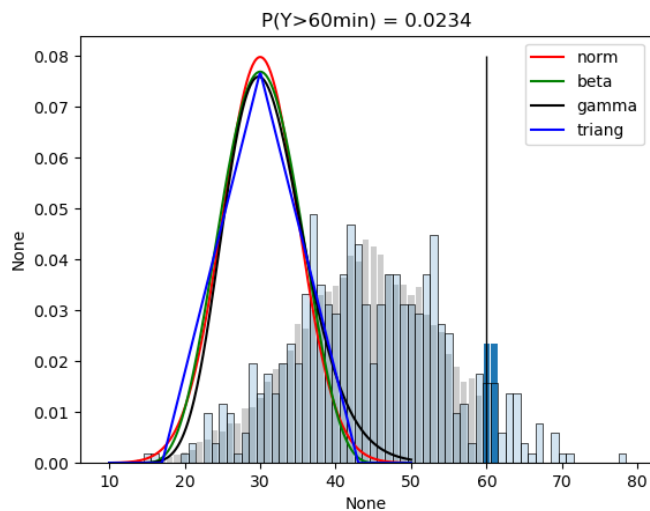


Рисунок 8.8 – Гістограми густини розподілу функцій: нормального norm(), бета beta(), гамма gamma(), трикутного triang() розподілу.

8.4. Контрольні питання

1. Що таке дисперсійний аналіз?
2. Що таке T-test?
3. Що таке Z-test?
4. Що таке F-test?
5. Що таке P-value?
6. Що демонструє гістограми густини розподілу?

ОФОРМЛЕННЯ І ОЦІНЮВАННЯ ПРАКТИЧНИХ РОБІТ

ПРЕДСТАВЛЕННЯ ЗВІТУ З ПРАКТИЧНОЇ РОБОТИ

Політика та контроль: Виконання та захист практичних робіт здійснюється під час самостійної роботи студентів у дистанційному режимі (з можливістю консультування з викладачем через електронну пошту, соціальні мережі). Політику навчальної дисципліни щодо оцінювання практичних робіт викладено у силабусі дисципліни «Програмна інженерія в біомедичних дослідженнях».

9.1. Звіт з практичної роботи

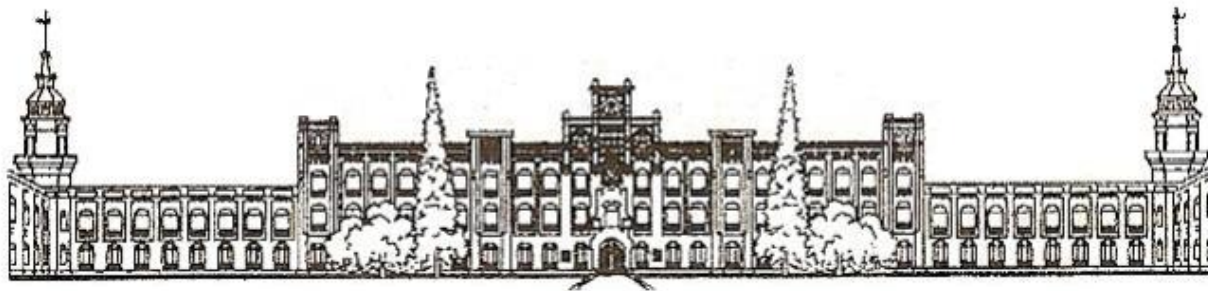
Відвідування практичних занять є бажаним, оскільки на них відбувається написання експрес-контрольних робіт або тестових завдань, а також відбувається захист практичних робіт.

Пропущені контрольні заходи (захист практичних робіт і виконання тестових завдань) обов'язково відпрацьовуються на наступних заняттях за умови виконання завдання, яке заплановано на поточному занятті, або на запланованих консультаціях.

Захист практичних робіт відбувається у формі проходження усної співбесіди з викладачем, за результатами якої студент отримує оцінки, які передбачені видами контролю та рейтинговою системою оцінювання результатів навчання (РСО).

Звіт з практичної роботи має бути представлений викладачу до початку виконання наступної практичної роботи. Звіт має містити наступні складові:

- Прізвище, ім'я, по-батькові здобувача вищої освіти, номер академічної групи.
- Номер практичної роботи.
- Тему та мету практичної роботи.
- Короткий конспект основних теоретичних відомостей.
- Результати виконання самостійної роботи.



МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
КАФЕДРА БІОМЕДИЧНОЇ ІНЖЕНЕРІЇ

ПРАТИЧНА РОБОТА № ____

НАЗВА: _____

ВИКОНАВ СТУДЕНТ ГРУПИ: _____

ПІБ: _____

ПЕРЕВІРИВ: _____

Київ 20__

9.2. Оцінювання практичних робіт

Виконання практичних робіт оцінюється за результатами захисту відповідних звітів. Результати виконання самостійної роботи, отримані студентом під час оформлення звіту, доповідаються у формі проходження усної співбесіди з викладачем.

Результати виконання практичних робіт оцінюються за такою шкалою (у відсотках до максимального балу):

90-100% «відмінно»: виконано всі завдання практичного заняття, здобувач освіти чітко та без помилок відповів на всі контрольні питання.

75-89% «добре»: виконано всі завдання практичного заняття; здобувач освіти відповів на всі контрольні питання із незначними зауваженнями.

60-74% «задовільно»: виконані майже всі завдання практичного заняття з зауваженнями; здобувач освіти відповів на всі контрольні питання із зауваженнями.

35-59% «незадовільно»: здобувач освіти не виконав чи виконав неправильно запропоновані завдання практичної роботи, відповів на контрольні питання з помилками чи не відповів на контрольні питання.

За результатами захисту звітів з практичних робіт студент отримує оцінки, які передбачені рейтинговою системою оцінювання результатів навчання (PCO).

Під час захисту практичних робіт необхідно дотримуватись норм академічної доброчесності та етичної поведінки. Норми етичної поведінки студентів і працівників визначені у розділі 2 Кодексу честі Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського» – <https://kpi.ua/code>.

Студент має право оскаржити результати контрольного заходу згідно затвердженого положення Про апеляції в КПІ імені Ігоря Сікорського (Наказ №НОН/128/2021 від 20.05.2021 р.) – <https://osvita.kpi.ua/index.php/node/182>.

ПЕРЕЛІК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ

1. Мінаков С.М. Інформатика. Лабораторний практикум: навч. посіб. для здобувачів ступеня бакалавра за освіт. програмою «Інжиніринг зварювання, лазерних та споріднених технологій» спец. 131 «Прикладна механіка» / С.М. Мінаков, А.В. Чорний, А.С. Мінаков, І.М. Вдовіченко, Д.М. Вдовіченко; КПІ ім. Ігоря Сікорського. – Київ: КПІ ім. Ігоря Сікорського, 2023. – 313 с. – <https://ela.kpi.ua/bitstreams/2a58b808-f7d1-4f31-879f-bcac040e7df6/download>
2. Івановський О.А, Парненко В.С. Інформатика. Програмування на Python: навч. посіб. для здобувачів ступеня бакалавра за освіт. програмою «Конструювання та дизайн машин» спец. 131 «Прикладна механіка» / О.А. Івановський, В.С. Парненко; Київ: КПІ ім. Ігоря Сікорського, 2023. – 232 с. – <https://ela.kpi.ua/bitstreams/c26afa66-3722-4411-a4e0-f8b7988dbc2d/download>
3. Новотарський М.А. Основи програмування алгоритмічною мовою Python: навч. посіб. для студ. освітньої програми «Комп'ютерні системи та мережі» спеціальності 123 «Комп'ютерна інженерія» / М.А. Новотарський. – Київ: КПІ ім. Ігоря Сікорського, 2022. – 701с. – <https://ela.kpi.ua/bitstreams/969a46b8-e5cb-4513-8b8a-1e6e267e0306/download>
4. Татарчук Д.Д. Об'єктно-орієнтоване програмування мовою Python: Конспект лекцій: навч. посіб. для студ. спеціальності 153 «Мікро- та наносистемна техніка» освітньої програми «Мікро- та наноелектроніка» / КПІ ім. Ігоря Сікорського; уклад.: Д. Д. Татарчук, Ю. В. Діденко. – Київ: КПІ ім. Ігоря Сікорського, 2021. – 129 с. – <https://ela.kpi.ua/bitstreams/672a2e48-69df-42c1-8c2c-e7eeb4e8c8c8/download>
5. Горобець О.Ю. Мова python: для інженерних та наукових задач: підруч. для здобувачів ступеня бакалавра за спеціальністю 162 «Біотехнології та біоінженерія», магістра за спеціальністю за спеціальністю 162 «Біотехнології та біоінженерія», бакалавра за спеціальністю 104 «Фізика та астрономія» та магістра за спеціальністю 104 «Фізика та астрономія» / О. Ю.

Горобець, С. В. Горобець, К. Ю. Хахно ; КПІ ім. Ігоря Сікорського. – 1-ше вид. – Київ: КПІ ім. Ігоря Сікорського, 2024. – 277 с. – <https://ela.kpi.ua/bitstreams/f67fb7d1-d4ac-4317-8007-e1774606ed49/download>

6. Крєневич А.П. Python у прикладах і задачах. Частина 2. Об'єктно-орієнтоване програмування. Навчальний посібник – К.: ВПЦ "Київський Університет", 2020. – 152 с. – <https://www.mechmat.univ.kiev.ua/wp-content/uploads/2021/01/python-u-prykladakh-i-zadachakh.-ch2-oop.pdf>

7. Лавєр В.О., Левчук О.М. Обробка зображень: навч.-метод. посіб. / В.О. Лавєр, О.М. Левчук. – Ужгород: ПП «АУТДОР - ШАРК», 2021. – 51 с. – <https://dspace.uzhnu.edu.ua/jspui/bitstream/lib/35667/1/Обробка%20зображень%202021.pdf>

8. Попов А.Ю. Теорія сигналів. Лабораторний практикум: навч. посіб. для здобувачів ступеня бакалавра зі спеціальності 153 Мікро- та наносистемна техніка / Уклад.: А.О. Попов, К.О. Іванько, Г.С. Порєва; – Київ: КПІ ім. Ігоря Сікорського, 2022. – 121 с. – <https://ela.kpi.ua/bitstreams/46faa968-71d2-4cf8-833e-e2cac33985be/download>

9. Попов А.О. Аналіз та розпізнавання біомедичних сигналів: навчальний посібник для здобувачів ступеня магістра зі спеціальності 153 Мікро- та наносистемна техніка / Уклад.: А.О. Попов, К.О. Іванько; – Київ: КПІ ім. Ігоря Сікорського, 2022. – 67 с. – <https://ela.kpi.ua/bitstreams/2dfcf5cb-cdf0-4748-bce2-2d2851746d75/download>

10. Куєсуль Н.М. Аналіз даних. Лабораторний практикум: навч. посіб. для студ. спеціальності 113 Прикладна математика / Н.М. Куєсуль, А.Ю. Шелєстов, С.А. Тарасєнко, Г.О. Яйлимова; КПІ ім. Ігоря Сікорського. – Київ: КПІ ім. Ігоря Сікорського, 2022. – 28 с. – <https://ela.kpi.ua/bitstreams/213801fb-45e7-4fd2-80ac-ed2b17c41e3a/download>

11. Ковалєнко С.М. Методичні вказівки до лабораторної роботи "Основи роботи з бібліотекою NumPy" з курсу "Обробка даних Python": для студентів спец. 121 Інженерія програмного забезпечення, 122 Комп'ютерні науки, 126 Інформаційні системи і технології / Уклад.: С.М. Ковалєнко, С.В.

Коваленко, О.В. Шматко. – Харків: Нац. техн. ун-т "Харків. політехн. ін-т", 2021. – 48 с. – <https://repository.kpi.kharkov.ua/bitstreams/d1edc454-42cc-4134-a7ed-8dbeb4de4bfd/download>

12. Мисов О.П. Методичні вказівки до практичних занять з дисципліни «Електронні пристрої автоматики» за освітнім рівнем «бакалавр» для студентів спеціальності 151 Автоматизація та комп'ютерно-інтегровані технології / Уклад.: О.П. Мисов, М.О. Савченко. – Дніпро: ДВНЗ УДХТУ, 2019. – 25 с. – <http://citm.ho.ua/Dist/Method/3-4825.pdf>

13. Парфенюк О.І. Технології об'єктно-орієнтованого та Web-програмування: методичні вказівки для здобувачів вищої освіти першого (бакалаврського) рівня зі спеціальності 151 Автоматизація та комп'ютерно-інтегровані технології / Парфенюк О.І., Присяжнюк О.В., Сафоник А.П. – Рівне: НУВГП, 2019. – 91 с. – <http://ep3.nuwm.edu.ua/16687/1/04-03-201%20%281%29.pdf>

14. Копей В.Б. Мова програмування Python для інженерів і науковців: навчальний посібник для виконання курсових і магістерських робіт під час підготовки фахівців першого (бакалаврського) та другого (магістерського) рівнів освіти за спеціальністю 131 Прикладна механіка / В.Б. Копей. – Івано-Франківськ: ІФНТУНГ, 2019. – 272 с. – <https://github.com/vkopey/Python-for-engineers-and-scientists/blob/master/Копей%20ВБ%20Мова%20програмування%20Python%20для%20інженерів%20і%20науковців%202019.pdf>

15. Анісімов А.В. Програмування числових методів мовою Python: підруч. / А.В. Анісімов, А.Ю. Дорошенко, С.Д. Погорілий, Я. Ю. Дорогий, Є.В. Глушко; за ред. А.В. Анісімова. – К.: Видавничо-поліграфічний центр "Київський університет", 2014. – 640 с. – https://ist.kpi.ua/wp-content/uploads/2017/05/Дорогий_Підручник.pdf

16. Дорогий Я.Ю. Програмування числових методів мовою Python: навч. посіб. / А.Ю. Дорошенко, С.Д. Погорілий, Я.Ю. Дорогий, Є.В. Глушко; за ред. А.В. Анісімова. – К.: Видавничо-поліграфічний центр "Київський

університет", 2013. – 463 с. – https://ist.kpi.ua/wp-content/uploads/2017/05/Дорогий_Посібник.pdf

17. Варфоломєєв А.Ю. Методичні вказівки до виконання лабораторних робіт з дисципліни «Системи технічного зору» для студентів спеціальностей 7.05090201, 8.05090201 «Радіоелектронні апарати та засоби» / Уклад.: А.Ю. Варфоломєєв, В.Г. Дзюба. – К.: НТУУ «КПІ», 2015. – 83 с. – <https://ela.kpi.ua/bitstreams/437e3015-ac79-4781-aa92-9a884c1b288a/download>

18. Яковенко А.В. Основи програмування. Python. Частина 1: підручник для студентів які навчаються за спеціальністю 122 «Комп'ютерні науки» спеціалізацією «Інформаційні технології в біології та медицині» / КПІ ім. Ігоря Сікорського; А.В. Яковенко. – Київ: КПІ ім. Ігоря Сікорського, 2018. – 195 с. – <https://ela.kpi.ua/bitstreams/dbbe8ff5-11d7-4a92-918c-d1445c3d20a7/download>

19. Шликов, В. В. Мікропроцесорна техніка. Практикум: навчальний посібник для студентів спеціальності 163 «Біомедична інженерія» та 152 «Метрологія та інформаційно-вимірювальна техніка» / В.В. Шликов; КПІ ім. Ігоря Сікорського. – Київ: КПІ ім. Ігоря Сікорського, 2018. – 145 с. – <https://ela.kpi.ua/bitstreams/b1236a82-061f-45f7-b598-3098b2c592f4/download>

20. Arduino Documentation: Arduino UNO R3 / Product Reference Manual. Hardware. SKU: A000066. – Modified: 17.02.2025. – 26 p. – Downloadable resources: <https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf>

21. Каталог Arduono.UA: датчики; медицина і здоров'я. – Ресурси для завантаження: <https://arduino.ua/cat6-atychiki>

22. Education Website Newbiely.COM: Arduino UNO R4 to Control the DC Motor. – Downloadable resources: <https://newbiely.com/tutorials/arduino-uno-r4/arduino-uno-r4-dc-motor>